
CaP-X: A Framework for Benchmarking and Improving Coding Agents for Robot Manipulation

Letian Fu^{*1,2} Justin Yu^{*2} Karim El-Refai^{*2} Ethan Kou^{*2} Haoru Xue^{*1,2} Huang Huang³ Wenli Xiao⁴
Fei-Fei Li³ Guanya Shi⁴ Jiajun Wu³ Shankar Sastry² Yuke Zhu¹
Ken Goldberg^{†2} Linxi “Jim” Fan^{†1}

Abstract

Code-as-Policy (CaP) is a paradigm in which a language or vision-language model generates executable robot control programs, yet its effectiveness as an autonomous controller for embodied manipulation remains underexplored. Prior CaP systems often rely on high-level, human-designed primitives, making it difficult to separate agent capability from designer-provided scaffolding. We present **CaP-X**, an open-access framework for systematically studying Code-as-Policy agents in robot manipulation. CaP-X includes four components. **CaP-Gym** is an interactive environment in which coding agents control robots by synthesizing and executing programs that compose perception and control primitives. Building on this foundation, **CaP-Bench** evaluates frontier language and vision-language models across varying levels of abstraction, interaction, and perceptual grounding. Across 12 models, the task success rates improve with human-crafted abstractions but degrade as these priors are removed, exposing a dependence on designer scaffolding. At the same time, we observe that scaling test-time computation with multi-turn interaction, structured execution feedback, visual differencing, automatic skill synthesis, and ensemble reasoning can substantially improve robustness even when agents operate over low-level primitives. These findings motivate **CaP-Agent0**, a training-free framework that achieves near human-level reliability on several manipulation tasks in simulation and on real embodiments. **CaP-RL** explores reinforcement learning with verifiable rewards to improve success rates and supports sim-to-real

transfer through a shared code-as-action-space interface. Together, CaP-X provides an open-access platform for advancing embodied coding agents. Project page: <https://capgym.github.io>

1. Introduction

Robots have long been controlled through explicit programs that combine perception, geometry, planning, and feedback (Fikes & Nilsson, 1971; Murray et al., 1994; Aeronautes et al., 1998; Siciliano et al., 2008). As robots advanced to continuous, higher-dimensional spaces, these representations were integrated with geometric motion planning (Khatib, 1986), evolving into Task-and-Motion Planning (TAMP) (Kaelbling & Lozano-Pérez, 2011). These *classical* control paradigms achieve robustness through explicit structure: human engineers manually write software that decomposes high-level goals into subtasks, compose perception and control modules, and handle failure and edge cases through trial-and-error and explicit logic. While this can offer strong interpretability and geometric precision guarantees, it relies heavily on human skill and expertise. The manual process can be very time-consuming and often produces task-specific solutions that are difficult to generalize to open-ended environments.

Driven by successes in foundation models (Devlin et al., 2018; Radford et al., 2018; 2019; Brown et al., 2020; Chowdhery et al., 2023; Achiam et al., 2023; Radford et al., 2021; Li et al., 2023), another robot control paradigm has emerged in the form of Vision-Language-Action (VLA) models (Brohan et al., 2023; Kim et al., 2024; Octo Model Team et al., 2024; Jang et al., 2022; Jiang et al., 2023; Reed et al., 2022; Open X-Embodiment Collaboration et al., 2024; Shah et al., 2023; Fu et al., 2024; Huang et al., 2025; Bjorck et al., 2025; TRI LBM Team et al., 2025; Physical Intelligence et al., 2025). These approaches learn from large-scale visuomotor datasets to achieve impressive performance on contact-rich tasks such as shirt folding and whole-body loco-manipulation. However, VLAs inherit the limitations of their training data and design: they lack interpretability

^{*}Equal contribution [†]Equal advising ¹NVIDIA ²UC Berkeley ³Stanford University ⁴Carnegie Mellon University. Correspondence to: Letian Fu <max.fu.letian@berkeley.edu>.

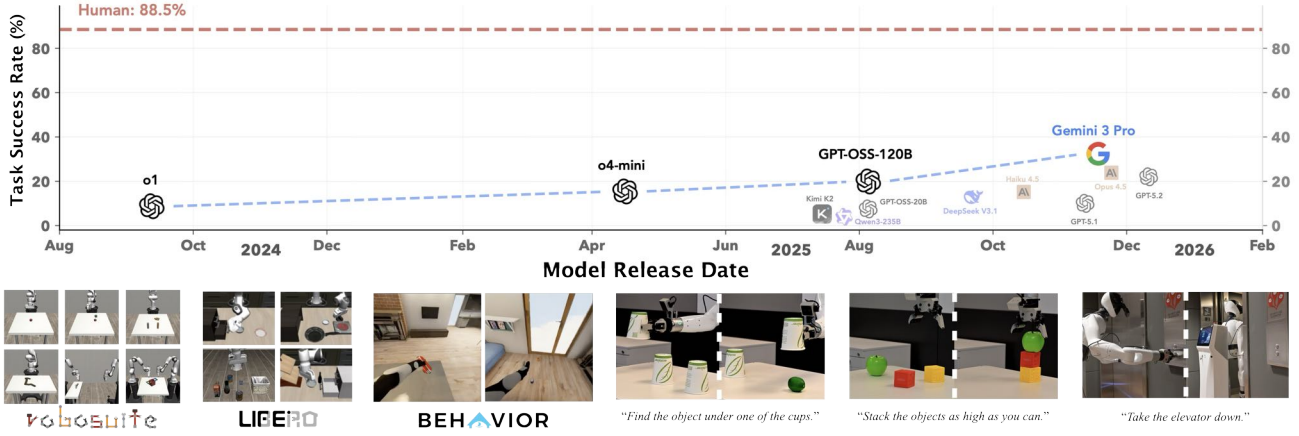


Figure 1. (Top) CaP-Bench Task Success Rate over Model Release Date: across 7 tasks and 12 models, we compare the success rates of model-generated robot-control programs against those of human expert-written programs. We find that while (vision-) language models have achieved capabilities comparable to humans in other domains (Jimenez et al., 2024; Rein et al., 2024; Hendrycks et al., 2020), they still trail behind human performance in writing code that controls robots for manipulation tasks. (Bottom): CaP-Gym integrates Robosuite (Zhu et al., 2020), LIBERO-PRO (Zhou et al., 2025), and BEHAVIOR (Li et al., 2024). We further present CaP-Agent0 (Section 4), a training-free agentic framework that recovers near human-level performance on several manipulation tasks and achieves success rates comparable to—and in some cases exceeding—those of post-trained VLAs, without any task-specific training data.

and struggle to generalize to changes in the environment, new robot embodiments, and long-horizon tasks without additional data collection and retraining.

Recent advances in coding capability of Large Language Models (LLMs) suggest a way to bridge these paradigms: using coding agents to replace the human engineer. Modern coding agents have demonstrated the ability to synthesize executable code, define functions, and debug failures on software engineering benchmarks (Jimenez et al., 2024). Unlike earlier language-conditioned planners limited to function calling (Tellex et al., 2011), today’s agents can construct mid- to low-level logic that closely resembles expert code.

Code-as-Policy (CaP) pioneers (Liang et al., 2023; Singh et al., 2023) explored this approach in applications to robotics; they used human-tuned high-level primitives (e.g., `stack_objs_in_order()`) that offer significant task-specific simplifications. As a result, it remains unclear how much of the observed performance in robot control stems from the agent itself versus the structure imposed by these primitives. In particular, prior work does not systematically characterize how agent performance changes as such scaffolding is reduced, nor to what extent increased test-time computation—through iterative debugging, skill synthesis, ensembled reasoning, or multimodal grounding—can compensate for operating over lower-level interfaces.

To address this gap, we introduce **CaP-X**, a unified framework for systematically evaluating and improving code-based robot control agents. At its core is **CaP-Gym**, an interactive environment in which agents directly control robots by generating and executing programs that compose perception and control primitives. CaP-Gym integrates 187 tasks from standard robot manipulation simulators (Robo-

suite (Zhu et al., 2020), LIBERO-PRO (Liu et al., 2023b; Zhou et al., 2025), and BEHAVIOR (Li et al., 2024)) under a shared primitive design that is intentionally compatible with both simulation and physical robot systems.

Building on CaP-Gym, we construct **CaP-Bench**, a benchmark designed to systematically study agentic capability along three axes: **Abstraction Level**: Varying the action space from human-crafted macros (High-Level) to atomic, fundamental primitives (Low-Level); **Temporal Interaction**: Comparing zero-shot single-turn program generation against multi-turn interaction to quantify capabilities in failure recovery and iterative reasoning; and **Perceptual Grounding**: Evaluating how different modalities of visual feedback impact the agent’s ability to ground task-relevant visual features into code generation. We instantiate CaP-Bench by focusing on a subset of 7 environments from CaP-Gym and evaluate 12 state-of-the-art open- and closed-source language models and vision-language models.

Guided by insights from CaP-Bench results, we derive **CaP-Agent0**, a training-free agentic framework that augments coding models with multi-turn interaction, visual grounding into text, an automatically synthesized task-agnostic skill library, and parallelized multi-model code generation. CaP-Agent0 achieves performance comparable to, and in some cases exceeding, human expert baselines on CaP-Bench tasks. Finally, we show that CaP-Gym supports **CaP-RL**—reinforcement learning on the coding agent itself. On-policy post-training with environment rewards improves task success, while synthesized programs transfer directly to real robots. This paper makes the following contributions:

1. CaP-Gym, a unified suite of interactive robot coding

Table 1. CaP-Bench evaluation tiers. Each column specifies a tier by how the agent accesses environment state, primitive abstraction, in-context primitive usage examples, and visual-grounding modality. S1–S4 are single-turn; M1–M4 allows multi-turn interaction.

Category	Characteristic	Single-Turn				Multi-Turn			
		S1	S2	S3	S4	M1	M2	M3	M4
Perception	Noiseless (State-Based)	✓							
	Noisy		✓	✓	✓	✓	✓	✓	✓
Primitive Abstraction	High-level	✓	✓			✓	✓	✓	
	Low-level			✓	✓				✓
In-Context Learning	Primitive Usage Examples			✓					✓
Visual-Grounding Modality	Multimodal Feedback						✓		
	Visual Diff. Module (VDM)							✓	✓

environments spanning tabletop, bimanual, and mobile manipulation tasks, designed for evaluating and training code-generating multimodal embodied agents.

2. CaP-Bench, a systematic benchmark that measures robot control performance across tasks and levels of primitives and modalities.
3. CaP-Agent0, a training-free, agentic harness combining multi-turn visual differencing, ensembled reasoning, and automatic skill library synthesis.
4. CaP-RL, reinforcement learning on the coding agent via environment reward.

2. CaP-Gym

CaP-Gym is a hierarchical control framework built on the standard **Gymnasium** interface (Brockman et al., 2016). It binds a Low-Level Environment loop (a physics simulator or the real world) with a stateful Code Executor loop. Architecturally, CaP-Gym preserves the native dynamics of underlying simulators, e.g., Robosuite (Zhu et al., 2020), LIBERO-PRO (Liu et al., 2023b; Zhou et al., 2025), and BEHAVIOR (Li et al., 2024), while exposing them through a Read-Eval-Print Loop (REPL) paradigm tailored for coding agents. In CaP-Gym, a code environment “turn” corresponds to one interaction between the coding agent and a specific robot task instance: the agent receives observations, generates a Python program, and the environment executes it to completion. The program may invoke multiple perception and control primitives, each of which can run the simulator or robot controller for multiple internal updates.

2.1. Low-Level Perception and Control Primitives

All computationally intensive perception and control primitives are implemented as stateless services (Christie et al., 2025), enabling high-throughput parallel evaluation.

Perception Primitives Agents access perceptual data from the environment through modular perception primitives that abstract raw sensor data into structured semantic objects,

e.g., SAM3 (Carion et al., 2025) for language-conditioned segmentation and Molmo 2 (Clark et al., 2026) for open-vocabulary pointing, alongside standard vision libraries like OpenCV (Bradski, 2000) and Open3D (Zhou et al., 2018).

Control Primitives Instead of directly emitting joint-space action commands, agents call motion planners or inverse kinematics solvers such as PyRoki (Kim et al., 2025). They can handle collision checking, reachability constraints, and action-space transformations, allowing agents to reason in a task-oriented Cartesian space while delegating execution feasibility to the controller.

3. CaP-Bench: Evaluating Frontier Models

Models. We evaluate 12 state-of-the-art vision-language and language models, including closed-source frontier models (Gemini-3-Pro (Google DeepMind, 2025), OpenAI GPT o1 (OpenAI, 2024), o4-mini (OpenAI, 2025d), 5.1 (OpenAI, 2025a) and 5.2 (OpenAI, 2025b), and Claude Haiku 4.5 (Anthropic, 2025a) and Opus 4.5 (Anthropic, 2025b), and open source models (OpenAI GPT-OSS-20B and 120B (OpenAI, 2025c), Qwen3 235B (QwenLM / Alibaba Cloud, 2025), Qwen-2.5-Coder-7B-Instruct, Kimi K2 Instruct (AI, 2025), and DeepSeek-V3.1-Terminus (DeepSeek-AI, 2024)).

Simulation Task Suite. Primary analysis is performed across 7 core tasks ranging from single-arm manipulation to bimanual coordination: *Cube Lift*, *Cube Stack*, *Spill Wipe*, *Peg Insertion*, *Cube Re-stack*, *Two-Arm Lift*, and *Two-Arm Handover*. Each task is evaluated with 100 trials per tier, where each tier specifies the available primitives, interaction mode, and feedback/grounding signal. These 7 tasks are an intentionally controlled core for ablating abstraction, iteration, and grounding under matched conditions; the full release of CaP-Gym ships 187 tasks (7 Robosuite + 130 LIBERO-PRO + 50 BEHAVIOR) for broader community evaluation. In Section 4, we further extend this analysis to diverse long-horizon settings using tasks from LIBERO-PRO (Zhou et al., 2025) and BEHAVIOR (Li et al., 2024).

Protocol. We evaluate models using **Zero-Shot**

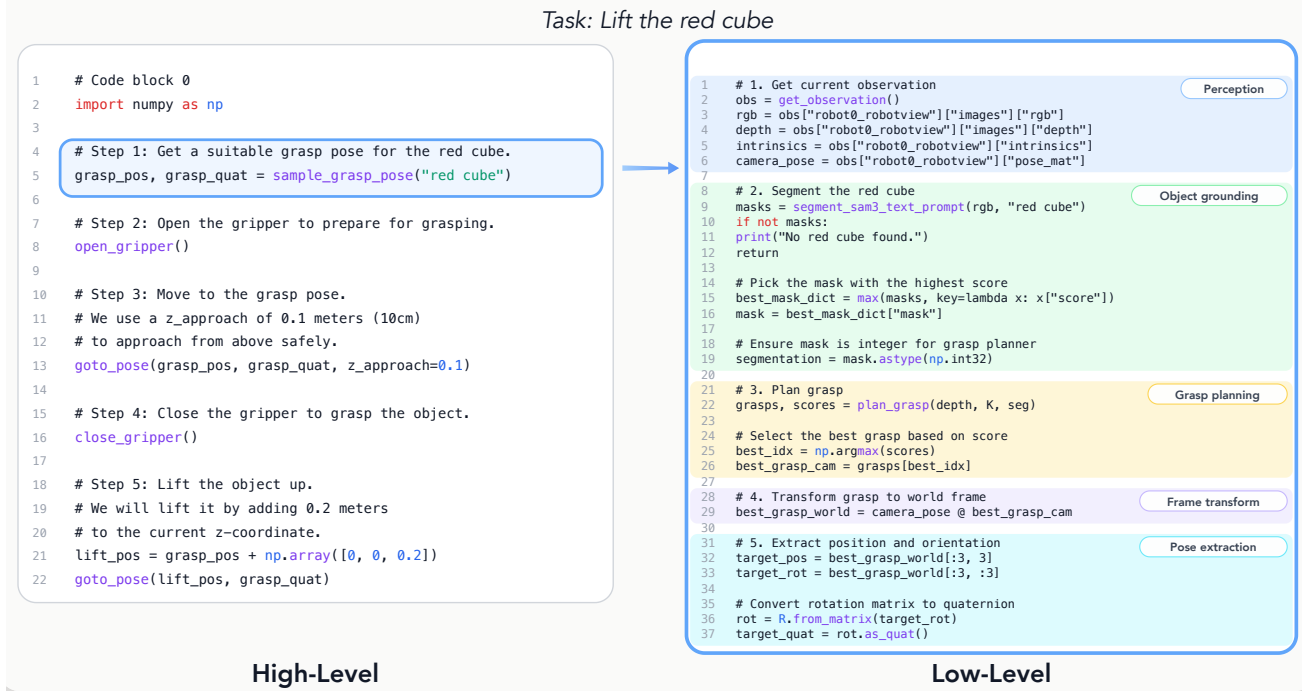


Figure 2. (Left) An example of code generated by Gemini-3-Pro for completing the task “lift the red cube” using the high-level primitives. (Right) Code generated by Gemini-3-Pro using just low-level primitives to achieve the same functionality as a single high-level primitive. For more details on the differences between high and low-level primitives, see Section 3.1 and Appendix G.

Pass@1 (Chen et al., 2021; Jimenez et al., 2024). Within a trial, agents may interact with the environment over one or multiple turns, observing execution feedback and generating subsequent code to recover from errors or extract additional information; the environment is never reset during the trial. Coding agent task performance is compared against that of human expert-written reference solutions under identical environments and primitives (see Appendix K.1). We introduce 4 single-turn tiers (S1-S4) and 4 multi-turn tiers (M1-M4) to CaP-Bench. Refer to Table 1 for a tabular comparison. The tiers vary three axes faced by any code-as-policy practitioner: (1) *primitive abstraction* (human macros vs. low-level primitives), (2) *level-of-iteration* (single-turn vs. multi-turn with structured execution feedback), and (3) *mode of grounding* (raw visual inputs vs. text descriptions from a separate VLM). CaP-Bench isolates these axes via independently controllable tiers.

3.1. Single-Turn Benchmarks (S1-S4)

High-Level (S1 & S2): These tiers evaluate the agent’s ability to reason with human-designed primitives. We evaluate this in two modes: **Privileged (S1)**, which uses ground-truth simulation state (masks and object poses), and **Non-Privileged (S2)**, which relies on real perception modules processing raw RGB-D inputs—the default setting for most prior work. We introduce S1 to disentangle high-level planning from perception noise, establishing a reasoning upper bound that allows us to distinguish between algorithmic

failures and visual estimation errors.

Low-Level (S3 & S4): In these tiers, human-designed abstractions are replaced by their constituent low-level primitives (e.g., `solve_ik()`, `sam3_text_prompt()`), drawn directly from the APIs of each underlying package—reflecting the interfaces that human developers use to control robots. We evaluate two settings: (S3), where documentation includes usage examples to scaffold low-level composition, and (S4), where examples are removed and the agent must reason about program structure solely from interface definitions (function signatures and docstrings). See Figure 2 for an illustration of high-level (S1 & S2) versus low-level (S3 & S4) primitives; full API details at each abstraction level are provided in Appendix G.

3.2. Multi-Turn Benchmarks (M1-M4)

Text-Only Multi-turn (M1): In this setting, the agent receives the standard output (`stdout`) and error traces (`stderr`) from the Python sandbox after each execution turn. This enables a state introspection loop: agents can proactively inject diagnostic print statements to surface hidden symbolic variables (e.g., perception estimates) and utilize these traces to diagnose logical failures and refine code without access to visual ground truth. All other multiturn tiers (M2-M4) retain access to these code execution traces.

Multimodal (M2): The environment pipes the current RGB observation back into the agent’s context window. The M2

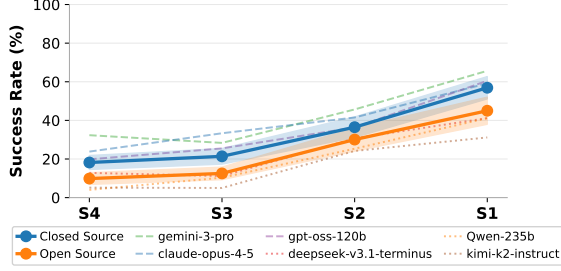


Figure 3. Average task success rate across open-source and closed-source models as primitive abstraction increases. As primitive abstraction increases S4 (per model performance illustrated in Figure 1) to S1 (high level primitives with full state observation), the success rate increases. This can only in part be attributed to reduced code correctness at S3-S4, see Figure 4.

tier is only available to multimodal foundation models that accept raw RGB images as input.

Visual Differencing Module (M3): We introduce the Visual Differencing Module (VDM), which uses a vision-language model to convert visual observations into structured natural language. VDM is provided with the task instruction alongside visual observations. In the first turn, it generates a scene description and extracts task-relevant visual attributes. In subsequent turns, it explicitly describes differences between the previous and current image observations and whether the coding agent has completed the task. The resulting text from the VDM is provided as part of the coding agent’s observation context for code generation.

Low-Level with VDM (M4): This tier has the same VDM as tier M3 and has access to the same low-level primitives and in-context usage examples as tier S3.

3.3. Discussion

Takeaway 1: A Significant Gap Persists Between Frontier Models and Human Experts in Single-Turn Evaluation. We benchmark open- and closed-source agents against human expert solutions under an identical set of perception and control primitives in a single-turn, zero-shot setting (S4). The human reference is a near-upper-bound: $N=7$ paper authors (each with 2+ years of robotics-programming experience) wrote single Python scripts at each tier using exactly the same API primitives available to the model, iterating and debugging until the solution achieved 88.5% average single-turn success (see Appendix K.1 for the protocol and effort budget). As shown in Figure 1, while closed-source models consistently outperform open-source alternatives and newer architectures exhibit stronger capabilities, none yet match the success rate of human-crafted programs in a zero-shot Pass@1 setting.

Takeaway 2: High-Level Abstractions Boost Performance but Limit Expressivity. Figure 3 shows a monotonic increase in task success as primitive abstraction in-

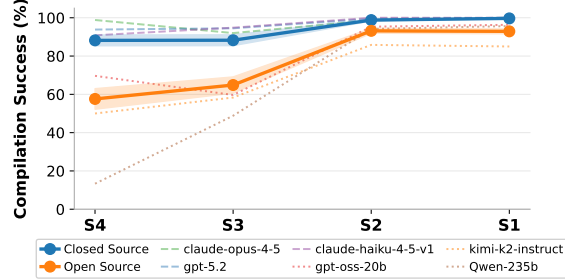


Figure 4. Code Execution Success Rate vs primitive abstraction. Increases, mirroring how prior Code-as-Policies (Liang et al., 2023) approaches relying on high-level primitives report strong zero-shot performance. By collapsing low-level perception, geometric reasoning, and control into human-designed primitives, these abstractions reduce the effective search space and allow models to focus on task sequencing.

However, this gain comes at the cost of expressivity. As abstraction increases, the agent’s action space is increasingly constrained by human priors, imposing a generality ceiling that masks failures in low-level reasoning. In contrast, performance degradation at lower abstraction levels (S3/S4) reflects the difficulty of code synthesis, while enabling expressive behaviors, such as hierarchical perception fallback strategies (Appendix F.2.2), that cannot be represented by fixed high-level primitives.

This observation motivates a scalable middle ground: rather than relying on human-designed abstractions, agents should be able to recover structure from low-level primitives themselves. In Section 4, we demonstrate this capability by enabling agents to distill successful execution traces into a reusable skill library. Consequently, we propose that generalist embodied coding agents be evaluated primarily on primitive-level performance, ensuring that success stems from robust reasoning rather than from the inductive biases of an over-engineered, often task-specific, API set.

Takeaway 3: Closing the Loop with Multi-turn and Visual Grounding Improves Performance. We study how *multi-turn interaction* and different forms of *visual grounding* mitigate the performance gaps identified in Takeaways 1 and 2. Allowing agents to iterate and inspect their own execution traces (`stdout/stderr`, M1) consistently improves performance across all models (Figure 5), highlighting the importance of explicit execution feedback for debugging and recovery.

Counter-intuitively, directly interleaving raw RGB observations at each turn (M2) degrades performance relative to the text-only M1 baseline. We hypothesize this degradation is due to a cross-modal alignment gap: foundation models are rarely trained to jointly reason over software coding and images of physical task execution, making raw visual inputs difficult to integrate effectively during code synthesis.

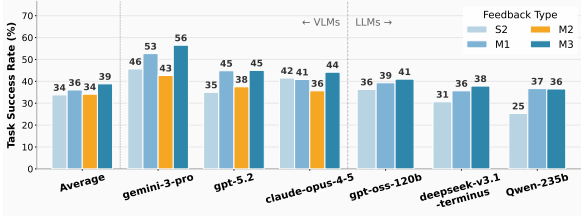


Figure 5. Comparison of single-turn (S2) and multi-turn tiers (M1-M3) across models. Enabling textual feedback through multi-turn (M1) improves task success rate in most models. Direct multimodal (M2) visual grounding reduced task success rate. We find that visual differencing into text (M3) instead of direct multimodal visual input consistently improves task success rate across both close and open source models.

Prior work (Hu et al., 2025; Wang et al., 2026) similarly observes that textually grounded feedback outperforms raw images, though primarily in structured environments with native text states. In contrast, robotic manipulation operates in unstructured, continuous settings without ground-truth textual descriptions. Here, the Visual Differencing Module (M3) bridges this gap by converting visual observations into structured natural language, substantially outperforming both naive image interleaving (M2) and execution-only feedback (M1) across all tasks (Figure 5).

Low-Level Primitives with Multi-turn Feedback (M4). Figure 6 shows that agents operating over low-level primitives augmented with multi-turn feedback not only surpass high-level single-turn (S2) but can reach parity with high-level multi-turn performance (M3). This supports a *test-time compute scaling* hypothesis: robustness can be synthesized at runtime by increasing an agent’s capacity for reasoning, verification, and self-correction over atomic primitives.

Across all multi-turn settings (M1-M4), both coding errors (e.g., exceptions) and physical execution failures (e.g., unstable grasps) are frequently recoverable through iterative interaction. In the absence of explicit visual grounding mechanisms (M1), successful agents compensate by proactively instrumenting perception primitives to expose symbolic state—such as object poses—and performing explicit checks for task completion (e.g., verifying relative object heights to confirm stacking). Visual grounding (M2-M4) reduces the burden of such self-instrumentation but does not eliminate explicit state verification behavior through perception primitives. Instead, the strongest performance emerges when agents combine structured feedback with iterative reasoning, framing multi-turn interaction as a mechanism for hypothesis testing and error recovery in embodied control.

4. CaP-Agent0: An Agentic Framework for Robot Control

Based on the failure modes and insights identified in CaP-Bench, we introduce **CaP-Agent0**, a training-free, agentic

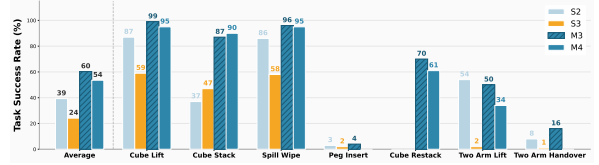


Figure 6. Evaluation of Gemini-3-Pro across four tiers of the benchmark. Multi-turn with visual differencing and low-level API (M4) significantly outperforms both single-turn low-level API (S3), and more notably, single-turn high-level API (S2).

infrastructure for robot control that augments base foundation models with a specialized multi-turn reasoning loop and a dynamically synthesized skill library. The architecture of CaP-Agent0 is designed directly to address three key gaps identified in the benchmark. We demonstrate the efficacy of each design choice in CaP-Agent0 by running ablation studies using the most capable model identified from CaP-Bench (Gemini-3-Pro) bootstrapped with each design choice, shown in Figure 8. A visual walkthrough of the agentic framework is presented in Figure 7.

1. Multi-turn Visual Differencing (VDM): Directly adopting the insights from Takeaway 3, CaP-Agent0 integrates the Visual Differencing Module as part of the per-turn observation. By grounding observations in structured text rather than raw pixels, the agent mitigates the specific cross-modal alignment failures identified in the M2 tier.

2. Auto-Synthesized and Persistent Skill Libraries: In analyzing low-level S3 and S4 code generations from CaP-Bench, we found that capable models routinely synthesize helper functions to perform robotics data manipulations.

Motivated by agentic systems that accumulate reusable tools over time (Wang et al., 2023a), **CaP-Agent0** introduces an *automatically synthesized, task-agnostic skill library* that persists across trials. Rather than requiring the agent to repeatedly re-derive low-level utilities, the library onboards commonly recurring implementation patterns and offloads fragile low-level logic, allowing the coding agent to focus on high-level semantic planning. Importantly, unlike fixed human-designed high-level APIs, these skills are *discovered*: they emerge from successful executions and retain the expressivity of low-level interfaces while improving robustness through reuse.

The library is constructed via an automated synthesis pipeline that can be executed by the agent itself. Specifically, we collect all successful S3-tier rollouts pooled across all 12 models and 7 Robosuite tasks (i.e., the library is not model-specific), extract function definitions via regular-expression matching, and prompt Gemini-3-Pro to identify frequently recurring, task-agnostic logic. This yields a compact library of 9 verified, task-agnostic primitives (Appendix H.1). While the current implementation performs a single synthesis pass, the process is inherently *iterative*. As additional successful executions are accumulated, the agent can con-

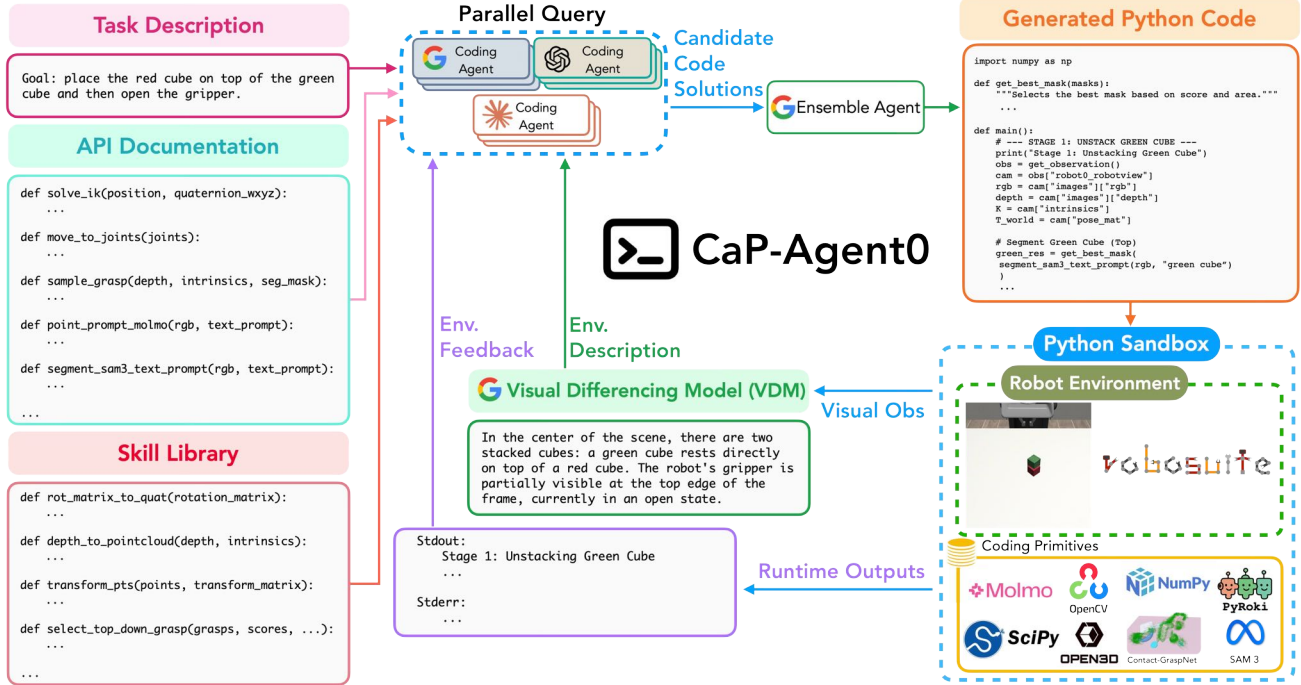


Figure 7. CaP-Agent0. CaP-Agent0 incorporates an auto-synthesized skill library of auxiliary utility generated by coding agents during CaP-Bench, a visual differencing model (VDM) which provides a textual description of the initial scene and for each subsequent turn what changes have occurred in the scene since, and a parallel reasoning system where multiple coding agents are provided the same prompt. These coding agents then generate candidate code solutions that may solve the task and the ensemble agent must then synthesize these code generations into a final code snippet which is then executed in the Python sandbox containing the robot environment, whether that environment may be a simulator like Robosuite or on a real robot. For more details see Section 4.

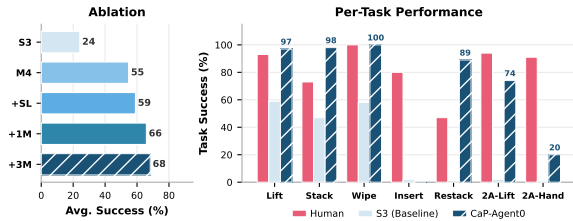


Figure 8. Ablation for CaP-Agent0. (Left): Combining VDM (M4), skill library (+SL), and parallel queries (+1M: Gemini-3-Pro, +3M: Gemini-3-Pro, GPT-5.2, and Claude Opus) significantly improves over single-turn setup with low-level API. (Right): On 4 out of 7 CaP-Bench tasks, CaP-Agent0 achieves comparable or better success rates than human expert code in a single-turn setting. *tinue* to update its skill library over time. A complete list of the nine synthesized functions currently utilized by CaP-Agent0 is provided in Appendix H.1.

3. Parallel Reasoning: Results from tiers S2 and S3 indicate that failures often stem from insufficient test-time exploration rather than a lack of capability. To address this, CaP-Agent0 employs a parallel reasoning strategy inspired by recent ensemble methods (Pan et al., 2025; Jin et al., 2025; Rodionov et al., 2025). At each turn, the system concurrently samples candidate solutions via two configurations: Single-model: 9 queries to one model. Multi-model: 3 queries each to GPT-5.2, Claude Opus 4.5, and Gemini-3-Pro. To maximize output diversity, we vary sampling

temperatures (see Appendix H.2). A central coding agent then synthesizes these candidates into a final code snippet. This parallel approach is applied to both code generation and to the decision-making process for multi-turn continuations.

4.1. CaP-Agent0 Performance on CaP-Bench

Evaluated over 100 trials per task, CaP-Agent0 significantly outperforms single-turn baselines by integrating visual differencing, a self-synthesized skill library, and parallel reasoning (Figure 8). Despite operating solely on low-level primitives, the system achieves success rates comparable to or exceeding human-written programs on 4 out of 7 tasks, narrowing the gap toward expert-level performance.

4.2. CaP-Bench++

The 7 core tasks of CaP-Bench isolate abstraction, iteration, and grounding under matched conditions; CaP-Bench++ extends this to compare coding agents directly against (i) state-of-the-art VLA policies and (ii) iterated human-written code on a broader task distribution. Rather than running all 12 models across every extended task—prohibitively expensive at this scale—we focus the comparison on CaP-Agent0. We further evaluate the performance of CaP-Agent0 on subsets of tasks from LIBERO-PRO and BEHAVIOR. We summarize the results of CaP-Agent0 on 30 manipulation tasks

Table 2. LIBERO-PRO (Zhou et al., 2025) performance of OpenVLA (Kim et al., 2024), π_0 (Black et al., 2024), $\pi_{0.5}$ (Physical Intelligence et al., 2025) and CaP-Agent0 on the **libero-object**, **libero-goal**, and **libero-spatial** benchmarks under initial position perturbations (Pos) and instruction perturbations (Task) averaged across tasks. Detailed individual task-wise performance can be found in Appendix J.

Method	libero-object		libero-goal		libero-spatial	
	Pos (Avg.)	Task (Avg.)	Pos (Avg.)	Task (Avg.)	Pos (Avg.)	Task (Avg.)
OpenVLA	0.00	0.00	0.00	0.00	0.00	0.00
π_0	0.00	0.00	0.00	0.00	0.00	0.00
$\pi_{0.5}$	0.17	0.01	0.38	0.00	0.20	0.01
CaP-Agent0	0.22	0.18	0.26	0.17	0.12	0.14

Table 3. Results on BEHAVIOR (Li et al., 2024) Tasks

Task	Nav. Success Rate			Task. Success Rate		
	Human	S3	CaP-Agent0	Human	S3	CaP-Agent0
Pick up Radio	88%	72%	80%	36%	24%	56%
Pick up Soda Can	80%	52%	84%	72%	32%	72%

from LIBERO-PRO in Table 2 and compare them against three state-of-the-art VLA methods: OpenVLA (Kim et al., 2024), π_0 (Black et al., 2024), and $\pi_{0.5}$ (Physical Intelligence et al., 2025). Although these VLA methods are training-based, CaP-Agent0 is a training-free method that has comparable or exceeds the performance of VLA post-training on these tasks. LIBERO-PRO (Zhou et al., 2025) extends the LIBERO (Liu et al., 2023b) benchmark by increasing the perturbations of tasks by reasonable amounts along object attribute perturbations, initial position perturbations, instruction perturbations, and environmental perturbations. CaP-Agent0 and these VLA methods are evaluated along initial position perturbations (Pos) and instruction perturbations (Task). Under Pos perturbations, the initial positions of objects in the scene are swapped with one another (ex: the position of the frypan and moka pot) and under Task perturbations, the instruction is changed so that another object in the scene is manipulated (ex: “Put the moka pot on the stove” $\rightarrow$ “Put the frypan on the stove”). Since VLAs are trained on a different instruction distribution, they perform poorly under task perturbations, whereas CaP-Agent0 remains robust to instruction variations. Furthermore, CaP-Agent0 achieves performance comparable to $\pi_{0.5}$ under initial position perturbations.

We evaluate our method on two long-horizon mobile manipulation tasks in BEHAVIOR, where an R1Pro wheel-based humanoid is required to pick up a radio from a table and a soda can from the floor. The results over 25 trials for each task are summarized in Table 3, reporting both navigation success rates and task completion success rates. In both tasks, the robot may start with the target object outside its field of view and must actively search for and navigate toward it. Navigation is considered successful if the robot reaches a location within 1 m of the target.

The robot can adjust its camera in both horizontal and ver-

tical directions and move both its base and arm to reach the object, resulting in a substantially larger action space than in tabletop manipulation settings. Moreover, potential collisions with surrounding furniture may prevent the robot from reaching the desired pose, leading to partial execution of planned trajectories and increased task complexity.

In the radio pickup task, although the robot often successfully locates and approaches the object, it may lose sight of it or encounter severe occlusions when navigating too closely, due to its limited field of view. This frequently results in missing or poor grasp poses and constitutes a major failure mode for both S3 and the human policy. In contrast, CaP-Agent0 mitigates this issue by repositioning the robot to obtain a better view, achieving substantially higher success rates. For the soda can pickup task, the small object size makes accurate vertical camera alignment critical for localization. S3 often fails by adjusting the camera vertically too early, leading to unsuccessful searches within the time limit. CaP-Agent0 adapts its search strategy based on feedback, improving navigation success. Furthermore, grasping may fail when the can is knocked over, further reducing S3’s completion rate. CaP-Agent0 can resample grasp poses after such disturbances and successfully recover, achieving significantly higher task success and similar performance as human expert.

4.3. Let the Agent Experience the Real World

The design of the CaP-Gym environment loop deliberately allows it to directly interface with real-world robot perception and control interfaces, and we additionally demonstrate zero-shot performance of CaP-Agent0 in unseen real-world tasks on real robot embodiments including the Franka Panda and AgiBot G1 without requiring any major cross-embodiment modifications (with the exception of single arm to bimanual control primitive modifications). With no post-training, off-the-shelf VLMs such as Gemini-3-Pro and Claude Opus 4.5 can perform complex long-horizon robotics reasoning and manipulation tasks following natural language instructions. For example, when asked to “find the object under one of the cups” (Figure 12), it mechanically searches through all cups with closed-loop feedback from vision. When the robot is asked to solve a math problem presented in the physical world (Figure 13), it perceives the visual cue, thinks, and selects the correct blocks on the first attempt. We also demonstrate embodied reasoning ability in Figure 15, where CaP-Agent0 exhibits common sense physics reasoning and understands the sensible stacking order for objects of various shapes. Optionally, a human-in-the-loop can also interactively correct the robot’s behavior by providing additional feedback in between turns. More details are documented in Appendix B.

Table 4. Impact of RL Post-Training in Sim and Real. Comparison of success rates between the base model, our RL post-trained agent, and human experts. Simulation results are averaged over 100 trials per task, while real-world deployment on a Franka Emika robot is evaluated over 25 trials.

Method	Simulation (N=100)			Real World (N=25)	
	Cube Lift	Cube Stack	Spill Wipe	Cube Lift	Cube Stack
Human Expert	93%	73%	100%	92%	84%
Qwen 2.5 Coder 7B	25%	4%	30%	24%	12%
Qwen w/ CaP-RL	80%	44%	93%	84%	76%

5. CaP-RL

CaP-Gym enables on-policy reinforcement learning with verifiable rewards (RLVR) directly on the coding agent. To demonstrate this, we apply Group Relative Policy Optimization (GRPO) (Shao et al., 2024; Guo et al., 2025) to post-train a Qwen2.5-Coder-7B-Instruct base model (Hui et al., 2024).

Methodology. We RL post-train on three tasks: *Cube Lift*, *Cube Stack*, and *Spill Wipe*. To ensure stable convergence, we train using the privileged state-based APIs of tier S1. This avoids the noisy reward signals present in tier S2, where compounding perception and control errors can cause otherwise correct programs to fail during execution, introducing credit assignment ambiguity similar to that observed in G1 (Chen et al., 2025).

Simulation Results. Post-training for 50 iterations per task significantly improves code compilation rates and strategic robustness. When evaluated on S2 (noisy perception), the RL post-trained model achieves substantial gains over the base model, as detailed in Table 4.

Sim-to-Real Transfer. A key property of CaP-RL is that what transfers across the sim-to-real boundary is the *code-as-action-space*: the agent learns to compose shared perception and control tools that are fixed across simulation and reality, rather than mapping raw visual features to motor commands. We validate this on a Franka Emika robot, where the agent retains high success rates for cube lifting (84%) and stacking (76%), demonstrating that strategies learned in simulation remain robust under real-world perception noise on these tasks. Please refer to Appendix E for comparing code generation before and after RL post-training.

6. Related Work

Code as Policies. A growing line of work explores programmatic robot control, where LLMs generate executable code that orchestrates perception and control modules (Gupta & Kembhavi, 2023; Yao et al., 2022). In robotics, this paradigm spans grounding plans in affordances (Ahn et al., 2022), composing APIs for closed-loop behaviors (Singh et al., 2023; Liang et al., 2023), generating Python programs over perception APIs (Huang et al., 2023; Mu et al., 2024; Goldberg et al., 2025), and modular vision-language agen-

tic pipelines (Shi et al., 2025; Huang et al., 2022b; Team et al., 2025). Structured intermediate representations such as PDDL (Aeronautiques et al., 1998) and Signal Temporal Logic (Liu et al., 2023a; Chen et al., 2024) and persistent state tracking (Yoneda et al., 2024) improve plan reliability, while executable code is empirically a superior agent action representation (Wang et al., 2024a), with agents benefiting from iterative self-refinement (Shinn et al., 2023; Madaan et al., 2023) and inference-time sampling (Wang et al., 2023b; Snell et al., 2024). Despite this progress, most coding-agent work on robot control still relies on high-level, human-crafted APIs encoding significant task structure. CaP-Gym exposes the full primitive stack down to joint-level perception and control, and CaP-Bench evaluates frontier models across abstraction tiers and inference-time strategies.

Skill Synthesis and RL with LLM-Generated Code. A large body of work uses LLMs as *static* code generators for reward functions, curricula, and skill synthesis with a frozen LLM and a separate trained policy (Ma et al., 2024a; Yu et al., 2023; Liang et al., 2024; Ma et al., 2024b; Du et al., 2023; Wang et al., 2024b; Ahn et al., 2024); complementarily, RL with verifiable rewards (RLVR) improves the model itself across reasoning, code, and agentic settings (Guo et al., 2025; Shao et al., 2024; Wei et al., 2025; Pan et al., 2024; Feng et al., 2026). CaP-RL extends RLVR to robot manipulation by directly fine-tuning the language model via GRPO on physics-simulation outcomes, rather than using the LLM to produce reward code for a separate policy.

Benchmarks for robotics and embodied agents. Robotic manipulation benchmarks (Zhu et al., 2020; Liu et al., 2023b; Li et al., 2024; Mees et al., 2022; James et al., 2019; Yu et al., 2020) evaluate fixed policy interfaces rather than executable program synthesis with tiered APIs and multi-turn debugging; code benchmarks (Chen et al., 2021; Jimenez et al., 2024; Pan et al., 2024) lack embodied perception; and embodied-agent benchmarks (Yang et al., 2025; Chen et al., 2025; Wang et al., 2026; Liu et al., 2024; Shridhar et al., 2020) broaden to multimodal interactive environments but do not require executable robot-control code across abstraction tiers. CaP-Gym targets this intersection.

7. Conclusion

We introduce CaP-X, a unified framework for benchmarking and improving coding agents for robot control. CaP-X consists of CaP-Gym, CaP-Bench, CaP-Agent0, and CaP-RL, enabling controlled evaluation across abstraction levels, interaction modes, and learning paradigms. CaP-X frames robot control as a problem of machine intelligence—where agent design, inference-time computation, perception, and control are co-studied—providing a testbed for evaluating and advancing general-purpose embodied intelligence.

Impact Statement

This paper presents work whose goal is to advance the field of machine learning. There are many potential societal consequences of our work, none of which we feel must be specifically highlighted here.

References

- Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., Aleman, F. L., Almeida, D., Altenschmidt, J., Altman, S., Anadkat, S., et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- Aeronautiques, C., Howe, A., Knoblock, C., McDermott, I. D., Ram, A., Veloso, M., Weld, D., Sri, D. W., Barrett, A., Christianson, D., et al. Pddl—the planning domain definition language. *Technical Report, Tech. Rep.*, 1998.
- Ahn, M., Brohan, A., Brown, N., Chebotar, Y., Cortes, O., David, B., Finn, C., Fu, C., Gopalakrishnan, K., Hausman, K., Herzog, A., Ho, D., Hsu, J., Ibarz, J., Ichter, B., Irpan, A., Jang, E., Jauregui Ruano, R., Jeffrey, K., Jesmonth, S., Joshi, N., Julian, R., Kalashnikov, D., Kuang, Y., Lee, K.-H., Levine, S., Lu, Y., Luu, L., Parada, C., Pastor, P., Quiambao, J., Rao, K., Reymann, J., Ryoo, M., Salazar, G., Sanketi, P., Sayed, K., Singh, J., Sontakke, S., Stone, A., Tan, C., Tran, H., Vanhoucke, V., Vega, S., Vuong, Q., Watkins, C., Welker, S., Wohlhart, P., Wu, J., Xia, F., Xiao, T., Xu, P., Xu, S., Yan, M., Zeng, A., and Zheng, Y. Do as I can, not as I say: Grounding language in robotic affordances. In *Conference on Robot Learning (CoRL)*, 2022. URL <https://arxiv.org/abs/2204.01691>.
- Ahn, M., Dwibedi, D., Finn, C., Arenas, M. G., Gopalakrishnan, K., Hausman, K., Ichter, B., Irpan, A., Joshi, N., Julian, R., Kirmani, S., Leal, I., Lee, E., Levine, S., Lu, Y., Maddineni, S., Rao, K., Sadigh, D., Sanketi, P., Sermanet, P., Vuong, Q., Welker, S., Xia, F., Xiao, T., Xu, P., Xu, S., and Xu, Z. AutoRT: Embodied foundation models for large scale orchestration of robotic agents. *arXiv preprint arXiv:2401.12963*, 2024. URL <https://arxiv.org/abs/2401.12963>.
- AI, M. Kimi-k2-instruct (revision 2f7e011), 2025. URL <https://huggingface.co/moonshotai/Kimi-K2-Instruct>.
- Anthropic. Introducing claude haiku 4.5. <https://www.anthropic.com/news/claude-haiku-4-5>, Oct 2025a. Announcement of Claude Haiku 4.5, Anthropic’s newest small, efficient model.
- Anthropic. Introducing claude opus 4.5. <https://www.anthropic.com/news/claude-opus-4-5>, Nov 2025b. Official announcement of Claude Opus 4.5, Anthropic’s flagship model with advanced coding and agentic capabilities.
- Bjorck, J., Castañeda, F., Cherniadev, N., Da, X., Ding, R., Fan, L., Fang, Y., Fox, D., Hu, F., Huang, S., et al. Gr00t n1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*, 2025.
- Black, K., Brown, N., Driess, D., Esmail, A., Equi, M., Finn, C., Fusai, N., Groom, L., Hausman, K., Ichter, B., Jakubczak, S., Jones, T., Ke, L., Levine, S., Li-Bell, A., Mothukuri, M., Nair, S., Pertsch, K., Shi, L. X., Tanner, J., Vuong, Q., Walling, A., Wang, H., and Zhilinsky, U. π_0 : A vision-language-action flow model for general robot control, 2024. URL <https://arxiv.org/abs/2410.24164>.
- Bradski, G. The opencv library. *Dr. Dobb’s Journal: Software Tools for the Professional Programmer*, 25(11):120–123, 2000.
- Brockman, G., Cheung, V., Pettersson, L., Schneider, J., Schulman, J., Tang, J., and Zaremba, W. Openai gym. *arXiv preprint arXiv:1606.01540*, 2016.
- Brohan, A., Brown, N., Carbajal, J., Chebotar, Y., Chen, X., Chormanski, K., Ding, T., Driess, D., Dubey, A., Finn, C., Florence, P., Fu, C., Arenas, M. G., Gopalakrishnan, K., Han, K., Hausman, K., Herzog, A., Hsu, J., Ichter, B., Irpan, A., Joshi, N., Julian, R., Kalashnikov, D., Kuang, Y., Leal, I., Lee, L., Lee, E., Levine, S., Lu, Y., Michalewski, H., Mordatch, I., Pertsch, K., Rao, K., Reymann, K., Ryoo, M., Salazar, G., Sanketi, P., Sermanet, P., Singh, J., Singh, A., Soricut, R., Tran, H., Vanhoucke, V., Vuong, Q., Wahid, A., Welker, S., Wohlhart, P., Wu, J., Xia, F., Xiao, T., Xu, P., Xu, S., Yu, T., and Zitkovich, B. RT-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning (CoRL)*, 2023. URL <https://arxiv.org/abs/2307.15818>.
- Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al. Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33: 1877–1901, 2020.
- Carion, N., Gustafson, L., Hu, Y.-T., Debnath, S., Hu, R., Suris, D., Ryali, C., Alwala, K. V., Khedr, H., Huang, A., Lei, J., Ma, T., Guo, B., Kalla, A., Marks, M., Greer, J., Wang, M., Sun, P., Rädle, R., Afouras, T., Mavroudi, E., Xu, K., Wu, T.-H., Zhou, Y., Momeni, L., Hazra, R., Ding, S., Vaze, S., Porcher, F., Li, F., Li, S., Kamath, A., Cheng, H. K., Dollár, P., Ravi, N., Saenko, K., Zhang, P., and Feichtenhofer, C. Sam 3: Segment anything with concepts, 2025. URL <https://arxiv.org/abs/2511.16719>.

- Chen, L., Gao, H., Liu, T., Huang, Z., Sung, F., Zhou, X., Wu, Y., and Chang, B. G1: Bootstrapping perception and reasoning abilities of vision-language model via reinforcement learning. *arXiv preprint arXiv:2505.13426*, 2025.
- Chen, M., Tworek, J., Jun, H., Yuan, Q., de Oliveira Pinto, H. P., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B., Gray, S., Ryder, N., Pavlov, M., Power, A., Kaiser, L., Bavarian, M., Winter, C., Tillet, P., Such, F. P., Cummings, D., Plappert, M., Chantzis, F., Barnes, E., Herbert-Voss, A., Guss, W. H., Nichol, A., Paino, A., Tezak, N., Tang, J., Babuschkin, I., Balaji, S., Jain, S., Saunders, W., Hesse, C., Carr, A. N., Leike, J., Achiam, J., Misra, V., Morikawa, E., Radford, A., Knight, M., Brundage, M., Murati, M., Mayer, K., Welinder, P., McGrew, B., Amodei, D., McCandlish, S., Sutskever, I., and Zaremba, W. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- Chen, Y., Arkin, J., Dawson, C., Zhang, Y., Roy, N., and Fan, C. AutoTAMP: Autoregressive task and motion planning with LLMs as translators and checkers. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2024. URL <https://arxiv.org/abs/2306.06531>.
- Chowdhery, A., Narang, S., Devlin, J., Bosma, M., Mishra, G., Roberts, A., Barham, P., Chung, H. W., Sutton, C., Gehrmann, S., et al. Palm: Scaling language modeling with pathways. *Journal of Machine Learning Research*, 24(240):1–113, 2023.
- Christie, T., Trylesinski, M., and contributors. Uvicorn: Asgi web server for python. <https://pypi.org/project/uvicorn/>, 2025. URL <https://uvicorn.dev>. Version 0.40.0, BSD-3-Clause License.
- Clark, C., Zhang, J., Ma, Z., Park, J. S., Salehi, M., Tripathi, R., Lee, S., Ren, Z., Kim, C. D., Yang, Y., Shao, V., Yang, Y., Huang, W., Gao, Z., Anderson, T., Zhang, J., Jain, J., Stoica, G., Han, W., Farhadi, A., and Krishna, R. Molmo2: Open weights and data for vision-language models with video understanding and grounding, 2026. URL <https://arxiv.org/abs/2601.10611>.
- DeepSeek-AI. Deepseek-v3 technical report, 2024. URL <https://arxiv.org/abs/2412.19437>.
- Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018.
- Du, Y., Watkins, O., Wang, Z., Colas, C., Darrell, T., Abbeel, P., Gupta, A., and Andreas, J. Guiding pretraining in reinforcement learning with large language models. In *International Conference on Machine Learning (ICML)*, 2023. URL <https://arxiv.org/abs/2302.06692>.
- Feng, J., Huang, S., Qu, X., Zhang, G., Qin, Y., Zhong, B., Jiang, C., Chi, J., and Zhong, W. Retool: Reinforcement learning for strategic tool use in LLMs. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=tRklnoFsmz>.
- Fikes, R. E. and Nilsson, N. J. Strips: A new approach to the application of theorem proving to problem solving. *Artificial intelligence*, 2(3-4):189–208, 1971.
- Fu, L., Huang, H., Datta, G., Chen, L. Y., Panitch, W. C.-H., Liu, F., Li, H., and Goldberg, K. In-context imitation learning via next-token prediction. *arXiv preprint arXiv:2408.15980*, 2024.
- Goldberg, A., Kondap, K., Qiu, T., Ma, Z., Fu, L., Kerr, J., Huang, H., Chen, K., Fang, K., and Goldberg, K. Blox-net: Generative design-for-robot-assembly using VLM supervision, physics simulation, and a robot with reset. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2025. URL <https://arxiv.org/abs/2409.17126>.
- Google DeepMind. Gemini 3 pro model card. <https://storage.googleapis.com/deepmind-media/Model-Cards/Gemini-3-Pro-Model-Card.pdf>, Nov 2025. Model card for the Gemini 3 Pro multimodal AI model, published November 2025.
- Guo, D., Yang, D., Zhang, H., Song, J., Zhang, R., Xu, R., Zhu, Q., Ma, S., Wang, P., Bi, X., et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- Gupta, T. and Kembhavi, A. Visual programming: Compositional visual reasoning without training. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 14953–14962, 2023. URL <https://arxiv.org/abs/2211.11559>.
- Hendrycks, D., Burns, C., Basart, S., Zou, A., Mazeika, M., Song, D., and Steinhardt, J. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300*, 2020.
- Hu, L., Huo, M., Zhang, Y., Yu, H., Xing, E. P., Stoica, I., Rosing, T., Jin, H., and Zhang, H. Image-bench: How good are llms at playing games? *arXiv preprint arXiv:2505.15146*, 2025.

- Huang, H., Fu, L., Danielczuk, M., Kim, C. M., Tam, Z., Ichnowski, J., Angelova, A., Ichter, B., and Goldberg, K. Mechanical search on shelves with efficient stacking and destacking of objects. In *The International Symposium of Robotics Research*, pp. 205–221. Springer, 2022a.
- Huang, H., Liu, F., Fu, L., Wu, T., Mukadam, M., Malik, J., Goldberg, K., and Abbeel, P. OTTER: A vision-language-action model with text-aware visual feature extraction. In *Forty-second International Conference on Machine Learning*, 2025. URL <https://openreview.net/forum?id=UHF0km7R5M>.
- Huang, S., Jiang, Z., Dong, H., Qiao, Y., Gao, P., and Li, H. Instruct2act: Mapping multi-modality instructions to robotic actions with large language model. *arXiv preprint arXiv:2305.11176*, 2023. URL <https://arxiv.org/abs/2305.11176>.
- Huang, W., Xia, F., Xiao, T., Chan, H., Liang, J., Florence, P., Zeng, A., Tompson, J., Mordatch, I., Chebotar, Y., et al. Inner monologue: Embodied reasoning through planning with language models. *arXiv preprint arXiv:2207.05608*, 2022b.
- Hui, B., Yang, J., Cui, Z., Yang, J., Liu, D., Zhang, L., Liu, T., Zhang, J., Yu, B., Lu, K., et al. Qwen2. 5-coder technical report. *arXiv preprint arXiv:2409.12186*, 2024.
- James, S., Ma, Z., Arrojo, D. R., and Davison, A. J. Rl-bench: The robot learning benchmark & learning environment. *arXiv e-prints*, art. *arXiv preprint arXiv:1909.12271*, 2019.
- Jang, E., Irpan, A., Khansari, M., Kappler, D., Ebert, F., Lynch, C., Levine, S., and Finn, C. Bc-z: Zero-shot task generalization with robotic imitation learning. In *Conference on Robot Learning*, 2022.
- Jiang, Y., Gupta, A., Zhang, Z., Wang, G., Dou, Y., Chen, Y., Fei-Fei, L., Anandkumar, A., Zhu, Y., and Fan, L. VIMA: General robot manipulation with multimodal prompts. *International Conference on Machine Learning (ICML)*, 2023.
- Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., and Narasimhan, K. R. SWE-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=VTF8yNQm66>.
- Jin, T., Cheng, E. Y., Ankner, Z., Saunshi, N., Elias, B. M., Yazdanbakhsh, A., Ragan-Kelley, J., Subramanian, S., and Carbin, M. Learning to keep a promise: Scaling language model decoding parallelism with learned asynchronous decoding. *arXiv preprint arXiv:2502.11517*, 2025.
- Kaelbling, L. P. and Lozano-Pérez, T. Hierarchical task and motion planning in the now. In *2011 IEEE international conference on robotics and automation*, pp. 1470–1477. IEEE, 2011.
- Khatib, O. Real-time obstacle avoidance for manipulators and mobile robots. *The international journal of robotics research*, 5(1):90–98, 1986.
- Khattab, O., Singhvi, A., Maheshwari, P., Zhang, Z., Santhanam, K., Vardhamanan, S., Haq, S., Sharma, A., Joshi, T. T., Moazam, H., Miller, H., Zaharia, M., and Potts, C. Dspy: Compiling declarative language model calls into self-improving pipelines. 2024.
- Kim, C. M., Yi, B., Choi, H., Ma, Y., Goldberg, K., and Kanazawa, A. Pyroki: A modular toolkit for robot kinematic optimization. In *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2025. URL <https://arxiv.org/abs/2505.03728>.
- Kim, M. J., Pertsch, K., Karamcheti, S., Xiao, T., Balakrishna, A., Nair, S., Rafailov, R., Foster, E., Lam, G., Sanketi, P., Vuong, Q., Kollar, T., Burchfiel, B., Tedrake, R., Sadigh, D., Levine, S., Liang, P., and Finn, C. Openvla: An open-source vision-language-action model, 2024. URL <https://arxiv.org/abs/2406.09246>.
- Li, C., Zhang, R., Wong, J., Gokmen, C., Srivastava, S., Martín-Martín, R., Wang, C., Levine, G., Ai, W., Martinez, B., Yin, H., Lingelbach, M., Hwang, M., Hiranaka, A., Garlanka, S., Aydin, A., Lee, S., Sun, J., Anvari, M., Sharma, M., Bansal, D., Hunter, S., Kim, K.-Y., Lou, A., Matthews, C. R., Villa-Renteria, I., Tang, J. H., Tang, C., Xia, F., Li, Y., Savarese, S., Gweon, H., Liu, C. K., Wu, J., and Fei-Fei, L. Behavior-1k: A human-centered, embodied ai benchmark with 1,000 everyday activities and realistic simulation. *arXiv preprint arXiv:2403.09227*, 2024.
- Li, J., Li, D., Savarese, S., and Hoi, S. Blip-2: Bootstrapping language-image pre-training with frozen image encoders and large language models. *arXiv preprint arXiv:2301.12597*, 2023.
- Liang, J., Huang, W., Xia, F., Xu, P., Hausman, K., Ichter, B., Florence, P., and Zeng, A. Code as policies: Language model programs for embodied control. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 9493–9500. IEEE, 2023.
- Liang, W., Wang, S., Wang, H.-J., Bastani, O., Jayaraman, D., and Ma, Y. J. Eurekaverse: Environment curriculum generation via large language models. In *Conference on Robot Learning (CoRL)*, 2024. URL <https://arxiv.org/abs/2411.01775>.

- Liu, B., Jiang, Y., Zhang, X., Liu, Q., Zhang, S., Biswas, J., and Stone, P. LLM+P: Empowering large language models with optimal planning proficiency. *arXiv preprint arXiv:2304.11477*, 2023a. URL <https://arxiv.org/abs/2304.11477>.
- Liu, B., Zhu, Y., Gao, C., Feng, Y., Liu, Q., Zhu, Y., and Stone, P. Libero: Benchmarking knowledge transfer for lifelong robot learning. *Advances in Neural Information Processing Systems*, 36:44776–44791, 2023b.
- Liu, X., Yu, H., Zhang, H., Xu, Y., Lei, X., Lai, H., Gu, Y., Ding, H., Men, K., Yang, K., Zhang, S., Deng, X., Zeng, A., Du, Z., Zhang, C., Shen, S., Zhang, T., Su, Y., Sun, H., Yao, M., Tang, J., Liu, Z., and Dong, Y. Agent-Bench: Evaluating LLMs as agents. In *International Conference on Learning Representations (ICLR)*, 2024. URL <https://arxiv.org/abs/2308.03688>.
- Ma, Y. J., Liang, W., Wang, G., Huang, D.-A., Bastani, O., Jayaraman, D., Zhu, Y., Fan, L., and Anandkumar, A. Eureka: Human-level reward design via coding large language models. In *International Conference on Learning Representations (ICLR)*, 2024a. URL <https://arxiv.org/abs/2310.12931>.
- Ma, Y. J., Liang, W., Wang, H.-J., Wang, S., Zhu, Y., Fan, L., Bastani, O., and Jayaraman, D. DrEureka: Language model guided sim-to-real transfer. In *Robotics: Science and Systems (RSS)*, 2024b. URL <https://arxiv.org/abs/2406.01967>.
- Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegrefe, S., Alon, U., Dziri, N., Prabhunoye, S., Yang, Y., Gupta, S., Majumder, B. P., Hermann, K., Welleck, S., Yazdanbakhsh, A., and Clark, P. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023. URL <https://arxiv.org/abs/2303.17651>.
- Mees, O., Hermann, L., Rosete-Beas, E., and Burgard, W. Calvin: A benchmark for language-conditioned policy learning for long-horizon robot manipulation tasks. *IEEE Robotics and Automation Letters*, 7(3):7327–7334, 2022.
- Mu, Y., Chen, J., Zhang, Q., Chen, S., Yu, Q., Ge, C., Chen, R., Liang, Z., Hu, M., Tao, C., Sun, P., Yu, H., Yang, C., Shao, W., Wang, W., Dai, J., Qiao, Y., Ding, M., and Luo, P. RoboCodeX: Multimodal code generation for robotic behavior synthesis. *arXiv preprint arXiv:2402.16117*, 2024. URL <https://arxiv.org/abs/2402.16117>.
- Murray, R. M., Li, Z., Sastry, S. S., and Sastry, S. S. *A Mathematical Introduction to Robotic Manipulation*. CRC Press, 1994.
- Octo Model Team, Ghosh, D., Walke, H., Pertsch, K., Black, K., Mees, O., Dasari, S., Hejna, J., Xu, C., Luo, J., Kreiman, T., Tan, Y., Chen, L. Y., Sanketi, P., Vuong, Q., Xiao, T., Sadigh, D., Finn, C., and Levine, S. Octo: An open-source generalist robot policy. In *Proceedings of Robotics: Science and Systems*, Delft, Netherlands, 2024.
- Open X-Embodiment Collaboration, O’Neill, A., Rehman, A., Maddukuri, A., Gupta, A., Padalkar, A., Lee, A., Pooley, A., Gupta, A., Mandlekar, A., Jain, A., Tung, A., Bewley, A., Herzog, A., Irpan, A., Khazatsky, A., Rai, A., Gupta, A., Wang, A., Kolobov, A., Singh, A., Garg, A., Kembhavi, A., Xie, A., Brohan, A., Raffin, A., Sharma, A., Yavary, A., Jain, A., Balakrishna, A., Wahid, A., Burgess-Limerick, B., Kim, B., Schölkopf, B., Wulfe, B., Ichter, B., Lu, C., Xu, C., Le, C., Finn, C., Wang, C., Xu, C., Chi, C., Huang, C., Chan, C., Agia, C., Pan, C., Fu, C., Devin, C., Xu, D., Morton, D., Driess, D., Chen, D., Pathak, D., Shah, D., Büchler, D., Jayaraman, D., Kalashnikov, D., Sadigh, D., Johns, E., Foster, E., Liu, F., Ceola, F., Xia, F., Zhao, F., Frujeri, F. V., Stulp, F., Zhou, G., Sukhatme, G. S., Salhotra, G., Yan, G., Feng, G., Schiavi, G., Berseth, G., Kahn, G., Wang, G., Su, H., Fang, H.-S., Shi, H., Bao, H., Amor, H. B., Christensen, H. I., Furuta, H., Walke, H., Fang, H., Ha, H., Mordatch, I., Radosavovic, I., Leal, I., Liang, J., Abou-Chakra, J., Kim, J., Drake, J., Peters, J., Schneider, J., Hsu, J., Bohg, J., Bingham, J., Wu, J., Gao, J., Hu, J., Wu, J., Sun, J., Luo, J., Gu, J., Tan, J., Oh, J., Wu, J., Lu, J., Yang, J., Malik, J., Silvério, J., Hejna, J., Booher, J., Tompson, J., Yang, J., Salvador, J., Lim, J. J., Han, J., Wang, K., Rao, K., Pertsch, K., Hausman, K., Go, K., Gopalakrishnan, K., Goldberg, K., Byrne, K., Oslund, K., Kawaharazuka, K., Black, K., Lin, K., Zhang, K., Ehsani, K., Lekkala, K., Ellis, K., Rana, K., Srinivasan, K., Fang, K., Singh, K. P., Zeng, K.-H., Hatch, K., Hsu, K., Itti, L., Chen, L. Y., Pinto, L., Fei-Fei, L., Tan, L., Fan, L. J., Ott, L., Lee, L., Weihs, L., Chen, M., Lepert, M., Memmel, M., Tomizuka, M., Itkina, M., Castro, M. G., Spero, M., Du, M., Ahn, M., Yip, M. C., Zhang, M., Ding, M., Heo, M., Srirama, M. K., Sharma, M., Kim, M. J., Kanazawa, N., Hansen, N., Heess, N., Joshi, N. J., Suenderhauf, N., Liu, N., Palo, N. D., Shafiullah, N. M. M., Mees, O., Kroemer, O., Bastani, O., Sanketi, P. R., Miller, P. T., Yin, P., Wohlhart, P., Xu, P., Fagan, P. D., Mitrano, P., Sermanet, P., Abbeel, P., Sundaresan, P., Chen, Q., Vuong, Q., Rafailov, R., Tian, R., Doshi, R., Mart’in-Mart’in, R., Bajjal, R., Scalise, R., Hendrix, R., Lin, R., Qian, R., Zhang, R., Mendonca, R., Shah, R., Hoque, R., Julian, R., Bustamante, S., Kirmani, S., Levine, S., Lin, S., Moore, S., Bahl, S., Dass, S., Sonawani, S., Song, S., Xu, S., Halder, S., Karamcheti, S., Adebola, S., Guist, S., Nasiriany, S., Schaal, S., Welker, S., Tian, S., Ramamoorthy, S., Dasari, S., Belkale, S., Park, S., Nair, S., Mirchandani, S., Osa, T., Gupta, T.,

- Harada, T., Matsushima, T., Xiao, T., Kollar, T., Yu, T., Ding, T., Davchev, T., Zhao, T. Z., Armstrong, T., Darrell, T., Chung, T., Jain, V., Vanhoucke, V., Zhan, W., Zhou, W., Burgard, W., Chen, X., Chen, X., Wang, X., Zhu, X., Geng, X., Liu, X., Liangwei, X., Li, X., Pang, Y., Lu, Y., Ma, Y. J., Kim, Y., Chebotar, Y., Zhou, Y., Zhu, Y., Wu, Y., Xu, Y., Wang, Y., Bisk, Y., Cho, Y., Lee, Y., Cui, Y., Cao, Y., Wu, Y.-H., Tang, Y., Zhu, Y., Zhang, Y., Jiang, Y., Li, Y., Li, Y., Iwasawa, Y., Matsuo, Y., Ma, Z., Xu, Z., Cui, Z. J., Zhang, Z., Fu, Z., and Lin, Z. Open x-embodiment: Robotic learning datasets and rt-x models, 2024.
- OpenAI. Openai o1 system card. <https://openai.com/index/openai-o1-system-card/>, Dec 2024. System card describing the OpenAI o1 large language model and its capabilities.
- OpenAI. Gpt-5.1. <https://openai.com/index/gpt-5-1/>, Nov 2025a. GPT-5.1: A smarter, more conversational ChatGPT.
- OpenAI. Introducing gpt-5.2. <https://openai.com/index/introducing-gpt-5-2/>, Dec 2025b. Official introduction of the GPT-5.2 model series as OpenAI’s most advanced large language model.
- OpenAI. gpt-oss-120b & gpt-oss-20b model card. https://cdn.openai.com/pdf/419b6906-9da6-406c-a19d-1bb078ac7637/oai_gpt-oss_model_card.pdf, 2025c. Model card for the GPT-OSS open-weight reasoning models including the 120B model.
- OpenAI. Openai o3 and o4-mini system card. <https://cdn.openai.com/pdf/2221c875-02dc-4789-800b-e7758f3722c1/o3-and-o4-mini-system-card.pdf>, April 2025d. System card for the o3 and o4-mini reasoning models (dated April 16, 2025).
- Pan, J., Wang, X., Neubig, G., Jaitly, N., Ji, H., Suhr, A., and Zhang, Y. Training software engineering agents and verifiers with swe-gym. *arXiv preprint arXiv:2412.21139*, 2024.
- Pan, J., Li, X., Lian, L., Snell, C., Zhou, Y., Yala, A., Darrell, T., Keutzer, K., and Suhr, A. Learning adaptive parallel reasoning with language models. *arXiv preprint arXiv:2504.15466*, 2025.
- Physical Intelligence, Black, K., Brown, N., Darpinian, J., Dhabalia, K., Driess, D., Esmail, A., Equi, M., Finn, C., Fusai, N., et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025.
- QwenLM / Alibaba Cloud. Qwen3-235b-a22b. <https://huggingface.co/Qwen/Qwen3-235B-A22B>, Apr 2025. Official model card for Qwen3-235B-A22B, a 235 billion parameter mixture-of-experts (MoE) large language model in the Qwen3 family.
- Radford, A., Narasimhan, K., Salimans, T., Sutskever, I., et al. Improving language understanding by generative pre-training. 2018.
- Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., Sutskever, I., et al. Language models are unsupervised multitask learners. *OpenAI Blog*, 1(8):9, 2019.
- Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., et al. Learning transferable visual models from natural language supervision. In *International conference on machine learning*, pp. 8748–8763. PMLR, 2021.
- Reed, S., Zolna, K., Parisotto, E., Colmenarejo, S. G., Novikov, A., Barth-Maron, G., Gimenez, M., Sulsky, Y., Kay, J., Springenberg, J. T., et al. A generalist agent. *arXiv preprint arXiv:2205.06175*, 2022.
- Rein, D., Hou, B. L., Stickland, A. C., Petty, J., Pang, R. Y., Dirani, J., Michael, J., and Bowman, S. R. GPQA: A graduate-level google-proof q&a benchmark. In *First Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=Ti67584b98>.
- Rodionov, G., Garipov, R., Shutova, A., Yakushev, G., Schultheis, E., Egiazarian, V., Sinitin, A., Kuznedelev, D., and Alistarh, D. Hogwild! inference: Parallel llm generation via concurrent attention. *arXiv preprint arXiv:2504.06261*, 2025.
- Shah, D., Sridhar, A., Dashora, N., Stachowicz, K., Black, K., Hirose, N., and Levine, S. ViNT: A Foundation Model for Visual Navigation. In *7th Annual Conference on Robot Learning (CoRL)*, 2023.
- Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Bi, X., Zhang, H., Zhang, M., Li, Y., Wu, Y., et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- Shi, J., Yang, R., Chao, K., Wan, B. S., Shao, Y. S., Lei, J., Qian, J., Le, L., Chaudhari, P., Daniilidis, K., et al. Maestro: Orchestrating robotics modules with vision-language models for zero-shot generalist robots. In *NeurIPS 2025 Workshop on Space in Vision, Language, and Embodied AI*, 2025.
- Shinn, N., Cassano, F., Berman, E., Gopinath, A., Narasimhan, K., and Yao, S. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*,

- volume 36, 2023. URL <https://arxiv.org/abs/2303.11366>.
- Shridhar, M., Thomason, J., Gordon, D., Bisk, Y., Han, W., Mottaghi, R., Zettlemoyer, L., and Fox, D. ALFRED: A benchmark for interpreting grounded instructions for everyday tasks. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020. URL <https://arxiv.org/abs/1912.01734>.
- Siciliano, B., Khatib, O., and Kröger, T. *Springer handbook of robotics*, volume 200. Springer, 2008.
- Singh, I., Blukis, V., Mousavian, A., Goyal, A., Xu, D., Tremblay, J., Fox, D., Thomason, J., and Garg, A. Prog-prompt: Generating situated robot task plans using large language models. In *IEEE International Conference on Robotics and Automation*, pp. 11523–11530. IEEE, 2023.
- Snell, C., Lee, J., Xu, K., and Kumar, A. Scaling LLM test-time compute optimally can be more effective than scaling model parameters. *arXiv preprint arXiv:2408.03314*, 2024. URL <https://arxiv.org/abs/2408.03314>.
- Team, G. R., Abdolmaleki, A., Abeyruwan, S., Ainslie, J., Alayrac, J.-B., Arenas, M. G., Balakrishna, A., Batchelor, N., Bewley, A., Bingham, J., et al. Gemini robotics 1.5: Pushing the frontier of generalist robots with advanced embodied reasoning, thinking, and motion transfer. *arXiv preprint arXiv:2510.03342*, 2025.
- Tellex, S., Kollar, T., Dickerson, S., Walter, M., Banerjee, A., Teller, S., and Roy, N. Understanding natural language commands for robotic navigation and mobile manipulation. In *Proceedings of the AAAI conference on artificial intelligence*, volume 25, pp. 1507–1514, 2011.
- TRI LBM Team, Barreiros, J., Beaulieu, A., Bhat, A., Cory, R., Cousineau, E., Dai, H., Fang, C.-H., Hashimoto, K., Irshad, M. Z., Itkina, M., Kuppuswamy, N., Lee, K.-H., Liu, K., McConachie, D., McMahon, I., Nishimura, H., Phillips-Grafflin, C., Richter, C., Shah, P., Srinivasan, K., Wulfe, B., Xu, C., Zhang, M., Alspach, A., Angeles, M., Arora, K., Guizilini, V. C., Castro, A., Chen, D., Chu, T.-S., Creasey, S., Curtis, S., Denitto, R., Dixon, E., Dusel, E., Ferreira, M., Goncalves, A., Gould, G., Guoy, D., Gupta, S., Han, X., Hatch, K., Hathaway, B., Henry, A., Hochshtein, H., Horgan, P., Iwase, S., Jackson, D., Karamcheti, S., Keh, S., Masterjohn, J., Mercat, J., Miller, P., Mitiguy, P., Nguyen, T., Nimmer, J., Noguchi, Y., Ong, R., Onol, A., Pfannenstiehl, O., Poyner, R., Rocha, L. P. M., Richardson, G., Rodriguez, C., Seale, D., Sherman, M., Smith-Jones, M., Tago, D., Tokmakov, P., Tran, M., Hoorick, B. V., Vasiljevic, I., Zakharov, S., Zolotas, M., Ambrus, R., Fetzer-Borelli, K., Burchfiel, B., Kress-Gazit, H., Feng, S., Ford, S., and Tedrake, R. A careful examination of large behavior models for multitask dexterous manipulation. *arXiv preprint arXiv:2507.05331*, 2025. URL <https://arxiv.org/abs/2507.05331>.
- Wang, G., Xie, Y., Jiang, Y., Mandlekar, A., Xiao, C., Zhu, Y., Fan, L., and Anandkumar, A. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023a.
- Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E., Narang, S., Chowdhery, A., and Zhou, D. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations (ICLR)*, 2023b. URL <https://arxiv.org/abs/2203.11171>.
- Wang, X., Chen, Y., Yuan, L., Zhang, Y., Li, Y., Peng, H., and Ji, H. Executable code actions elicit better LLM agents. In *International Conference on Machine Learning (ICML)*, 2024a. URL <https://arxiv.org/abs/2402.01030>.
- Wang, Y., Xian, Z., Chen, F., Wang, T.-H., Wang, Y., Fragkiadaki, K., Erickson, Z., Held, D., and Gan, C. RoboGen: Towards unleashing infinite data for automated robot learning via generative simulation. In *International Conference on Machine Learning (ICML)*, 2024b. URL <https://arxiv.org/abs/2311.01455>.
- Wang, Z., Zhang, J., Ge, J., Lian, L., Fu, L., Dunlap, L., Goldberg, K., Wang, X., Stoica, I., Chan, D. M., Min, S., and Gonzalez, J. E. Visgym: Diverse, customizable, scalable environments for multimodal agents. *arXiv preprint arXiv:2601.16973*, 2026. URL <https://arxiv.org/abs/2601.16973>.
- Wei, Y., Duchenne, O., Copet, J., Carbonneaux, Q., Zhang, L., Fried, D., Synnaeve, G., Singh, R., and Wang, S. I. SWE-RL: Advancing LLM reasoning via reinforcement learning on open software evolution. *arXiv preprint arXiv:2502.18449*, 2025. URL <https://arxiv.org/abs/2502.18449>.
- Yang, R., Chen, H., Zhang, J., Zhao, M., Qian, C., Wang, K., Wang, Q., Koripella, T. V., Movahedi, M., Li, M., et al. Embodiedbench: Comprehensive benchmarking multimodal large language models for vision-driven embodied agents. *arXiv preprint arXiv:2502.09560*, 2025.
- Yao, S., Zhao, J., Yu, D., Du, N., Shafraan, I., Narasimhan, K. R., and Cao, Y. React: Synergizing reasoning and acting in language models. In *The eleventh international conference on learning representations*, 2022.
- Yi, B., Kim, C. M., Kerr, J., Wu, G., Feng, R., Zhang, A., Kulhanek, J., Choi, H., Ma, Y., Tancik, M., and

- Kanazawa, A. Viser: Imperative, web-based 3d visualization in python, 2025. URL <https://arxiv.org/abs/2507.22885>.
- Yoneda, T., Fang, J., Li, P., Zhang, H., Jiang, T., Lin, S., Picker, B., Yunis, D., Mei, H., and Walter, M. R. Statler: State-maintaining language models for embodied reasoning and planning. In *IEEE International Conference on Robotics and Automation (ICRA)*, pp. 15083–15091, 2024. URL <https://arxiv.org/abs/2306.17840>.
- Yu, T., Quillen, D., He, Z., Julian, R., Hausman, K., Finn, C., and Levine, S. Meta-world: A benchmark and evaluation for multi-task and meta reinforcement learning. In *Conference on Robot Learning (CoRL)*, 2020. URL <https://arxiv.org/abs/1910.10897>.
- Yu, W., Gileadi, N., Fu, C., Kirmani, S., Lee, K.-H., Arenas, M. G., Chiang, H.-T. L., Erez, T., Hasenclever, L., Humplik, J., Ichter, B., Xiao, T., Xu, P., Zeng, A., Zhang, T., Heess, N., Sadigh, D., Tan, J., Tassa, Y., and Xia, F. Language to rewards for robotic skill synthesis. In *Conference on Robot Learning (CoRL)*, 2023. URL <https://arxiv.org/abs/2306.08647>.
- Zhou, Q.-Y., Park, J., and Koltun, V. Open3d: A modern library for 3d data processing. *arXiv preprint arXiv:1801.09847*, 2018.
- Zhou, X., Xu, Y., Tie, G., Chen, Y., Zhang, G., Chu, D., Zhou, P., and Sun, L. Libero-pro: Towards robust and fair evaluation of vision-language-action models beyond memorization, 2025. URL <https://arxiv.org/abs/2510.03827>.
- Zhu, Y., Wong, J., Mandlekar, A., and Martín-Martín, R. robosuite: A modular simulation framework and benchmark for robot learning. *arXiv:2009.12293*, 2020.

A. Future Works

Programmatic control performs well on long-horizon, reasoning-heavy tasks, but remains brittle for contact-rich behaviors that require tight visual servoing and continuous feedback (e.g., insertion or pouring). One promising direction is hybrid CaP-VLA policies, in which a coding agent manages high-level task logic and recovery while deferring low-level execution to VLA policies. Results from CaP-Bench further highlight several avenues for improving language-model-based agents, including stronger embodiment-aware planning and reasoning, more effective grounding of task-relevant visual information into code generation, improved test-time search, and agent or prompt optimization methods (Khattab et al., 2024). From a robotics perspective, robustness may further improve by incorporating optimization-based control primitives that allow agents to specify task-level constraints and account for collision avoidance during motion planning, rather than relying solely on inverse kinematics solutions that may be suboptimal when directly interpolated to in joint space. Expanding CaP-Gym to additional environments and classical robotics problems (e.g., active and interactive perception) would further stress-test agentic reasoning.

B. Interactive Real-World Setup

In this section, we document the interactive real-world experience with CaP-Agent0 centered around a chat-based web UI, where the user can propose new tasks to the robot, view the step-by-step tool-use results, and offer feedbacks for multi-turn improvement. We will show how this system enables a real-world robot (AgiBot G1) to zero-shot complete novel tasks.

B.1. User Interface

On the chat UI shown in Figure 9, the user start with choosing task configurations and models. Then they can view the scene description and code generated by the model. As the code is executed on the robot, they also see the step-by-step progress of each tool call, such as the segmentation masks returned by SAM3, Points annotated by Molmo 2, and wrist camera views when the robot arm moves. In between turns, the user can provide additional feedback to the model by typing in the chatbox on the bottom.

The UI also features a 3D visualization powered by Viser (Yi et al., 2025), which shows the live robot URDF, trajectories, and depth pointcloud.

B.2. Tools

CaP-Agent0, specifically on the AgiBot G1, has access to these APIs for perceiving, reasoning, and interacting.

- `get_observation`: returns a dictionary of the current head camera image (RGB, depth, and intrinsics), left and right wrist camera images, end-effector 6-DoF force-wrenches, and joint configuration.
- `segment_sam3_text_prompt`: given an image and a text prompt, returns a list of segmentation masks and scores.
- `segment_sam3_point_prompt`: given an image and a point coordinate on the image, returns a list of segmentation masks and scores.
- `point_prompt_molmo`: given an image and a text prompt, returns a list of points on the image pointed out by Molmo 2.
- `open_gripper`: opens gripper.
- `close_gripper`: closes gripper.
- `goto_pose`: Plans and executes IK to a pose specified by a position and orientation, optionally with a pre-grasp offset and max force-wrench.
- `matrix_to_pose_wxyz_xyz`: Converts a 4×4 transformation matrix to a pose represented in a quaternion and a 3D position.
- `pose_wxyz_xyz_to_matrix`: Converts a pose to a transformation matrix.
- `euler_to_quaternion_wxyz`: Converts extrinsic XYZ Euler angles to a quaternion.

The screenshot displays the CaP-X web interface, which is organized into four main horizontal sections:

- Task & Model Selection:** At the top, this section includes a file selector showing 'env_configs/agibot/agibot_solve_math.yaml' and a 'Load' button. Below this, the 'MODEL' is set to 'Gemini 3 Pro' and the 'SERVER' URL is 'http://127.0.0.1:8110/chat/completions'. A 'Stop' button and the filename 'agibot_solve_math.yaml' are also present.
- Model Response:** This section contains the task description: 'Goal: With your right arm, solve the math problem using the remaining cubes on the table' and a 'Note' about handling nan values in depth images. It also shows a 'Regenerate' button and a timestamp of 2:50:19 PM. The model's response is a Python code block for 'solve_math_problem_v2()'.
- Execution Details:** This section shows the progress of the task. It includes a 'Robot Says' step (Step 1) where the robot retries the math problem, and two 'SAM3 Text Segmentation' steps (Step 2 and Step 3) where the system queries for specific cubes (number 6 and number 7). A video frame shows the robot's gripper interacting with the cubes on a table.
- Human Input:** At the bottom, there is a text input area with the placeholder 'Model is generating...' and buttons for 'Send' and 'Skip'.

Figure 9. An example of the web UI, where users can select the task and model at the top, view the model’s responses and execution details in the middle, and provide instructions at the bottom.

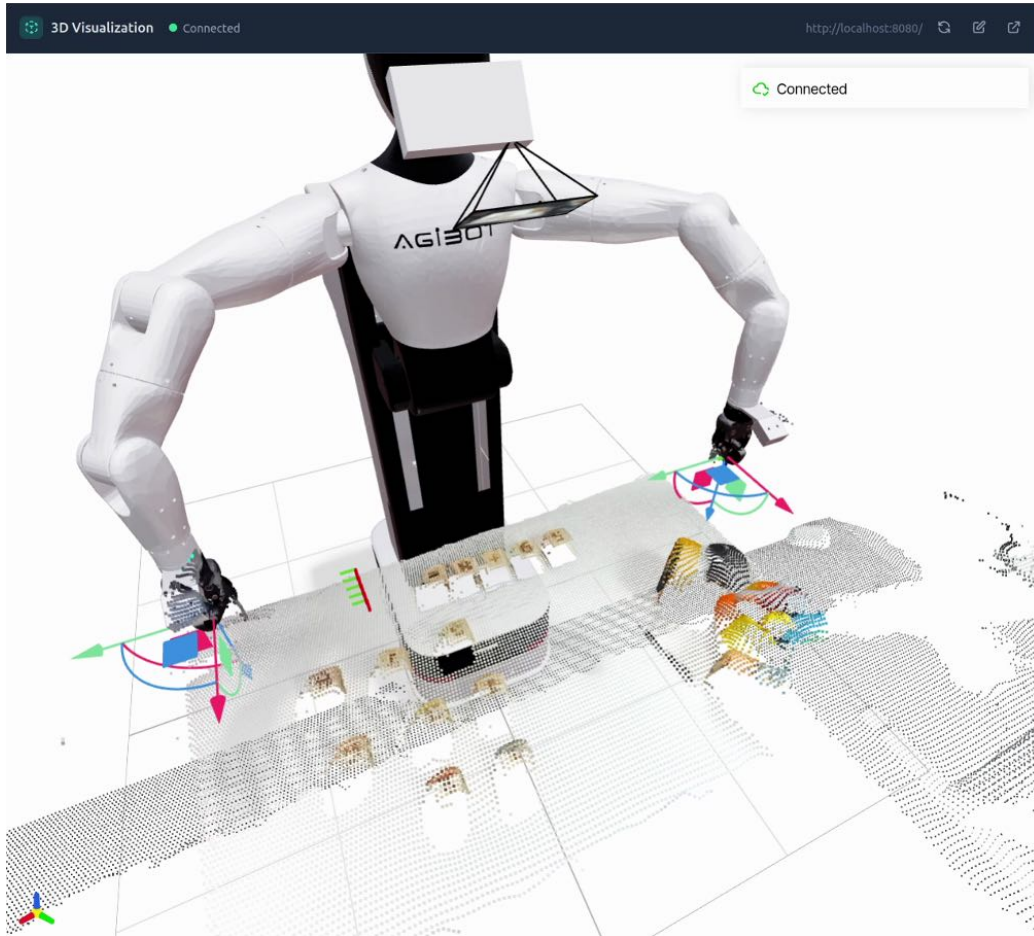


Figure 10. Viser (Yi et al., 2025) visualization section of the web UI.

- `quaternion_wxyz_to_euler`: Converts quaternion of extrinsic XYZ Euler angles.
- `query_vlm`: Ask a question with text and images to an expert VLM model (Gemini-3-Pro)
- `go_forward`: Move forward 1 meter.
- `turn_left_45_degrees`: Turn left 45 degrees.
- `turn_right_45_degrees`: Turn right 45 degrees.
- `goto_planar_position`: Plan and execute IK to a position with planar motion constraint, but no orientation (yaw) requirement.
- `convert_depth_to_pointcloud`: Given a depth image, intrinsics, and extrinsics, convert the depth image into a pointcloud.
- `forward_kinematics`: Given a robot configuration, compute the end-effector pose.
- `overlay_eef_axis_on_image`: Visualize end-effector pose on a camera image by rendering the 3D axis on top.
- `overlay_segmentation_masks`: Visualize segmentation masks on an image.
- `say_something`: Convey some intent to the user.

Among the tools, the 3D rigid body transformation helpers are provided although technically not strictly necessary. We find that when not provided, these functions are almost always written by the models who make mistakes from time to time.

Visualization tools such as `overlay_eef_axis_on_image` and `overlay_segmentation_masks` enables advanced multi-modal reasoning when the output images are passed to an expert VLM via `query_vlm`. This allows, for example, code execution conditioning based on if the expert VLM answered "which of these segmentation masks should I pick?".

B.3. Real World Tasks

In this section, we show visualizations of CaP-Agent0 completing tasks in the real world. All experiments are *zero-shot*. The agent’s execution details are condensed and presented as pseudo sub-steps, while CaP-Agent0 writes the underlying code in greater detail using tools available in B.2. Raw code example is available in Appendix F.

B.3.1. NEEDLE IN A HAYSTACK

In this task, we evaluate the ability of CaP-Agent0 to find “needles in a haystack.” Specifically, in a cluttered scene containing diverse objects, the robot is tasked with locating and grasping an auto pencil refill holder, a relatively uncommon item. Such uncommon objects are often challenging for end-to-end learning policies such as VLAs. In contrast, CaP-Agent0 leverages pretrained VLMs to successfully localize and retrieve the target object, as shown in Figure 11.

B.3.2. MECHANICAL SEARCH

In this task, three inverted cups are placed on a table, and the robot is instructed to retrieve a green lime hidden beneath one of them. This occluded object retrieval problem is commonly referred to as mechanical search (Huang et al., 2022a). Because the lime is concealed by the cups, the robot must systematically explore each cup to locate it. A representative planning and execution example is shown in Figure 12.

B.3.3. MULTIMODAL SYMBOLIC REASONING

In this task, we evaluate the multimodal symbolic reasoning capabilities of CaP-Agent0 by requiring it to solve a mathematical equation formed by numbered wooden blocks. The robot must first perceive the scene to interpret the equation and then grasp the correct block and place it in correct location to complete the solution, as shown in Figure 13.

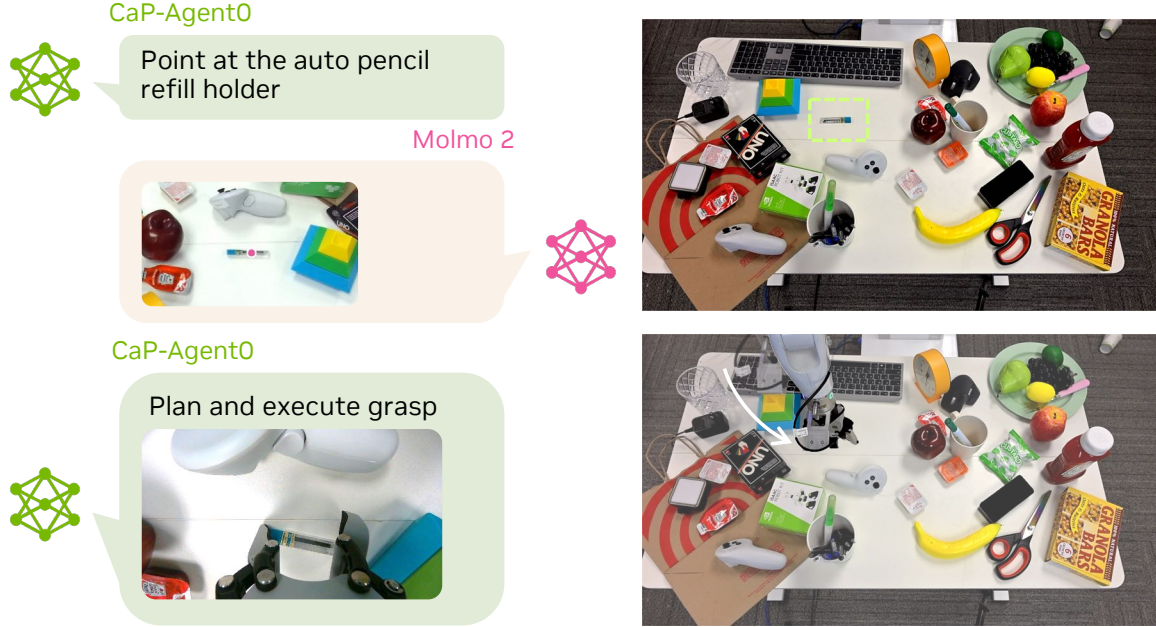


Figure 11. An example of CaP-Agent0 locating and grasping an auto pencil refill holder, where it uses Molmo2 to locate the object.

B.3.4. LEARNING FROM HUMAN FEEDBACK

CaP-Agent0 is able to incorporate human feedback and adjust its code generation accordingly to complete the task. As shown in Figure 14, the robot is instructed to pick up an apple but fails in the initial attempt due to an excessively high grasp pose. After receiving the feedback “grasped the apple too high,” CaP-Agent0 modifies the generated code and successfully completes the task on the second attempt.

B.3.5. EMBODIED REASONING

This task evaluates the embodied reasoning ability of CaP-Agent0, where the robot is asked to “stack objects as high as possible”. Because the scene contains both square and round objects, the robot must reason about their physical properties to determine a stable stacking strategy. As shown in Figure 15, CaP-Agent0 learns to place round objects on top of square ones to maximize stability.

B.3.6. TOOL GENERALIZATION WITH DOMAIN KNOWLEDGE

This task demonstrates the versatility of the code-as-policy interface, where CaP-Agent0 can use tools from arbitrary Python packages to assist with the task. We ask the robot to “take the elevator downstairs”, which suggests locating and pressing the button, and moving into the elevator. Since the robot is positioned at an angle with respect to the wall, it is not immediately obvious in which direction the button should be pushed. CaP-Agent0 is able to invoke SciPy RANSAC algorithm on the segmented wall pointcloud, and compute the surface normal direction, shown in Figure 16.

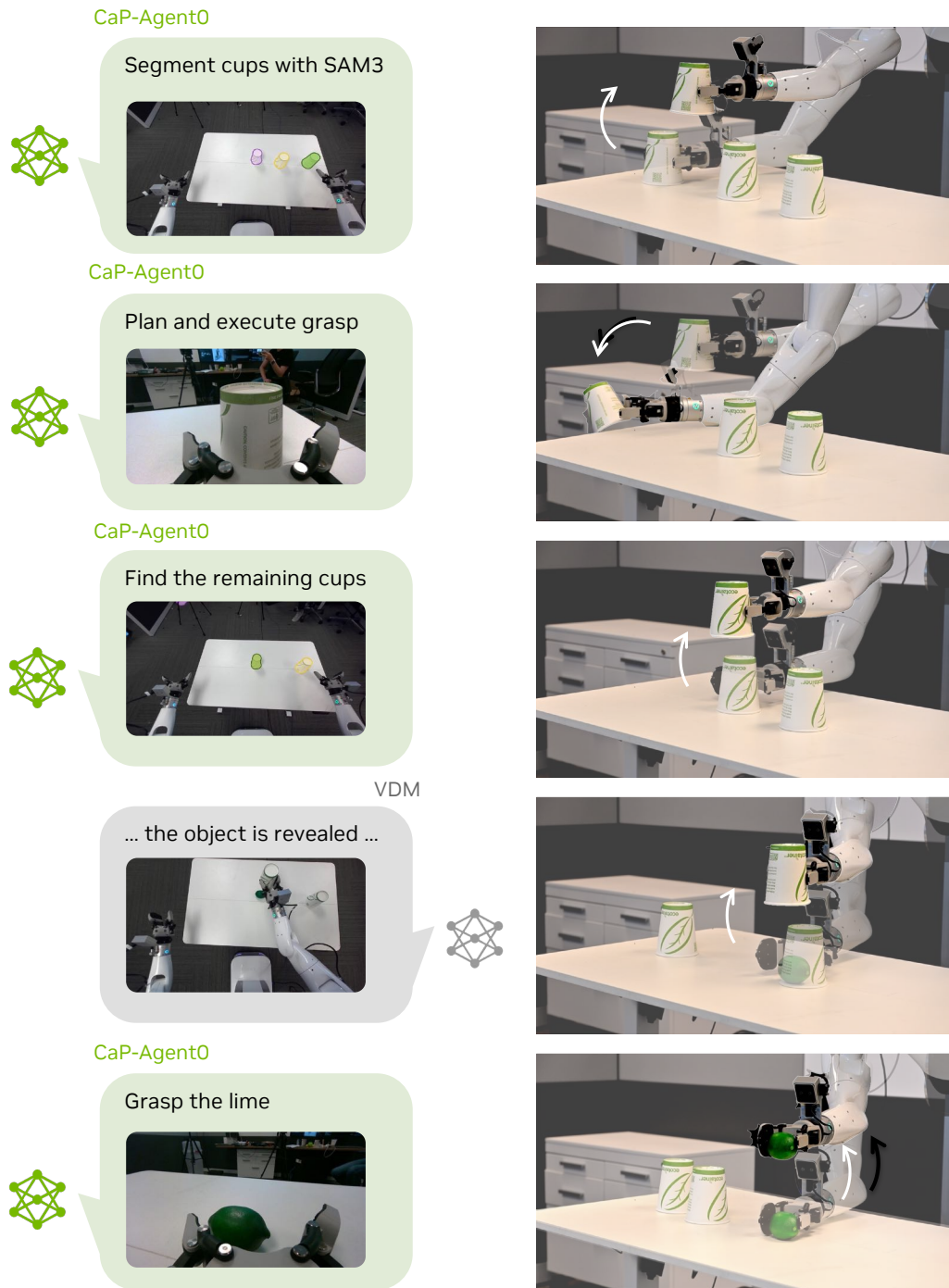


Figure 12. Mechanical search planning and execution example. The left column shows the agent planning process and the right column shows the physical robot execution.

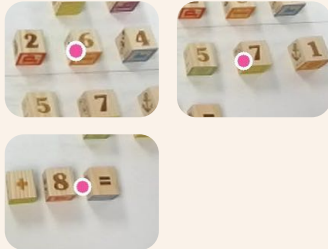
CaP-Agent0



Solving the problem requires putting "6" and "7" on the right of the equal block.

Point at the "6", "7", and the "equal" block.

Molmo 2



CaP-Agent0



Segment with SAM3 and compute bounding box



CaP-Agent0

Plan and execute grasps

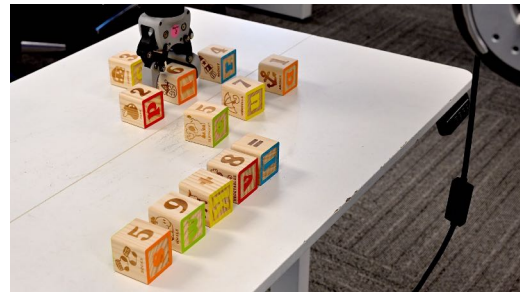
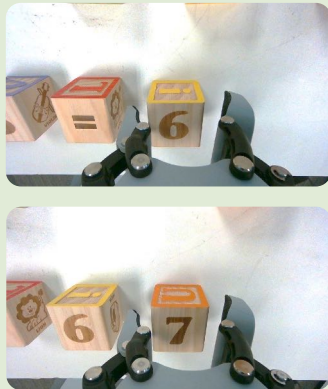


Figure 13. Example of a Multimodal Symbolic Reasoning task, where the robot is asked to solve the problem of 59 plus 8.

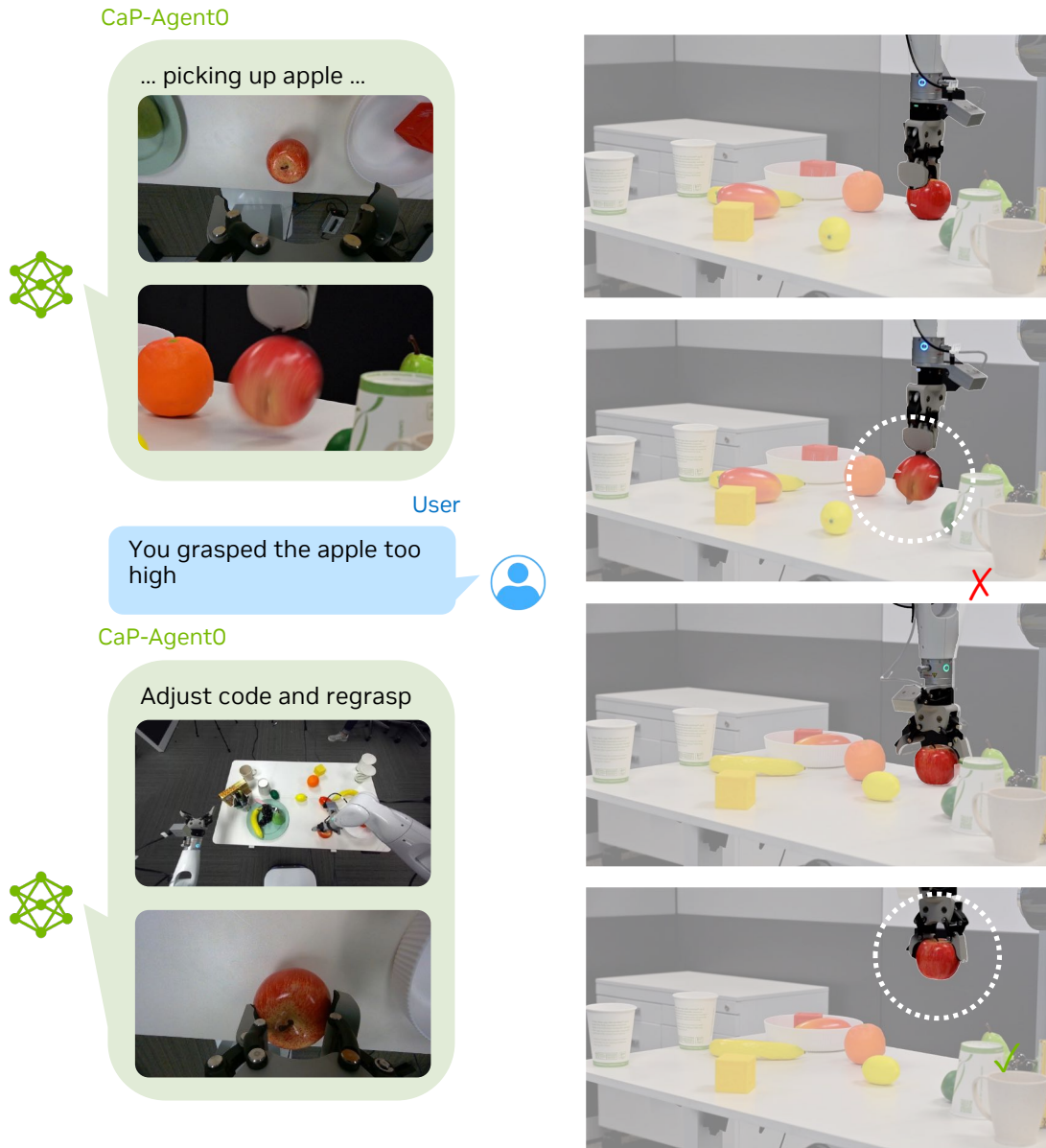


Figure 14. Example of picking up an apple. While the first attempt fails, CaP-Agent0 takes human feedback as input, adjusts the code and completes the task successfully.

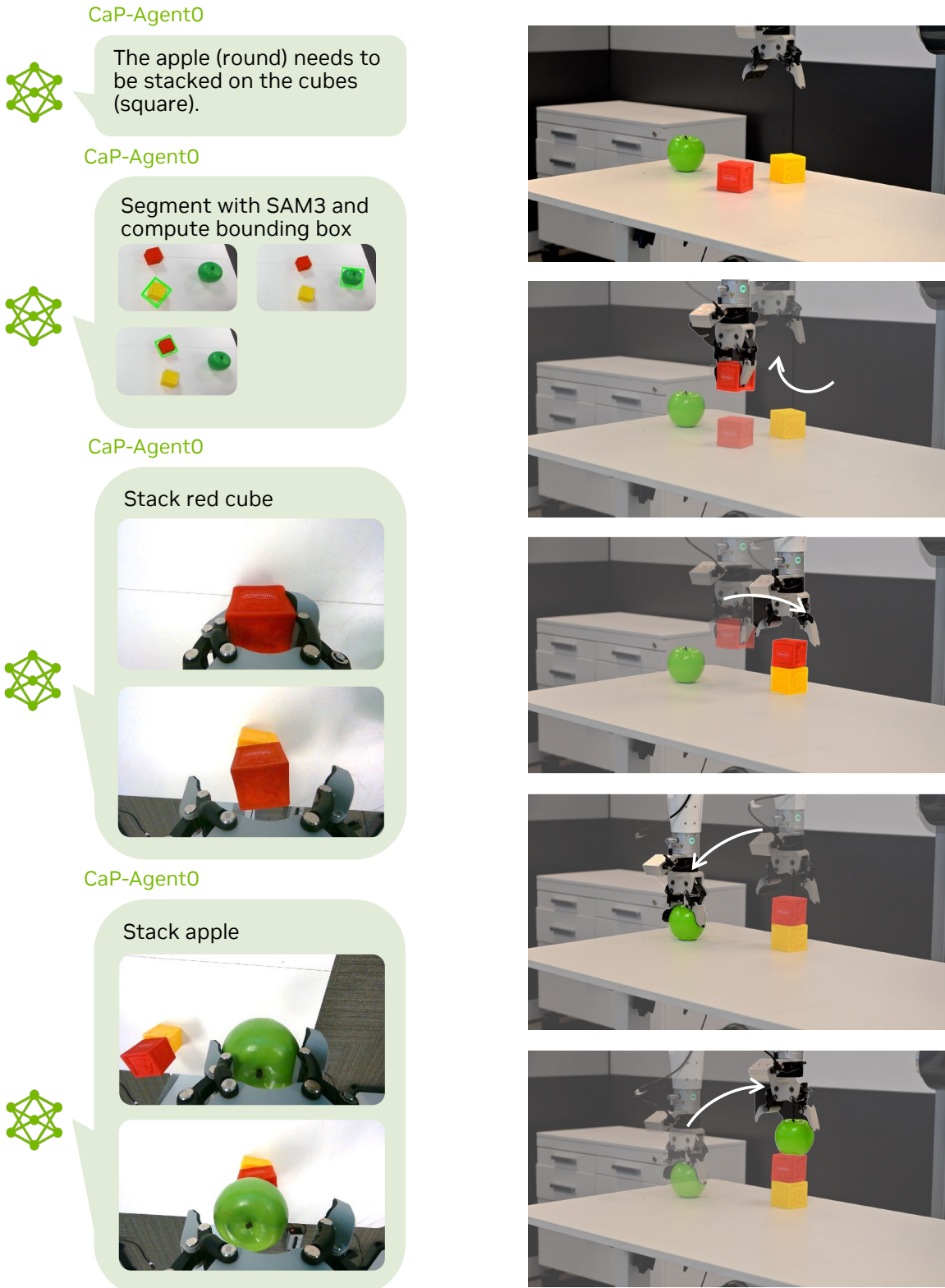


Figure 15. Example of CaP-Agent0 stacking round objects on top of square objects to form the tallest possible stable stack.

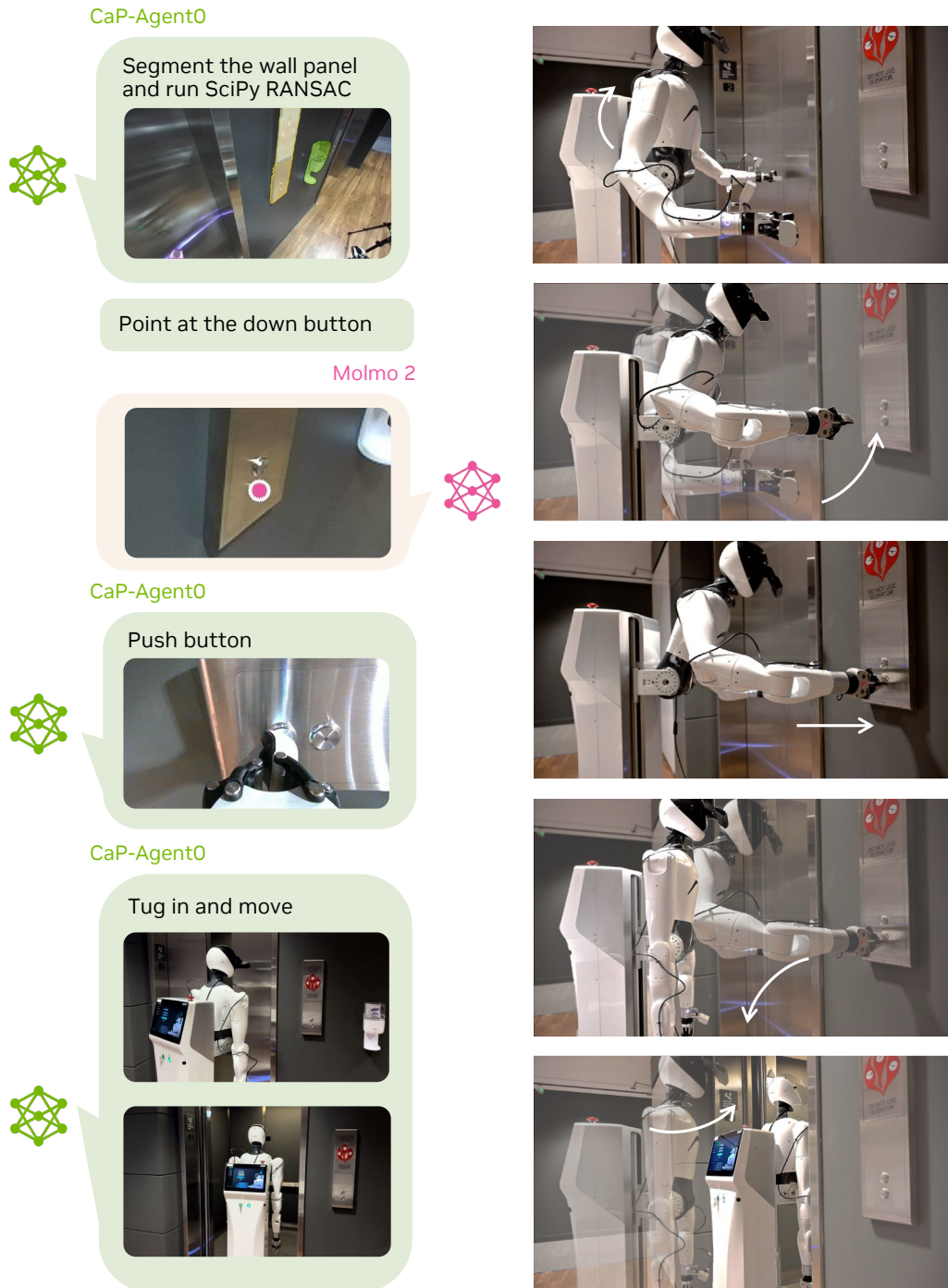


Figure 16. Example of robot pushing the elevator button.

C. Full Benchmark Table

We present the full results from CaP-Bench in Figure 17. From top to bottom are code compilation success rate, average dense reward, and average task success rate.

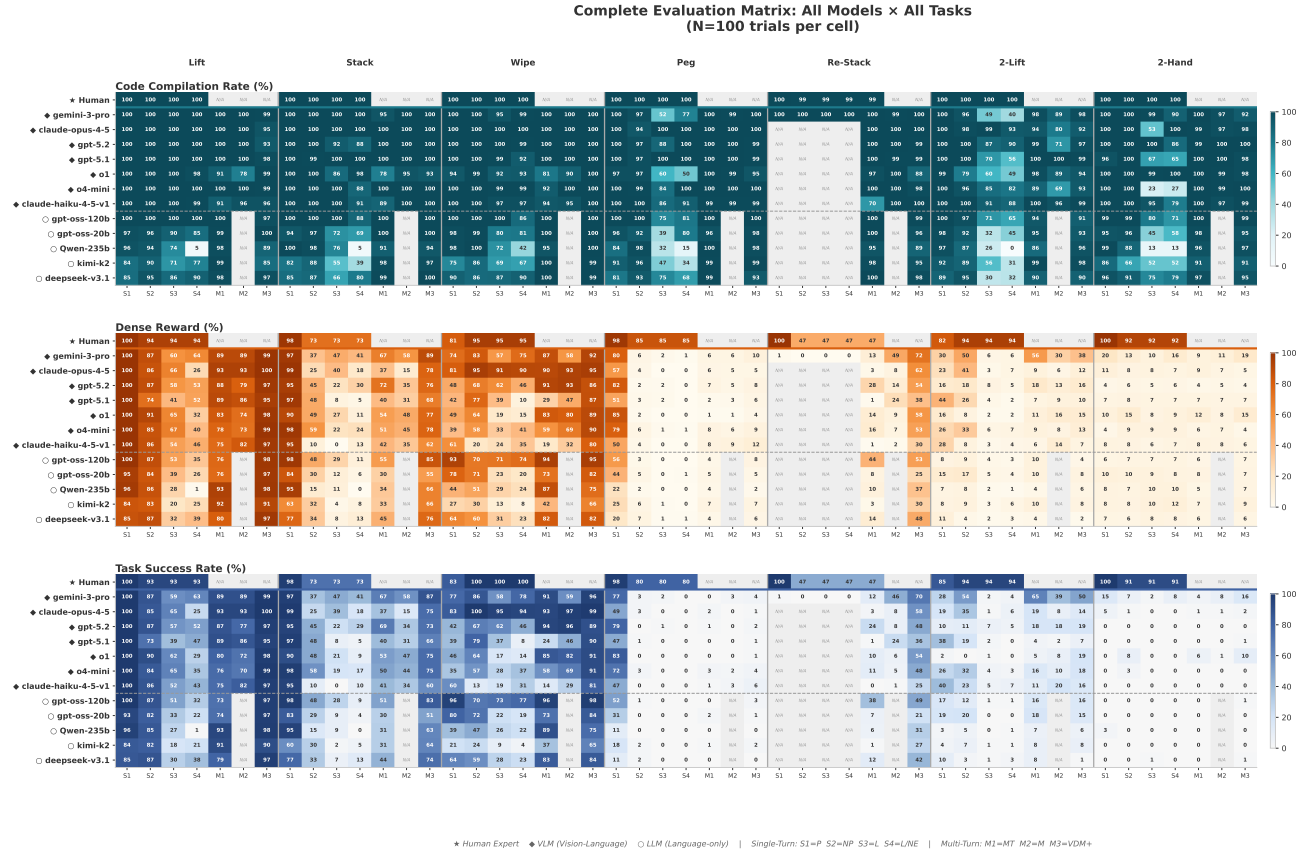


Figure 17. Full CaP-Bench results. From top to bottom are code compilation success rates, average task reward, and average task success rate. Note that for Re-Stack, with a simple task prompt, without visual input, we do not observe model writing code to first check the initial condition and then perform the task, resulting in failures. Therefore, we only evaluated all models for the multi-turn setup.

D. Additional Takeaways

In-Context Examples of API Usage improves performance. In the docstring of each API used in S4, we provided a brief description of the API, the expected datatype, and shape for each input and output. However, due to low code compilation successes from open source models in the initial experiments, we hypothesize that additional prompting may be necessary to increase model performance. Therefore, in addition to the components above, we added API usage examples in the docstring.

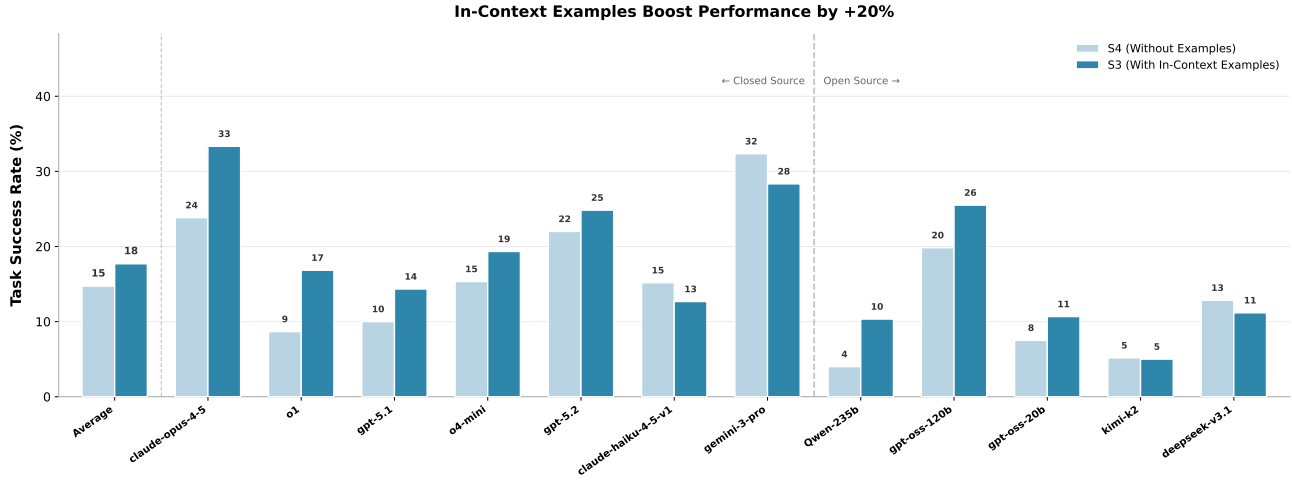


Figure 18. Per-model breakdown of usefulness of in-context API Usage

We present the results in Figure 3 and Figure 4, and a per-model breakdown in Figure 18. We observe that almost all models benefited from adding additional context of examples of how to use the low-level APIs.

E. Qualitative Analysis of CaP-RL Post-Training Effects

In this section, we analyze the qualitative behavioral shifts induced by on-policy reinforcement learning (GRPO) on the Qwen2.5-Coder-7B-Instruct base model. We focus on the *Stack Red Cube on Green Cube* task to illustrate how RL mitigates fundamental instruction-following failures and induces robust geometric reasoning. Methodologically, CaP-RL leverages the privileged S1 tier to stabilize training by removing perception noise. Below we show the code block examples before and after CaP-RL Post-Training.

Before CaP-RL Post-Training Model Behavior. Qwen2.5-Coder-7B-Instruct base model code generation example on the task *Stack Red Cube on Green Cube* prior to on-policy reinforcement learning with CaP-RL:

```
1 import numpy as np
2
3 # Get the pose of the green cube
4 green_cube_position, green_cube_quaternion_wxyz, _ = get_object_pose("green cube")
5
6 # Get the pose of the red cube
7 red_cube_position, red_cube_quaternion_wxyz, _ = get_object_pose("red cube")
8
9 # Calculate the placement position for the red cube
10 placement_position = np.array(green_cube_position) + np.array([0, 0, 0.05]) # 5 cm
   above the green cube
11
12 # Move the robot to the placement position
13 goto_pose(placement_position, (0, 0, 1, 0), z_approach=0.1) # Use a slight z_approach
   for precision
14
15 # Open the gripper to release the red cube
16 open_gripper()
```

After CaP-RL Post-Training Model Behavior. Code generation example of Qwen2.5-Coder-7B-Instruct model after on-policy reinforcement learning with CaP-RL on the task *Stack Red Cube on Green Cube* :

```
1 import numpy as np
2
3 # Get the positions and extents of the red and green cubes
4 red_cube_pos, _, red_cube_extent = get_object_pose("red cube", return_bbox_extent=True)
5 green_cube_pos, _, green_cube_extent = get_object_pose("green cube", return_bbox_extent
   =True)
6
7 # Calculate the placement position on top of the green cube
8 placement_pos = green_cube_pos + np.array([0, 0, red_cube_extent[2] / 2 +
   green_cube_extent[2] / 2])
9
10 # Sample a grasp pose for the red cube
11 grasp_pos, grasp_quat = sample_grasp_pose("red cube")
12
13 # Move to the grasp pose and approach
14 goto_pose(grasp_pos, grasp_quat, z_approach=0.1)
15
16 # Close the gripper to grasp the red cube
17 close_gripper()
18
19 # Move to the placement position
20 goto_pose(placement_pos, (0, 0, 1, 0), z_approach=0.1)
21
22 # Open the gripper to release the red cube
23 open_gripper()
```

Pre-RL Failure Modes: Step Skipping and Hallucinated State. Prior to RL post-training, the base model frequently exhibits "step skipping", a failure mode where the agent attempts to satisfy the final goal state without executing the necessary prerequisites. As shown in the pre-RL snippet, the model correctly identifies the target location (`placement_position`) but fails to grasp the red cube. It attempts to move the gripper directly to the placement target and open it, seemingly hallucinating that it is already holding the object.

Post-RL Improvement: Causal Sequencing and Geometric Generalization. After GRPO training, two critical improvements emerge:

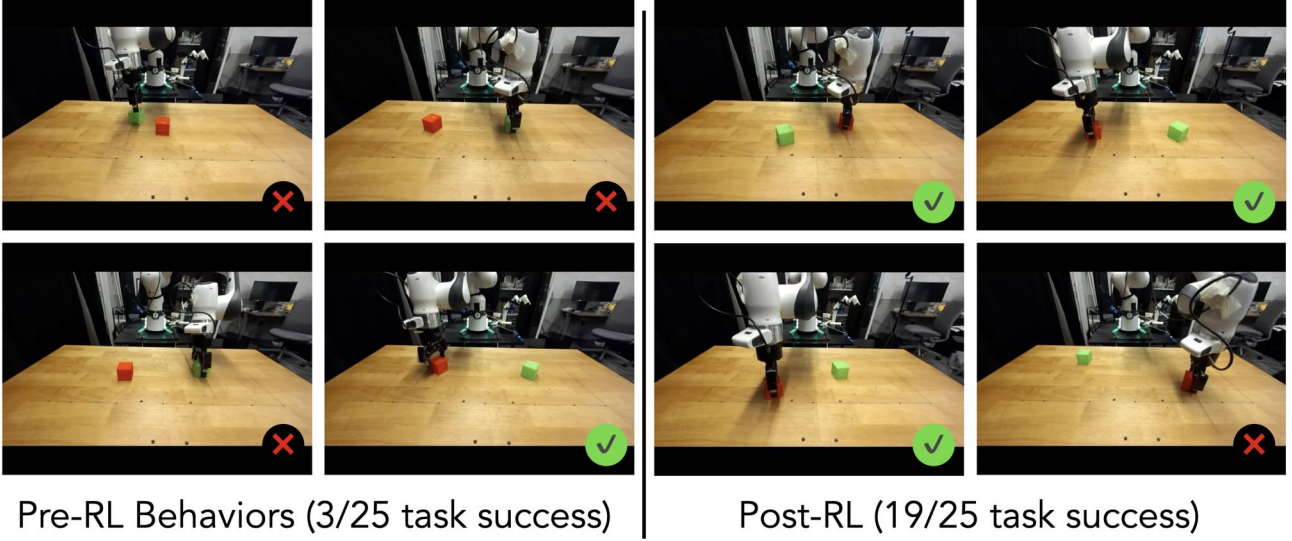


Figure 19. CaP-RL Finetuned Qwen-2.5-Coder-7B-Instruct deployed on the real task *put the red cube on the green cube*. Human Oracle code gets 21/25 task successes in real. Prior to reinforcement learning, we see behaviors such as grasp approaches on the green cube as the first action, rather than a more sensible approach on the red cube first.

1. **Causal Sequencing:** The model correctly synthesizes the full manipulation chain: *Identify* $\rightarrow$ *Grasp* $\rightarrow$ *Transport* $\rightarrow$ *Release*. It learns through environment interaction and reward feedback signals that the causal dependency that an object must be grasped (via `close_gripper`) before it can be placed.
2. **Dynamic Geometric Reasoning:** Instead of using hard-coded offsets, the RL-trained model utilizes the `return_bbox_extent=True` parameter to dynamically calculate the stacking height based on object dimensions ($\text{red_extent}[2]/2 + \text{green_extent}[2]/2$). This indicates a shift from memorization to grounded geometric reasoning.

Zero-shot Generalization to Non-privileged Physical World Setups. Although the model is only post-trained on the privileged S1 tier, the resulting policies demonstrate robust zero-shot transfer to the non-privileged S2 tier. Consequently, the model functions effectively in physical real-world setups, as shown in Figure 19.

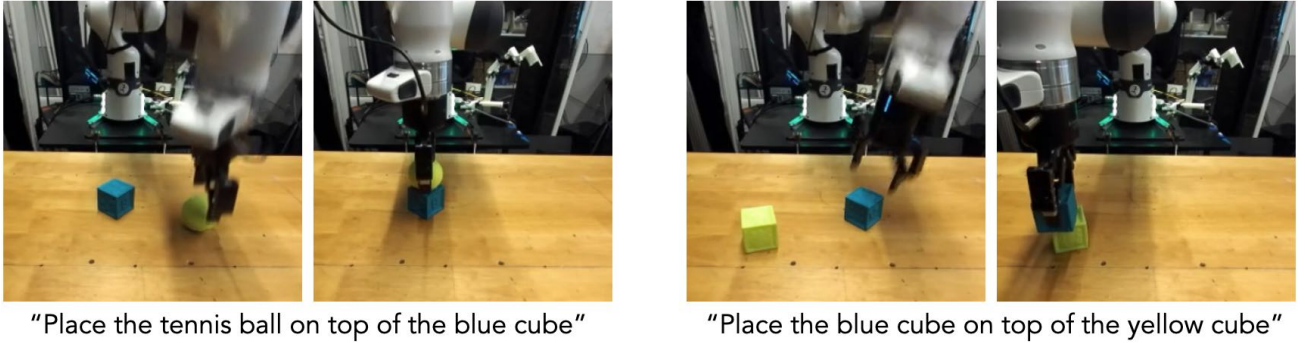


Figure 20. CaP-RL Finetuned Qwen-2.5-Coder-7B-Instruct successfully deployed on similar task variants. Demonstrates that base model is still capable of instruction following.

Zero-Shot Generalization to Task Variants. We observe instruction-following and zero-shot generalization to similar task variants after RL post-training, as seen in 20. The same policy successfully executes variants such as *"put the tennis ball on the green cube"* or tasks involving randomized object colors, as the underlying logic relies on abstract properties (bounding boxes) rather than overfitting to specific entity names or training instance scales.

F. CaP-Agent0 Case Studies

F.1. Real-World Case Studies

F.1.1. COMMON SENSE PHYSICS-AWARE TASK DECOMPOSITION



Figure 21. "Stack these as high as you can."

We validate CaP-Agent0 on a real-world Franka Panda platform using a set of geometrically heterogeneous objects: a yellow cube, a blue cube, and a tennis ball. The agent was given the open-ended instruction: *"stack these as high as you can."* with no additional prompt context outside of API usage documentation and the multimodal agent initial scene description.

Crucially, the task prompt provides neither a specific stacking sequence nor *any* semantic description of the scene or its objects. While a standard visuomotor policy might attempt to stack objects in detection order, potentially attempting to balance a cube on top of the spherical ball, CaP-Agent0 demonstrated common-sense embodied physical reasoning by deriving the only stable construction order.

This behavior is enabled by the CaP-Agent0 auxiliary VDM multimodal agent that extracts task-relevant image information before code generation begins. Prompted to "describe the initial state of the environment with the goal of the task in mind," the VLM emits the following context for the coding agent's initial turn:

"...The cubes appear to have flat surfaces suitable for stacking, while the tennis ball is spherical and would likely need to be placed on top or handled carefully."

Leveraging this textual grounding, the coding agent explicitly decomposed the problem based on stability constraints. The full code generated by the CaP-Agent0 agent for completing this task is found in Appendix I.1.

F.1.2. IMPLICIT MULTI-STEP REASONING FOR OBSTRUCTED GOALS

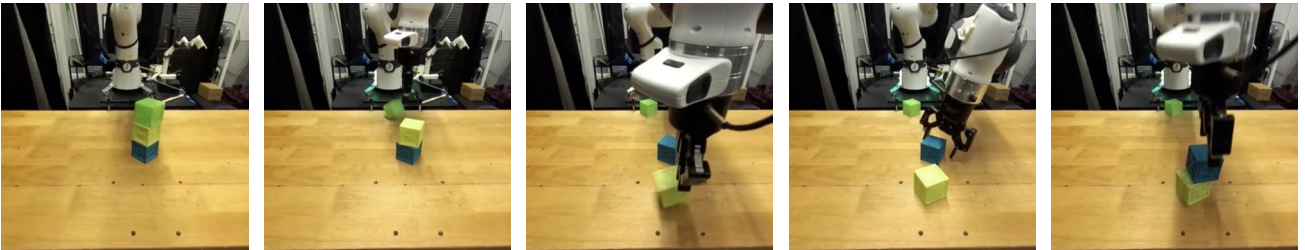


Figure 22. "Place the blue cube on top of the yellow cube."

We evaluate CaP-Agent0 on a task where the Franka Panda must place a blue cube on top of a yellow cube. In the initial state, the cubes form a three-object tower: blue is at the base, supporting yellow, which in turn supports a green cube (Figure 22).

The goal prompt *"place the blue cube on top of the yellow cube"* is linguistically underspecified; it contains no mention of the green obstruction nor the fact that the blue target currently supports the yellow destination.

CaP-Agent0 resolves this dependency through its auxiliary multimodal perception layer. Before code generation, the VLM provides the coding agent with a high-level strategic prior:

"...the blue cube is at the bottom of the stack, underneath the yellow cube, and the yellow cube is underneath a green cube. This implies a complex manipulation task where the robot will likely need to: 1. Remove the top (green) cube. 2. Remove the middle (yellow) cube to access the blue one, or move the stack. 3. Ultimately re-arrange them such that the blue cube ends up on top of the yellow one. The straightforward description is that the target blue cube is currently obstructed and supporting the target destination object (the yellow cube)."

This behavior demonstrates that programmatic agents, when augmented with a dedicated perception-to-text grounding layer, can derive complex manipulation strategies traditionally requiring extensive task-specific imitation learning data or manual state-machine design. The full executable rollout is provided in Appendix I.2.

F.2. Simulation Case Studies

F.2.1. VERIFICATION AND RETRY BEHAVIOR ENABLED BY MULTITURN DESIGN

This case study comes from the "pick up the red cube" task from Robosuite ported into CaP-Gym. Unlike one-shot code generation frameworks, the multi-turn architecture of CaP-Agent0 allows for emergent agentic resilience. In several simulation trials, we observed instances where a planned grasp was physically executed but failed to successfully lift the object due to physics-engine stochasticity or sub-optimal contact points. Rather than terminating the task upon failure, the agent utilized the subsequent turn's observation to verify that the goal condition (e.g., $z_{object} > z_{table}$) remained unfulfilled. As shown in the generated logic, the agent autonomously synthesized a recovery routine branch. This capacity to reason about execution failures and reset the environment state demonstrates that CaP-Agent0 can function as a closed-loop reactive agent rather than a static script generator.

```

1  ...
2
3  masks = segment_sam3_text_prompt(rgb, "red cube")
4
5  is_lifted = False
6  if not masks:
7      print("Could not find red cube to verify state. Assuming failed and moving to home pose.")
8      is_lifted = False
9  else:
10     best_mask = max(masks, key=lambda x: x["score"])["mask"]
11
12     y_idx, x_idx = np.where(best_mask)
13     if len(y_idx) > 0:
14         stride = 10
15         y_sample = y_idx[::stride]
16         x_sample = x_idx[::stride]
17
18         z_sample = depth[y_sample, x_sample]
19         if z_sample.ndim > 1: z_sample = z_sample.flatten()
20
21         # Filter invalid depth
22         valid = z_sample > 0
23         if np.any(valid):
24             z_sample = z_sample[valid]
25             x_sample = x_sample[valid]
26             y_sample = y_sample[valid]
27
28             fx = intrinsics[0, 0]
29             fy = intrinsics[1, 1]
30             cx = intrinsics[0, 2]
31             cy = intrinsics[1, 2]
32
33             x_c = (x_sample - cx) * z_sample / fx
34             y_c = (y_sample - cy) * z_sample / fy
35
36             points_c = np.stack([x_c, y_c, z_sample], axis=1)
37             points_c_homo = np.hstack([points_c, np.ones((len(points_c), 1))])
38             points_w = (cam_pose @ points_c_homo.T).T[:, :3]
39
40             mean_z = np.mean(points_w[:, 2])
41             print(f"Estimated mean Z of object: {mean_z:.4f}")
42
43             # If Z > 0.15, we assume it's lifted (table usually ~0, cube size ~0.05)

```

```

44         if mean_z > 0.15:
45             is_lifted = True
46             print("Object is confirmed to be lifted.")
47         else:
48             print("Object appears to be on the table.")
49
50 if is_lifted:
51     print("Task completed successfully.")
52 else:
53     print("Retrying grasp sequence...")
54     open_gripper()
55
56     # Move to home pose to get a better view (avoid occlusion from gripper)
57     home_pos = np.array([0.5, 0.0, 0.5])
58     home_quat = np.array([0.0, 1.0, 0.0, 0.0]) # w,x,y,z (Approx gripper pointing down)
59
60     move_to_joints(solve_ik(home_pos, home_quat))
61
62     # Re-perceive
63     obs = get_observation()
64     rgb = obs["robot0_robotview"]["images"]["rgb"]
65     depth_im = obs["robot0_robotview"]["images"]["depth"]
66     intrinsics = obs["robot0_robotview"]["intrinsics"]
67     cam_pose = obs["robot0_robotview"]["pose_mat"]
68
69     masks = segment_sam3_text_prompt(rgb, "red cube")
70     if not masks:
71         print("No red cube found after reset.")
72     else:
73         best_mask = max(masks, key=lambda x: x["score"])["mask"]
74         if depth_im.ndim == 3: depth_map = depth_im[:, :, 0]
75         else: depth_map = depth_im
76
77     grasp_poses, grasp_scores = plan_grasp(depth_map, intrinsics, best_mask.astype(
78         np.int32))
79
80     if len(grasp_poses) == 0:
81         print("No grasps found.")
82     else:
83         best_idx = np.argmax(grasp_scores)
84         best_pose_cam = grasp_poses[best_idx]
85
86         pose_world = cam_pose @ best_pose_cam
87         target_pos = pose_world[:3, 3]
88         target_rot = pose_world[:3, :3]
89         target_quat = rot_to_quat(target_rot)
90
91         # Grasp execution
92         # 1. Move to Pre-grasp
93         pre_pos = target_pos + np.array([0, 0, 0.1])
94         move_to_joints(solve_ik(pre_pos, target_quat))
95
96         # 2. Move to Grasp
97         move_to_joints(solve_ik(target_pos, target_quat))
98
99         # 3. Close
100         close_gripper()
101
102         # 4. Lift
103         lift_pos = target_pos + np.array([0, 0, 0.25])
104         move_to_joints(solve_ik(lift_pos, target_quat))
105         print("Retry finished.")
106     ...

```

F.2.2. PERCEPTION MODULE CASCADE FAILURE FALLBACKS

This case study comes from the "wipe up the brown spill" task from Robosuite ported into CaP-Gym. The model frequently constructs robust perception pipelines that utilize hierarchical fallbacks. It understands that a generic text prompt might fail or be imprecise. In several trials, the model employs a VLM (Molmo) to visually ground the target ("brown spill") into a specific 2D point, and then uses that point to prompt a segmentation model (SAM3) for a precise mask. If this high-precision

method fails, it falls back to broader text prompts like "stain" or "brown object".

```

1  ...
2
3  # 2. Try detection using Molmo (VLM) for better semantic understanding
4  print("Attempting to detect spill using Molmo...")
5  molmo_res = point_prompt_molmo(rgb, "brown spill")
6  # Molmo returns dict[str, tuple], getting the first/only value
7  spill_pt = None
8  if molmo_res:
9      # Check if we got a valid coordinate
10     for key, pt in molmo_res.items():
11         if pt[0] is not None and pt[1] is not None:
12             spill_pt = pt
13             print(f"Molmo found spill at: {spill_pt}")
14             break
15
16 masks = []
17 if spill_pt:
18     # Use the point to segment with SAM3
19     print(f"Segmenting with SAM3 point prompt at {spill_pt}")
20     masks = segment_sam3_point_prompt(rgb, spill_pt)
21 else:
22     # Fallback to text prompt with different wording if Molmo fails
23     print("Molmo failed. Trying SAM3 text prompt 'stain'...")
24     masks = segment_sam3_text_prompt(rgb, "stain")
25     if not masks:
26         print("Fallback to 'brown liquid'...")
27         masks = segment_sam3_text_prompt(rgb, "brown liquid")
28 if not masks:
29     print("No masks found after all attempts. Cannot proceed.")
30     return
31 ...

```

F.2.3. GEOMETRIC REASONING FOR GRASP SELECTION

This case study illustrates how CaP-Agent0 enhances sub-symbolic planners by injecting context-aware strategy. While modules like *Contact-GraspNet* infer many 6D grasp poses along with confidence scores, the learned scoring lacks awareness of the broader task setting. For this *tabletop* scenario, the agent correctly identified that a top-down approach strategy is inherently more reliable than side-approaches, regardless of their raw grasp scores.

To enforce this strategy, the agent autonomously synthesized a geometric constraint wrapper. By computing the alignment between grasp approach vectors and the world vertical, the generated code filtered out high-scoring but suboptimal side-grasps, explicitly selecting a vertical candidate to maximize stability. The agent also synthesized an autonomous fallback branch: if this strict strategy yielded no candidates, the logic reverted to the global maximum score to prevent execution stagnation. This demonstrates the agent’s capacity to wrap generic tools in context-specific logic, bridging the gap between raw geometric perception and high-level task semantics.

Crucially, the automatically synthesized skill library (Appendix H.1) contains an analogous grasp filter, serving as a primary example of the agent’s capacity to synthesize task-agnostic geometric logic into its persistent codebase.

```

1  ...
2
3  # 2. Grasp Planning
4  print("Planning grasp...")
5  grasp_poses, grasp_scores = plan_grasp(depth, intrinsics, seg_map)
6  if len(grasp_poses) == 0:
7      print("Error: No grasp poses found.")
8      return
9
10 # 3. Filter and Selection
11 # Transform all grasps to world frame to check orientation
12 best_score = -1.0
13 best_grasp_world = None
14 # We iterate through high-scoring grasps
15 indices = np.argsort(grasp_scores)[::-1] # Descending order
16 print(f"Found {len(indices)} candidates. Evaluating top 10...")
17 for idx in indices[:20]:
18     grasp_camera = grasp_poses[idx]
19     grasp_world = transform_camera_to_world(grasp_camera, camera_pose)

```

```

20     # Check approach vector (Z-axis of rotation matrix)
21     # We want the gripper to approach somewhat downwards (negative Z in world)
22     rot_mat = grasp_world[:3, :3]
23     approach_vec = rot_mat[:, 2] # The Z-axis
24     # Dot product with world down vector (0, 0, -1)
25     # If dot product is positive (angle < 90), it's pointing somewhat down.
26     # Actually, if approach is (0,0,-1), dot with (0,0,-1) is 1.
27     # We want approach_z to be negative.
28     if approach_vec[2] < -0.5: # At least 60 degrees downward tilt
29         print(f"Candidate {idx}: Score {grasp_scores[idx]:.3f}, Approach Z: {
approach_vec[2]:.3f} (Good)")
30         best_grasp_world = grasp_world
31         break
32     else:
33         # print(f"Candidate {idx}: Score {grasp_scores[idx]:.3f}, Approach Z: {
approach_vec[2]:.3f} (Skipping, too horizontal)")
34         pass
35     if best_grasp_world is None:
36         print("Warning: No steep top-down grasp found. Falling back to highest score.")
37         best_grasp_world = transform_camera_to_world(grasp_poses[indices[0]],
camera_pose)
38     target_pos = best_grasp_world[:3, 3]
39     target_rot = best_grasp_world[:3, :3]
40     target_quat = rotation_matrix_to_quaternion(target_rot)
41     ...

```

F.2.4. CODE GENERATION ENSEMBLING INCREASES ROBUSTNESS

In this case study, we analyze the differences in code generation for a VDM agent which uses a single query vs a model ensemble. All generated code is for the cube restack task. The following is a portion of the initial generation from single query.

```

1     ...
2     import numpy as np
3
4     # Step 1: Unstack the green cube from the red cube.
5     # Get grasp pose for the green cube (currently on top)
6     green_grasp_pos, green_grasp_quat = sample_grasp_pose('green cube')
7
8     # Approach and pick up the green cube
9     open_gripper()
10    goto_pose(green_grasp_pos, green_grasp_quat, z_approach=0.1)
11    close_gripper()
12    ...

```

Notice how there is no fallback for the `sample_grasp_pose()` generation. When the initial code was executed, the outputted grasp was invalid. The model reasoned that the grasp was invalid because it was too close to the origin, so in the subsequent generation, the model retroactively implemented a fallback which only prevents the failure case it just encountered.

```

1     ...
2     # Based on the logs, the previous execution seems to have proceeded through the
logic,
3     # but looking at the sampled grasp pose for the red cube: [ 0.04235514  0.08625826
-0.04337643]
4     # compared to the get_object_pose for red cube: [ 0.68666898  0.12055038
-0.09239209]
5     # The sampled grasp pose for the red cube seems completely wrong (close to robot
base origin 0,0,0).
6     # This likely caused the robot to fail picking up the red cube or crash/move
incorrectly in Step 2.
7     ...
8     # Get poses
9     red_pos, _, _ = get_object_pose('red cube')
10    green_pos, _, green_bbox = get_object_pose('green cube', return_bbox_extent=True)
11
12    # Sample grasp for red cube
13    red_grasp_pos, red_grasp_quat = sample_grasp_pose('red cube')
14

```

```

15 # Sanity check: if grasp pose is near 0,0,0, fall back to object pose + offset
16 if np.linalg.norm(red_grasp_pos) < 0.2:
17     # Fallback: use object position directly for grasp center, oriented top-down
18     print("Warning: Sampled grasp pose seems invalid. Using object position
19     fallback.")
20     red_grasp_pos = red_pos.copy()
21     red_grasp_quat = np.array([0, 0, 1, 0]) # Top down for fallback
22     ...

```

On the other hand, generations from a model ensemble tend to anticipate failure cases and preemptively implement fallbacks as shown through the following initial generation:

```

1 ...
2 def pick_object(mask, depth, intrinsics, extrinsics, z_lift=0.2):
3     """Plans and executes a grasp for the object defined by the mask."""
4     # Plan grasp
5     grasps, scores = plan_grasp(depth, intrinsics, mask.astype(np.int32))
6     best_grasp_world, _ = select_top_down_grasp(grasps, scores, extrinsics,
7     vertical_threshold=0.7)
8
9     if best_grasp_world is None:
10        # Fallback to centroid grasp if grasp planner fails
11        print("Grasp planner failed, falling back to centroid grasp.")
12        stats = get_object_stats(mask, depth, intrinsics, extrinsics)
13        if stats is None:
14            raise RuntimeError("Cannot compute object stats for fallback grasp.")
15
16        # Create a default top-down orientation
17        R_down = np.array([[1.0, 0.0, 0.0], [0.0, -1.0, 0.0], [0.0, 0.0, -1.0]])
18        quat = rotation_matrix_to_quaternion(R_down)
19        pos = stats["center"].copy()
20        pos[2] = stats["max_z"] - 0.02 # Grasp slightly below top
21    else:
22        pos, quat = decompose_transform(best_grasp_world)
23
24    # Execute sequence
25    pre_grasp = pos.copy()
26    pre_grasp[2] += 0.12 # 12cm above
27
28    open_gripper()
29    move_tcp(pre_grasp, quat)
30    move_tcp(pos, quat)
31    ...

```

G. Low-Level Perception and Control Primitives

This section provides the complete specifications for the low-level perception and control primitives used in CaP-Bench, including function signatures, interface definitions, and documentation strings. These primitives constitute the exact specifications provided to agents in Tier S3. Tier S4 utilizes an identical primitive set, but with in-context usage examples (the Example: sections within docstrings) stripped. Tiers S1 and S2 operate on high-level abstractions that are constructed by composing these fundamental primitives.

```

1 from typing import Any
2 import numpy as np
3 import open3d as o3d
4 import viser.transforms as vtf
5 from PIL import Image
6 from scipy.spatial.transform import Rotation as SciRotation
7 from capx.envs.base_env import BaseEnv
8 from capx.integrations import pyroki_snippets as pks # type: ignore
9 from capx.integrations.base_api import ApiBase
10 from capx.integrations.grasp_graspnet import init_contact_graspnet
11 from capx.integrations.molmo import init_molmo
12 from capx.integrations.pyroki import init_pyroki
13 from capx.integrations.sam3 import init_sam3, init_sam3_point_prompt
14
15 class S3(ApiBase):
16     """
17     Robot perception and control primitives for the S3 CaP-Bench Tier

```

```

18 """
19 def __init__(
20     self,
21     env: BaseEnv,
22     tcp_offset: list[float] | None = [0.0, 0.0, -0.107],
23     bimanual: bool = False,
24 ) -> None:
25     super().__init__(env)
26     self._TCP_OFFSET = np.array(tcp_offset, dtype=np.float64)
27     self.grasp_net_plan_fn = init_contact_graspnet()
28     self.sam3_seg_fn = init_sam3()
29     self.sam3_point_prompt_fn = init_sam3_point_prompt()
30     self.molmo_point_fn = init_molmo()
31
32     self.ik_solve_fn = init_pyroki()
33     self.trajopt_plan_fn = init_pyroki_trajopt()
34     self.cfg = None
35     self.bimanual = bimanual
36
37 def functions(self) -> dict[str, Any]:
38     fns = {
39         "get_observation": self.get_observation,
40         "segment_sam3_text_prompt": self.segment_sam3_text_prompt,
41         "segment_sam3_point_prompt": self.segment_sam3_point_prompt,
42         "point_prompt_molmo": self.point_prompt_molmo,
43         "plan_grasp": self.plan_grasp,
44         "get_oriented_bounding_box_from_3d_points": self.
45     }
46     if self.bimanual:
47         fns["solve_ik_arm0"] = self.solve_ik_arm0
48         fns["solve_ik_arm1"] = self.solve_ik_arm1
49         fns["move_to_joints_both"] = self.move_to_joints_both
50         fns["move_to_joints_arm0"] = self.move_to_joints_arm0
51         fns["move_to_joints_arm1"] = self.move_to_joints_arm1
52         fns["open_gripper_arm0"] = self.open_gripper_arm0
53         fns["close_gripper_arm0"] = self.close_gripper_arm0
54         fns["open_gripper_arm1"] = self.open_gripper_arm1
55         fns["close_gripper_arm1"] = self.close_gripper_arm1
56     else:
57         fns["solve_ik"] = self.solve_ik
58         fns["move_to_joints"] = self.move_to_joints
59         fns["open_gripper"] = self.open_gripper
60         fns["close_gripper"] = self.close_gripper
61
62     return fns
63
64 def get_observation(self) -> dict[str, Any]:
65     """Get the observation of the environment.
66     Returns:
67         observation:
68             A dictionary containing the observation of the environment.
69             The dictionary contains the following keys:
70             - ["robot0_robotview"]["images"]["rgb"]: Current color camera image as
71             a numpy array of shape (H, W, 3), dtype uint8.
72             - ["robot0_robotview"]["images"]["depth"]: Current depth camera image
73             as a numpy array of shape (H, W, 1), dtype float32.
74             - ["robot0_robotview"]["intrinsicss"]: Camera intrinsic matrix as a
75             numpy array of shape (3, 3), dtype float64.
76             - ["robot0_robotview"]["pose_mat"]: Camera extrinsic matrix as a numpy
77             array of shape (4, 4), dtype float64.
78
79     """
80     return self._env.get_observation()
81
82 # ----- #
83 # Vision models: Sam3 segmentation ----- #
84
85 def segment_sam3_point_prompt(
86     self,
87     rgb: np.ndarray,
88     point_coords: tuple[float, float],

```

```

86 ) -> list[dict[str, Any]]:
87     """Run SAM3 segmentation on an RGB image, optionally conditioned on an image
    coordinate point prompt.
88
89     Args:
90         rgb:
91             RGB image array of shape (H, W, 3), dtype uint8.
92         point_coords:
93             (x, y) pixel coordinates of the point prompt.
94
95     Returns:
96         masks:
97             A list of dictionaries. Each dict may contain:
98
99             - "mask": np.ndarray of shape (H, W), dtype bool or uint8,
100                       where True/1 means the pixel belongs to the instance.
101             - "score": float confidence score.
102
103     Example:
104         >>> rgb = obs["robot0_robotview"]["images"]["rgb"]
105         >>> masks = segment_sam3_point_prompt(rgb, (100, 100))
106     """
107     return self.sam3_point_prompt_fn(Image.fromarray(rgb), point_coords)
108
109 def segment_sam3_text_prompt(
110     self,
111     rgb: np.ndarray,
112     text_prompt: str,
113 ) -> list[dict[str, Any]]:
114     """Run SAM3 segmentation on an RGB image conditioned on a text prompt.
115
116     Args:
117         rgb:
118             RGB image array of shape (H, W, 3), dtype uint8.
119         text_prompt:
120             Text prompt for SAM3 segmentation.
121
122     Returns:
123         masks:
124             A list of dictionaries. Each dict may contain:
125
126             - "mask": np.ndarray of shape (H, W), dtype bool or uint8,
127                       where True/1 means the pixel belongs to the instance.
128             - "box": list [x1, y1, x2, y2] in pixel coordinates.
129             - "score": float confidence score.
130
131     Example:
132         >>> rgb = obs["robot0_robotview"]["images"]["rgb"]
133         >>> masks = segment_sam3(rgb, text_prompt="red mug")
134     """
135     return self.sam3_seg_fn(rgb, text_prompt=text_prompt)
136
137 # ----- #
138 # Molmo point prompt
139 # ----- #
140 def point_prompt_molmo(
141     self,
142     image: np.ndarray,
143     text_prompt: str,
144 ) -> dict[str, tuple[int | None, int | None]]:
145     """Use Molmo to point to a coordinate in the image based on a text prompt.
146
147     Args:
148         image: np.ndarray: The RGB image to process. Shape: (H, W, 3), dtype uint8.
149         text_prompt: str: The text prompt to point to.
150
151     Returns:
152         dict[str, tuple[int | None, int | None]]: Pixel coordinates for each
153         object query; (None, None) if parsing failed.
154     """
155     return self.molmo_point_fn(Image.fromarray(image), objects=[text_prompt])
156
157 def get_oriented_bounding_box_from_3d_points(self, points: np.ndarray) -> dict[str,

```

```

Any]:
158     """Get the oriented bounding box from 3D points.
159
160     Args:
161         points: np.ndarray: The 3D points to get the oriented bounding box from.
162             Shape: (N, 3), dtype float64.
163
164     Returns:
165         dict[str, Any]: The oriented bounding box. The dictionary contains the
166         following keys:
167             - "center": np.ndarray: The center of the oriented bounding box in
168             point cloud frame.
169             - "extent": np.ndarray: The extent of the oriented bounding box.
170             - "R": np.ndarray: The rotation matrix of the oriented bounding box in
171             point cloud frame.
172     """
173     o3d_points = o3d.geometry.PointCloud()
174     o3d_points.points = o3d.utility.Vector3dVector(points)
175     o3d_points, ind = o3d_points.remove_statistical_outlier(nb_neighbors=20,
176     std_ratio=2.0)
177     obb = o3d_points.get_oriented_bounding_box()
178     return {
179         "center": obb.center,
180         "extent": obb.extent,
181         "R": obb.R,
182     }
183
184 # ----- #
185 # Grasp planner (Contact-GraspNet)
186 # ----- #
187
188 def plan_grasp(
189     self,
190     depth: np.ndarray,
191     intrinsics: np.ndarray,
192     segmentation: np.ndarray,
193 ) -> tuple[np.ndarray, np.ndarray]:
194     """Plan grasp candidates using Contact-GraspNet for a single instance.
195
196     This is a thin wrapper around the Contact-GraspNet planner. It does not
197     apply any camera/world transforms or TCP offsets: the caller is
198     responsible for transforming the resulting grasp poses into the desired
199     frame and applying TCP offsets if necessary.
200
201     Args:
202         depth:
203             Depth image in meters.
204             Shape: (H, W) or (H, W, 1), dtype float32/float64.
205         intrinsics:
206             Camera intrinsic matrix.
207             Shape: (3, 3), dtype float64.
208         segmentation:
209             Instance segmentation map where each integer > 0 corresponds to a
210             unique object instance ID.
211             Shape: (H, W) or (H, W, 1), dtype int32/int64.
212
213     Returns:
214         grasp_poses:
215             np.ndarray of shape (K, 4, 4), dtype float64.
216             Homogeneous transforms for each candidate grasp IN THE CAMERA FRAME.
217         grasp_scores:
218             np.ndarray of shape (K,), dtype float64.
219             Confidence score for each candidate grasp.
220
221     Example:
222     """
223     >>> cam = obs["robot0_robotview"]
224     >>> rgb = cam["images"]["rgb"]
225     >>> depth = cam["images"]["depth"][:, :, 0]
226     >>> sam3_results = sam3_seg_fn(rgb, text_prompt="red mug")
227     >>> best = max(sam3_results, key=lambda d: d["score"])
228     >>> mask = best["mask"]
229     >>> K = cam["intrinsics"]
230     >>> grasp_sample_tf, grasp_scores = plan_grasp(
231         ...     depth=depth,

```

```

226         ...     intrinsics=K,
227         ...     segmentation=mask,
228         ... )
229     >>> best_idx = grasp_scores.argmax()
230     >>> best_T = grasp_poses[best_idx] # (4, 4)
231     >>> camera_extrinsics = cam["pose_mat"]
232     >>> grasp_sample_world_frame = camera_extrinsics @ best_T
233     """
234     if depth.ndim == 3 and depth.shape[-1] == 1:
235         depth = depth[:, :, 0]
236     if segmentation.ndim == 3 and segmentation.shape[-1] == 1:
237         segmentation = segmentation[:, :, 0]
238
239     grasp_sample, grasp_scores, _ = self.grasp_net_plan_fn(
240         depth,
241         intrinsics,
242         segmentation,
243         1,
244         z_range=[0.2, 3.5] if self.is_handover else [0.2, 2.0],
245         forward_passes=1 if self.is_handover else 3,
246     )
247
248     grasp_sample_tf = (
249         0, 0.12]))
250         vtf.SE3.from_matrix(grasp_sample) @ vtf.SE3.from_translation(np.array([0,
251         0, 0.12]))
252         ).as_matrix()
253
254     return grasp_sample_tf, grasp_scores
255
256     # ----- #
257     # IK / motion primitives
258     # ----- #
259
260     def solve_ik(
261         self,
262         position: np.ndarray,
263         quaternion_wxyz: np.ndarray,
264     ) -> np.ndarray:
265         """Solve inverse kinematics for the panda_hand link.
266
267         Args:
268             position:
269                 Target position in world frame.
270                 Shape: (3,), dtype float64.
271             quaternion_wxyz:
272                 Target orientation as a unit quaternion in world frame.
273                 Shape: (4,), [w, x, y, z], dtype float64.
274
275         Returns:
276             joints:
277                 np.ndarray of shape (7,), dtype float64.
278                 Joint angles for the 7 DoF Franka arm.
279
280         Example:
281             >>> target_pos = np.array([0.5, 0.0, 0.3])
282             >>> target_quat = np.array([1.0, 0.0, 0.0, 0.0]) # identity, wxyz
283             >>> joints = solve_ik(target_pos, target_quat)
284             >>> move_to_joints(joints)
285
286         """
287         pos = np.asarray(position, dtype=np.float64).reshape(3)
288         quat_wxyz = np.asarray(quaternion_wxyz, dtype=np.float64).reshape(4)
289         quat_xyzw = np.array(
290             [quat_wxyz[1], quat_wxyz[2], quat_wxyz[3], quat_wxyz[0]], dtype=np.float64
291         )
292         rot = SciRotation.from_quat(quat_xyzw)
293         offset_pos = pos + rot.apply(self._TCP_OFFSET)
294
295         prev_cfg = self.cfg
296
297         for i in range(15): # run w/ multiple iterations when using vel_cost ik solver
298             self.cfg = self.ik_solve_fn(
299                 target_pose_wxyz_xyz=np.concatenate([quat_wxyz, offset_pos]),
300                 prev_cfg=prev_cfg,
301             )

```

```

298         if prev_cfg is not None:
299             if np.allclose(self.cfg, prev_cfg, atol=1e-3):
300                 break
301             else:
302                 prev_cfg = self.cfg
303         joints = np.asarray(self.cfg[:-1], dtype=np.float64).reshape(7)
304         return joints
305
306     # Single arm control APIs
307
308     def move_to_joints(self, joints: np.ndarray) -> None:
309         """Move the robot to a given joint configuration in a blocking manner.
310
311         Args:
312             joints:
313                 Target joint angles for the 7-DoF Franka arm.
314                 Shape: (7,), dtype float64.
315
316         Returns:
317             None
318
319         Example:
320             >>> joints = np.array([0.0, -0.5, 0.0, -2.0, 0.0, 1.5, 0.8])
321             >>> move_to_joints(joints)
322             """
323         joints = np.asarray(joints, dtype=np.float64).reshape(7)
324         self._env.move_to_joints_blocking(joints)
325
326     def open_gripper(self) -> None:
327         """Open gripper fully.
328
329         Args:
330             None
331             """
332         self._env._set_gripper(1.0)
333         for _ in range(30):
334             self._env._step_once()
335
336     def close_gripper(self) -> None:
337         """Close gripper fully.
338
339         Args:
340             None
341             """
342         self._env._set_gripper(0.0)
343         for _ in range(30):
344             self._env._step_once()
345
346     # Dual arm control APIs
347     def move_to_joints_both(self, joints0: np.ndarray, joints1: np.ndarray) -> None:
348         """Move the arms 0 and 1 to a given joint configuration in a blocking manner
349         simultaneously.
350
351         Args:
352             joints0:
353                 Target joint angles for the 7-DoF Franka arm 0.
354                 Shape: (7,), dtype float64.
355             joints1:
356                 Target joint angles for the 7-DoF Franka arm 1.
357                 Shape: (7,), dtype float64.
358             """
359         self._env.move_to_joints_blocking_both(joints0, joints1)
360
361     def move_to_joints_arm0(self, joints: np.ndarray) -> None:
362         """Move the robot arm 0 to a given joint configuration in a blocking manner.
363
364         Args:
365             joints:
366                 Target joint angles for the 7-DoF Franka arm 0.
367                 Shape: (7,), dtype float64.
368             """
369         joints = np.asarray(joints, dtype=np.float64).reshape(7)

```

```

370         self._env.move_to_joints_blocking(joints)
371
372     def move_to_joints_arm1(self, joints: np.ndarray) -> None:
373         """Move the robot arm 1 to a given joint configuration in a blocking manner.
374
375         Args:
376             joints:
377                 Target joint angles for the 7-DoF Franka arm 1.
378                 Shape: (7,), dtype float64.
379         """
380         joints = np.asarray(joints, dtype=np.float64).reshape(7)
381         self._env.move_to_joints_blocking_arm1(joints)
382
383     def open_gripper_arm0(self) -> None:
384         """Open gripper fully for Arm 0 (robot0).
385         Args:
386             None
387         Returns:
388             None
389         """
390         self._env._set_gripper(1.0)
391         for _ in range(30):
392             self._env._step_once()
393
394     def close_gripper_arm0(self) -> None:
395         """Close gripper fully for Arm 0 (robot0).
396         Args:
397             None
398         Returns:
399             None
400         """
401         self._env._set_gripper(0.0)
402         for _ in range(30):
403             self._env._step_once()
404     def open_gripper_arm1(self) -> None:
405         """Open gripper fully for Arm 1 (robot1).
406         Args:
407             None
408         Returns:
409             None
410         """
411         self._env._set_gripper_arm1(1.0)
412         for _ in range(30):
413             self._env._step_once()
414
415     def close_gripper_arm1(self) -> None:
416         """Close gripper fully for Arm 1 (robot1).
417         Args:
418             None
419         Returns:
420             None
421         """
422         self._env._set_gripper_arm1(0.0)
423         for _ in range(30):
424             self._env._step_once()
425
426     def solve_ik_arm0(self, position: np.ndarray, quaternion_wxyz: np.ndarray) -> np.
427         ndarray:
428         """Solve inverse kinematics for the panda_hand link for Arm 0 (robot0)."""
429         pos = np.asarray(position, dtype=np.float64).reshape(3)
430         quat_wxyz = np.asarray(quaternion_wxyz, dtype=np.float64).reshape(4)
431         quat_xyzw = np.array(
432             [quat_wxyz[1], quat_wxyz[2], quat_wxyz[3], quat_wxyz[0]], dtype=np.float64
433         )
434         rot = SciRotation.from_quat(quat_xyzw)
435         offset_pos = pos + rot.apply(self._TCP_OFFSET)
436
437         prev_cfg = self.cfg
438         for i in range(15):
439             self.cfg = self.ik_solve_fn(
440                 target_pose_wxyz_xyz=np.concatenate([quat_wxyz, offset_pos]),
441                 prev_cfg=prev_cfg,

```

```

442         if prev_cfg is not None:
443             if np.allclose(self.cfg, prev_cfg, atol=1e-3):
444                 break
445             else:
446                 prev_cfg = self.cfg
447
448         joints = np.asarray(self.cfg[:-1], dtype=np.float64).reshape(7)
449         return joints
450
451     def solve_ik_arm1(self, position: np.ndarray, quaternion_wxyz: np.ndarray) -> np.
452     ndarray:
453         """Solve inverse kinematics for the panda_hand link for Arm 1 (robot1)."""
454         if not hasattr(self._env, "move_to_joints_blocking_arm1"):
455             raise RuntimeError("Environment does not support Arm 1 control")
456
457         if not hasattr(self._env, "base_link_wxyz_xyz_0") or not hasattr(
458             self._env, "base_link_wxyz_xyz_1"
459         ):
460             raise RuntimeError("Environment does not provide base transforms.")
461
462         pose_arm0_base = vtf.SE3.from_rotation_and_translation(
463             rotation=vtf.SO3(wxyz=quaternion_wxyz),
464             translation=position,
465         )
466         base0_transform = vtf.SE3(wxyz_xyz=self._env.base_link_wxyz_xyz_0)
467         pose_world = base0_transform @ pose_arm0_base
468
469         base1_transform = vtf.SE3(wxyz_xyz=self._env.base_link_wxyz_xyz_1)
470         base1_transform_inv = base1_transform.inverse()
471         pose_arm1_base = base1_transform_inv @ pose_world
472
473         pos = np.asarray(pose_arm1_base.translation(), dtype=np.float64).reshape(3)
474         quat_wxyz = np.asarray(pose_arm1_base.rotation().wxyz, dtype=np.float64).
475         reshape(4)
476         quat_xyzw = np.array(
477             [quat_wxyz[1], quat_wxyz[2], quat_wxyz[3], quat_wxyz[0]], dtype=np.float64
478         )
479         rot = SciRotation.from_quat(quat_xyzw)
480         offset_pos = pos + rot.apply(self._TCP_OFFSET)
481
482         prev_cfg = self.cfg
483         for i in range(15):
484             self.cfg = self.ik_solve_fn(
485                 target_pose_wxyz_xyz=np.concatenate([quat_wxyz, offset_pos]),
486                 prev_cfg=prev_cfg,
487             )
488             if prev_cfg is not None:
489                 if np.allclose(self.cfg, prev_cfg, atol=1e-3):
490                     break
491                 else:
492                     prev_cfg = self.cfg
493         joints = np.asarray(self.cfg[:-1], dtype=np.float64).reshape(7)
494         return joints

```

H. CaP-Agent0 Details

H.1. Synthesized Task-Agnostic Skill Library

In this section, we list all nine agent-synthesized function definitions, interfaces, documentation strings, and low-level implementations added to the CaP-Agent0 skill library.

```

1  def rotation_matrix_to_quaternion(R: np.ndarray) -> np.ndarray:
2      """
3      Convert a 3x3 rotation matrix to a unit quaternion [w, x, y, z].
4
5      Implements the robust Sheppard's method (checking trace and diagonal elements)
6      to avoid numerical instability when the trace is close to zero.
7      Args:
8          R: (3, 3) rotation matrix.
9      Returns:
10         np.array: [w, x, y, z] unit quaternion.

```

```

11     """
12     tr = np.trace(R)
13     if tr > 0:
14         S = np.sqrt(tr + 1.0) * 2
15         w = 0.25 * S
16         x = (R[2, 1] - R[1, 2]) / S
17         y = (R[0, 2] - R[2, 0]) / S
18         z = (R[1, 0] - R[0, 1]) / S
19     elif (R[0, 0] > R[1, 1]) and (R[0, 0] > R[2, 2]):
20         S = np.sqrt(1.0 + R[0, 0] - R[1, 1] - R[2, 2]) * 2
21         w = (R[2, 1] - R[1, 2]) / S
22         x = 0.25 * S
23         y = (R[0, 1] + R[1, 0]) / S
24         z = (R[0, 2] + R[2, 0]) / S
25     elif R[1, 1] > R[2, 2]:
26         S = np.sqrt(1.0 + R[1, 1] - R[0, 0] - R[2, 2]) * 2
27         w = (R[0, 2] - R[2, 0]) / S
28         x = (R[0, 1] + R[1, 0]) / S
29         y = 0.25 * S
30         z = (R[1, 2] + R[2, 1]) / S
31     else:
32         S = np.sqrt(1.0 + R[2, 2] - R[0, 0] - R[1, 1]) * 2
33         w = (R[1, 0] - R[0, 1]) / S
34         x = (R[0, 2] + R[2, 0]) / S
35         y = (R[1, 2] + R[2, 1]) / S
36         z = 0.25 * S
37     return np.array([w, x, y, z])
38
39 def decompose_transform(T: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
40     """
41     Decompose a 4x4 homogeneous transformation matrix into position and quaternion.
42     Args:
43         T: (4, 4) homogeneous transformation matrix.
44     Returns:
45         tuple:
46             - position: (3,) np.array
47             - quaternion: (4,) np.array [w, x, y, z]
48     """
49     position = T[:3, 3]
50     R = T[:3, :3]
51     quat = rotation_matrix_to_quaternion(R)
52     return position, quat
53
54 def depth_to_point_cloud(depth_img: np.ndarray, intrinsics: np.ndarray) -> np.
55 ndarray:
56     """
57     Convert a depth image to a 3D point cloud in the Camera Frame.
58     Args:
59         depth_img: (H, W) depth map in meters.
60         intrinsics: (3, 3) camera intrinsic matrix.
61     Returns:
62         np.array: (H, W, 3) image of 3D coordinates.
63     """
64     if depth_img.ndim == 3:
65         depth_img = depth_img[:, :, 0]
66
67     h, w = depth_img.shape
68     fx = intrinsics[0, 0]
69     fy = intrinsics[1, 1]
70     cx = intrinsics[0, 2]
71     cy = intrinsics[1, 2]
72
73     # Vectorized grid generation
74     y_grid, x_grid = np.mgrid[0:h, 0:w]
75
76     z = depth_img
77     x = (x_grid - cx) * z / fx
78     y = (y_grid - cy) * z / fy
79
80     return np.dstack((x, y, z))
81
82 def mask_to_world_points(
83     mask: np.ndarray, depth: np.ndarray, intrinsics: np.ndarray, extrinsics: np.

```

```

ndarray
) -> np.ndarray:
    """
    Convert specific pixels defined by a binary mask into 3D points in the World
    Frame.
    Args:
        mask: (H, W) binary mask (0 or 1).
        depth: (H, W) depth map.
        intrinsics: (3, 3) camera intrinsics.
        extrinsics: (4, 4) camera-to-world pose matrix.
    Returns:
        np.array: (N, 3) array of valid 3D points in world coordinates.
    """
    # Get pixel coordinates
    ys, xs = np.where(mask > 0)
    if len(ys) == 0:
        return np.empty((0, 3))

    z_vals = depth[ys, xs]

    # Filter invalid depth
    valid = z_vals > 0
    ys = ys[valid]
    xs = xs[valid]
    z = z_vals[valid]

    fx = intrinsics[0, 0]
    fy = intrinsics[1, 1]
    cx = intrinsics[0, 2]
    cy = intrinsics[1, 2]

    # Deproject to Camera Frame
    x_cam = (xs - cx) * z / fx
    y_cam = (ys - cy) * z / fy

    # Stack to (N, 3)
    points_cam = np.stack([x_cam, y_cam, z], axis=-1)

    # Transform to World Frame
    # Create homogeneous coordinates (N, 4)
    points_cam_hom = np.hstack([points_cam, np.ones((len(points_cam), 1))])
    points_world_hom = (extrinsics @ points_cam_hom.T).T

    return points_world_hom[:, :3]

def pixel_to_world_point(
    u: int, v: int, z: float, intrinsics: np.ndarray, extrinsics: np.ndarray
) -> np.ndarray:
    """
    Deproject a single pixel to a 3D world point.
    Args:
        u, v: Pixel coordinates (col, row).
        z: Depth at that pixel.
        intrinsics: (3, 3) matrix.
        extrinsics: (4, 4) matrix.
    Returns:
        np.array: [x, y, z] in world frame.
    """
    fx = intrinsics[0, 0]
    fy = intrinsics[1, 1]
    cx = intrinsics[0, 2]
    cy = intrinsics[1, 2]

    x_cam = (u - cx) * z / fx
    y_cam = (v - cy) * z / fy

    p_cam = np.array([x_cam, y_cam, z, 1.0])
    p_world = extrinsics @ p_cam
    return p_world[:3]

def transform_points(points: np.ndarray, transform_matrix: np.ndarray) -> np.
ndarray:
    """

```

```

153     Apply a 4x4 homogeneous transform to a set of 3D points.
154     Args:
155         points: (N, 3) or (H, W, 3) array of points.
156         transform_matrix: (4, 4) homogeneous transformation matrix.
157     Returns:
158         np.array: Transformed points with same shape as input.
159     """
160     original_shape = points.shape
161     # Flatten to (N, 3)
162     points_reshaped = points.reshape(-1, 3)
163
164     # Convert to homogeneous (N, 4)
165     ones = np.ones((points_reshaped.shape[0], 1))
166     points_hom = np.hstack((points_reshaped, ones))
167
168     # Apply transform: (4,4) @ (4,N) -> (4,N) -> Transpose back to (N,4)
169     points_transformed = (transform_matrix @ points_hom.T).T
170
171     # Return to (N, 3) and original shape
172     return points_transformed[:, :3].reshape(original_shape)
173
174 def interpolate_segment(
175     p1: np.ndarray, p2: np.ndarray, step: float = 0.03
176 ) -> list[np.ndarray]:
177     """
178     Generate waypoints along a line segment between two 3D points.
179     Args:
180         p1: Start point (3,).
181         p2: End point (3,).
182         step: Distance between waypoints in meters.
183     Returns:
184         list[np.ndarray]: List of points including p1 and p2.
185     """
186     dist = np.linalg.norm(p2 - p1)
187     if dist < 1e-6:
188         return [p1]
189     num_points = int(np.ceil(dist / step))
190     # Using linspace to ensure we hit the start and end exactly
191     return [p1 + (p2 - p1) * t for t in np.linspace(0, 1, num_points + 1)]
192
193 def normalize_vector(v: np.ndarray) -> np.ndarray:
194     """
195     Normalize a vector to unit length.
196     Args:
197         v: (3,) vector.
198     Returns:
199         np.array: (3,) unit vector.
200     """
201     norm = np.linalg.norm(v)
202     if norm < 1e-6:
203         return v
204     return v / norm
205
206 def select_top_down_grasp(
207     grasps: np.ndarray,
208     scores: np.ndarray,
209     cam_to_world: np.ndarray,
210     vertical_threshold: float = 0.8,
211 ) -> tuple:
212     """
213     Selects the best grasp that aligns the gripper vertically (Top-Down).
214     Args:
215         grasps: (N, 4, 4) Grasp poses in camera frame.
216         scores: (N,) Grasp scores.
217         cam_to_world: (4, 4) Extrinsic matrix.
218         vertical_threshold: Dot product threshold (1.0 is perfectly vertical).
219     Returns:
220         tuple: (best_grasp_world_matrix, best_score) or (None, -inf)
221     """
222     best_grasp = None
223     best_score = -np.float64("inf")
224
225     # World Z axis (vertical)

```

```

226     world_z = np.array([0, 0, 1])
227
228     for i, g_camera in enumerate(grasps):
229         # Transform grasp to world frame
230         g_world = cam_to_world @ g_camera
231
232         # Extract rotation
233         R = g_world[:3, :3]
234
235         # Assuming Gripper Z or Y is the approach vector depending on gripper
236         # definition.
237         # For Franka/Robotiq, the approach vector is usually the Z-axis of the end
238         # effector.
239         gripper_approach = R[:, 2]
240
241         # Check alignment with negative World Z (pointing down)
242         # Dot product should be close to -1 for top-down
243         alignment = -np.dot(gripper_approach, world_z)
244
245         if alignment > vertical_threshold:
246             if scores[i] > best_score:
247                 best_score = scores[i]
248                 best_grasp = g_world
249
250     return best_grasp, best_score

```

H.2. Model Ensemble Temperature Details

In both single and multi-model settings, 9 candidates responses are generated. For single model, we query Gemini-3-Pro 9 times with temperatures 0.1, 0.2, 0.3,..., 0.9. For multi-model, we query Gemini-3-Pro, Claude-Opus-4.5, and GPT-5.2 3 times each with temperatures 0.1, 0.5, and 0.9.

H.3. Model Ensemble Prompt

This section contains the prompts the coding agent uses to generate the final solution from the candidates for both the initial generation and subsequent multi-turn attempts. Text inside curly braces “{}” represent fstring placeholders.

User prompt for both initial and subsequent generations:

```

1     Synthesize the best solution.
2
3     <original_task_description>
4     {original prompt for candidates}
5     </original_task_description>
6
7     <candidate_solutions>
8     {candidate generations}
9     </candidate_solutions>

```

System prompt for initial generation:

```

1     You are synthesizing {# of candidate generations} candidate Python solutions into
2     one optimal program.
3
4     SYNTHESIS RULES:
5     1. Analyze critically and assume no candidate is fully correct
6     2. Prefer explicit checks over assumptions
7     3. Combine the best ideas from multiple candidates when appropriate
8     4. If candidates disagree fundamentally, choose the more robust approach
9
10    OUTPUT FORMAT (strict):
11    You may include reasoning before the fenced code block.
12    Output ONLY ONE fenced code block (``python...``) containing the complete final
13    solution.
14    Do NOT include any other code blocks or code snippets outside this single block.

```

System prompt for subsequent generations:

```

1  You are synthesizing {# of generations} candidate responses for a multi-turn robot
2  control task.
3
4  DECISION ANALYSIS:
5  - {regenerate_count} candidates voted REGENERATE
6  - {finish_count} candidates voted FINISH
7
8  SYNTHESIS RULES:
9  1. Analyze critically and assume no candidate is fully correct
10 2. Prefer explicit checks over assumptions
113. Combine the best ideas from multiple candidates when appropriate
124. If candidates disagree fundamentally, choose the more robust approach
135. Combine best code ideas from REGENERATE candidates
14
15 OUTPUT FORMAT (strict):
16 - You may include brief reasoning first
17 - Then output "REGENERATE" on its own line followed by exactly ONE fenced code
18 block, OR output "FINISH" on its own line

```

H.4. Multi-turn prompt incentivizing debugging

We noticed that one failure case was inaccurate verification of task completion. Therefore, we experimented with a modified multi-turn prompt which incentivizes verification and debugging, however this did not empirically improve success rate.

	Cube Lift	Cube Stack	Spill Wipe	Peg Insert	Cube Restack	Two Arm Lift	Two Arm Handover	Avg.
3M	97	98	100	0	89	74	20	68.29
3M + debug	94	100	98	0	88	66	12	65.43

Table 5. Task completions between 3M and 3M + debug.

Modified multi-turn prompt:

```

1  You are acting as experienced debugger and task completion verifier. You will make
2  no assumptions without explicit evidence.
3  Your task is to determine whether the program has actually completed the
4  intended task, and to fix all bugs if any exist.
5  You can treat the observed differences between the current and previous state of
6  the environment that are provided to you as a source of evidence, but they are not
7  guaranteed to be accurate.
8
9  The following code blocks have been executed so far:
10  ```python
11  {executed_code}
12  ```
13  The current console stdout from the most recent code execution is:
14  ```
15  {console_stdout}
16  ```
17  The current console stderr from the most recent code execution is:
18  ```
19  {console_stderr}
20  ```
21
22  Do NOT trust printed outputs or logs blindly.
23
24  Proceed as follows to verify task completion:
25  1. Line-by-line inspect the executed code
26  2. Cross-check the code's behavior against the console stdout and stderr
27  3. Identify explicit, concrete evidence that each required step of the task was
28  completed
29  4. Treat missing evidence, implicit assumptions, or partial signals as failure
30
31  Decision Rules:
32  - If the task is verifiably complete, respond with the single word 'FINISH'.
33  - If the task is not verifiably complete:
34  1. Identify specific bugs, failures, missing steps, or incorrect assumptions.

```

```

30     2. Explain why each issue prevents task completion, citing evidence from
31     stdout, stderr, or code behavior.
32     3. If the same approach has been attempted multiple times without success, you
33     MUST try a fundamentally different strategy.
34     4. Brainstorm multiple strategies to fix or verify these issues in the next
35     code generation.
36     5. Synthesize your brainstorming into a single improved Python program that
37     addresses all identified issues and verifiably completes the task.
38     6. Respond with the single word 'REGENERATE' followed immediately by new
39     Python code in a fenced code block (```python...```)
40
41     If you choose to regenerate code, you must include your reasoning process only
42     AS COMMENTS at the start of the code explaining:
43     - What strategies/APIs have already been tried and why they were insufficient
44     - How your solution effectively addresses the identified issues
45
46     To reiterate, you must only respond with EXACTLY ONE of the following:
47     - The word 'FINISH' if you decide to stop generating code
48     - The word 'REGENERATE' followed immediately by new Python code in a SINGLE
49     fenced code block (```python...```).

```

H.5. Model Ensemble Decreases Average Turn Count

We observed that applying a model ensemble for code generations decreases the average turn count. We noticed that M4 tends to retroactively implement bug fixes and API fallbacks, while the model ensemble preemptively anticipates failures beforehand, resulting in more robust code and lower turn count. See Appendix F.2.4 for a case study on retroactive vs preemptive behavior.

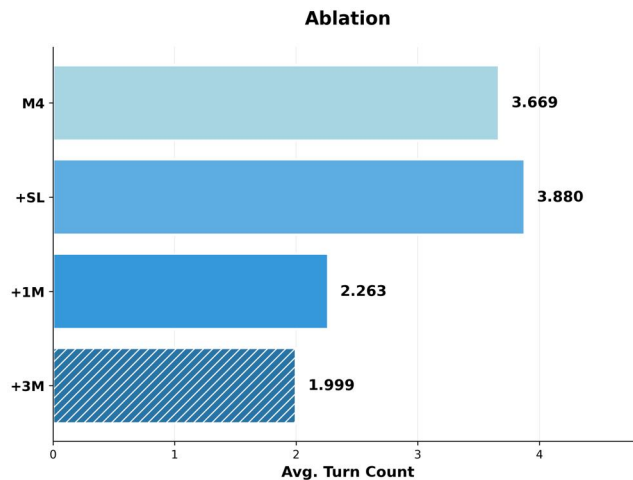


Figure 23. CaP-Agent0 decreases average turn count compared to M4

I. Select Case Studies - Full Generated Code

I.1. Stack These as High as You Can

Generated by Gemini-3-Pro as the coding agent and Gemini-3-Pro as the VDM agent.

```

1 import numpy as np
2
3 def get_best_mask(masks):
4     if not masks:
5         return None
6     return max(masks, key=lambda x: x["score"])
7
8 def get_object_metrics(mask, depth, intrinsics, extrinsics):
9     """Returns center (x,y,z), top_z, bottom_z for a given object mask."""
10    points = mask_to_world_points(mask, depth, intrinsics, extrinsics)
11    if points.shape[0] == 0:

```

```

12         return None, None, None
13
14     # Get Oriented Bounding Box
15     bbox = get_oriented_bounding_box_from_3d_points(points)
16     center = bbox["center"]
17
18     # Calculate robust top and bottom Z from points
19     zs = points[:, 2]
20     if len(zs) == 0:
21         return None, None, None
22
23     top_z = np.percentile(zs, 98) # Use 98th percentile to filter outliers
24     bottom_z = np.percentile(zs, 2) # Use 2nd percentile
25
26     return center, top_z, bottom_z
27
28 def execute_pose_move(pose_matrix):
29     """Moves robot to the specified 4x4 pose matrix."""
30     pos, quat = decompose_transform(pose_matrix)
31     joints = solve_ik(pos, quat)
32     move_to_joints(joints)
33
34 def pick_and_place(obj_mask, target_center, target_top_z, depth, intrinsics, cam_pose):
35     """
36     Picks object defined by obj_mask and places it on target defined by target_center
37     and target_top_z.
38     """
39     # 1. Analyze Object to be picked
40     obj_center, obj_top, obj_bottom = get_object_metrics(obj_mask, depth, intrinsics,
41     cam_pose)
42     if obj_center is None:
43         print("Could not calculate object metrics.")
44         return
45
46     # 2. Plan Grasp
47     seg_map = obj_mask.astype(np.int32)
48     # plan_grasp returns poses in CAMERA FRAME
49     grasp_poses_cam, grasp_scores = plan_grasp(depth, intrinsics, seg_map)
50
51     if len(grasp_poses_cam) == 0:
52         print("No grasps generated by planner.")
53         return
54
55     # Select best top-down grasp.
56     # select_top_down_grasp takes grasps in CAMERA FRAME and returns the best in WORLD
57     # FRAME.
58     best_grasp_world, best_score = select_top_down_grasp(grasp_poses_cam, grasp_scores,
59     cam_pose, vertical_threshold=0.8)
60
61     if best_grasp_world is None:
62         print("No strictly vertical grasp found. Trying relaxed threshold.")
63         best_grasp_world, best_score = select_top_down_grasp(grasp_poses_cam,
64         grasp_scores, cam_pose, vertical_threshold=0.5)
65
66     if best_grasp_world is None:
67         print("No valid grasp found even with relaxed threshold.")
68         return
69
70     print(f"Executing grasp with score {best_score}")
71
72     # Calculate offset from TCP to bottom of object
73     # TCP Z at grasp - Object Bottom Z
74     grasp_z = best_grasp_world[2, 3]
75     tcp_to_bottom_offset = grasp_z - obj_bottom
76
77     # 3. Execute Pick
78     # Move to Pre-Grasp (Safety Hover)
79     pre_grasp = best_grasp_world.copy()
80     pre_grasp[2, 3] += 0.10
81     execute_pose_move(pre_grasp)
82
83     # Move to Grasp
84     execute_pose_move(best_grasp_world)

```

```

80     close_gripper()
81
82     # Lift Up
83     lift_pose = best_grasp_world.copy()
84     lift_pose[2, 3] += 0.20
85     execute_pose_move(lift_pose)
86
87     # 4. Execute Place
88     # Calculate Place Pose
89     place_pose = best_grasp_world.copy()
90     place_pose[0, 3] = target_center[0]
91     place_pose[1, 3] = target_center[1]
92     # Target Z for TCP = Target Surface Z + Offset + Padding
93     # Adding a small padding (1cm) to ensure we don't collide hard,
94     # but close enough to drop safely.
95     place_pose[2, 3] = target_top_z + tcp_to_bottom_offset + 0.015
96
97     # Move to Pre-Place (Safety Hover)
98     pre_place = place_pose.copy()
99     pre_place[2, 3] += 0.15
100    execute_pose_move(pre_place)
101
102    # Move to Place
103    execute_pose_move(place_pose)
104    open_gripper()
105
106    # Move to Post-Place (Lift Up)
107    execute_pose_move(pre_place)
108
109
110    # --- Main Execution ---
111
112    # 1. Reset
113    open_gripper()
114
115    # 2. Perception & Strategy: Stack Blue Cube on Yellow Cube
116    obs = get_observation()
117    rgb = obs["robot0_robotview"]["images"]["rgb"]
118    depth = obs["robot0_robotview"]["images"]["depth"]
119    intrinsics = obs["robot0_robotview"]["intrinsics"]
120    cam_pose = obs["robot0_robotview"]["pose_mat"]
121
122    # Find Yellow Cube (Base)
123    masks_yellow = segment_sam3_text_prompt(rgb, "yellow cube")
124    yellow_data = get_best_mask(masks_yellow)
125
126    # Find Blue Cube (Object 1)
127    masks_blue = segment_sam3_text_prompt(rgb, "blue cube")
128    blue_data = get_best_mask(masks_blue)
129
130    if yellow_data is not None and blue_data is not None:
131        # Get base metrics
132        yellow_center, yellow_top, _ = get_object_metrics(yellow_data["mask"], depth,
133                                                         intrinsics, cam_pose)
134
135        if yellow_center is not None:
136            print("Stacking Blue Cube on Yellow Cube...")
137            pick_and_place(blue_data["mask"], yellow_center, yellow_top, depth, intrinsics,
138                           cam_pose)
139        else:
140            print("Could not compute metrics for Yellow Cube")
141    else:
142        print("Could not find Blue or Yellow cube")
143
144    # 3. Perception & Strategy: Stack Ball on Blue Cube
145    # Re-observe because the blue cube has moved
146    obs = get_observation()
147    rgb = obs["robot0_robotview"]["images"]["rgb"]
148    depth = obs["robot0_robotview"]["images"]["depth"]
149    intrinsics = obs["robot0_robotview"]["intrinsics"]
150    cam_pose = obs["robot0_robotview"]["pose_mat"]

```

```

151 # Find Blue Cube again (New Base)
152 masks_blue_new = segment_sam3_text_prompt(rgb, "blue cube")
153 blue_data_new = get_best_mask(masks_blue_new)
154
155 # Find Tennis Ball (Object 2)
156 masks_ball = segment_sam3_text_prompt(rgb, "yellow tennis ball")
157 ball_data = get_best_mask(masks_ball)
158
159 if blue_data_new is not None and ball_data is not None:
160     # Get base metrics (blue cube is now the base)
161     blue_center_new, blue_top_new, _ = get_object_metrics(blue_data_new["mask"], depth,
162                                                            intrinsics, cam_pose)
163
164     if blue_center_new is not None:
165         print("Stacking Tennis Ball on Blue Cube...")
166         pick_and_place(ball_data["mask"], blue_center_new, blue_top_new, depth,
167                        intrinsics, cam_pose)
168     else:
169         print("Could not compute metrics for Blue Cube (new base)")
170 else:
171     print("Could not find Blue Cube or Tennis Ball for second stack")

```

I.2. Place the Blue Cube on Top of the Yellow Cube

Generated by Gemini-3-Pro as the coding agent and Gemini-3-Pro as the VDM agent.

```

1 import numpy as np
2
3 # --- Helper functions ---
4
5 def get_object_centroid(obs, text_prompt):
6     """
7     Finds the 3D centroid of an object specified by a text prompt.
8     Returns the centroid (x, y, z) in world frame and the mask.
9     """
10    rgb = obs["robot0_robotview"]["images"]["rgb"]
11    depth = obs["robot0_robotview"]["images"]["depth"]
12    intrinsics = obs["robot0_robotview"]["intrinsics"]
13    extrinsics = obs["robot0_robotview"]["pose_mat"]
14
15    # Segment object
16    results = segment_sam3_text_prompt(rgb, text_prompt)
17    if not results:
18        print(f"Object '{text_prompt}' not found.")
19        return None, None
20
21    best_result = max(results, key=lambda x: x["score"])
22    mask = best_result["mask"]
23
24    # Get 3D points
25    points = mask_to_world_points(mask, depth, intrinsics, extrinsics)
26    if points.shape[0] == 0:
27        print(f"No valid depth points for '{text_prompt}'.")
28        return None, None
29
30    centroid = np.mean(points, axis=0)
31    return centroid, mask
32
33 def pick_object(obs, text_prompt, z_offset=0.0):
34     """
35     Standard pick routine:
36     1. Segment object
37     2. Plan grasp
38     3. Execute pick trajectory (pre-grasp -> grasp -> close -> lift)
39     """
40    rgb = obs["robot0_robotview"]["images"]["rgb"]
41    depth = obs["robot0_robotview"]["images"]["depth"]
42    intrinsics = obs["robot0_robotview"]["intrinsics"]
43    extrinsics = obs["robot0_robotview"]["pose_mat"]
44
45    # 1. Segment
46    results = segment_sam3_text_prompt(rgb, text_prompt)

```

```

47 if not results:
48     print(f"Could not find {text_prompt} to pick.")
49     return False
50
51 # Get mask with highest score
52 best_res = max(results, key=lambda x: x["score"])
53 mask = best_res["mask"]
54
55 # 2. Plan Grasp
56 # Note: depth needs to be (H, W) or (H, W, 1) - API doc says (H, W) usually
57 # preferred for planner wrappers,
58 # but the function signature accepts (H, W, 1).
59 if depth.ndim == 3 and depth.shape[2] == 1:
60     depth_map = depth[:, :, 0]
61 else:
62     depth_map = depth
63
64 grasp_poses, grasp_scores = plan_grasp(depth_map, intrinsics, mask)
65
66 if len(grasp_scores) == 0:
67     print(f"No grasps found for {text_prompt}.")
68     return False
69
70 # Select best top-down grasp
71 best_pose, best_score = select_top_down_grasp(grasp_poses, grasp_scores, extrinsics)
72
73 if best_pose is None:
74     print("No valid top-down grasp found, falling back to highest score.")
75     best_idx = np.argmax(grasp_scores)
76     best_pose_cam = grasp_poses[best_idx]
77     best_pose = extrinsics @ best_pose_cam # Convert to world frame
78
79 # Decompose grasp pose
80 grasp_pos, grasp_quat = decompose_transform(best_pose)
81
82 # 3. Execute Pick
83 # Pre-grasp (hover 10cm above)
84 pre_grasp_pos = grasp_pos + np.array([0, 0, 0.1])
85
86 # Move to pre-grasp
87 joints = solve_ik(pre_grasp_pos, grasp_quat)
88 move_to_joints(joints)
89
90 # Open gripper
91 open_gripper()
92
93 # Move to grasp
94 joints = solve_ik(grasp_pos, grasp_quat)
95 move_to_joints(joints)
96
97 # Close gripper
98 close_gripper()
99
100 # Lift up (20cm)
101 lift_pos = grasp_pos + np.array([0, 0, 0.2])
102 joints = solve_ik(lift_pos, grasp_quat)
103 move_to_joints(joints)
104
105 return True
106
107 def place_at_position(position, height_offset=0.05):
108     """
109     Place currently held object at a specific world position.
110     Target orientation is usually top-down (gripper pointing down).
111     """
112     # Standard top-down orientation (gripper pointing -z)
113     # A common quaternion for top-down is pointing down z-axis.
114     # Let's assume the current grasp orientation is maintained or we define a fixed top-
115     # down.
116     # For simplicity, we often maintain the orientation we picked with, or reset to a
117     # known neutral top-down.
118     # Here, we will define a fixed top-down orientation [0, 1, 0, 0] (x-axis rotation

```

```

116 180 deg) or similar.
117 # However, to be safe, let's just use the current robot configuration's orientation
118 # or a hardcoded one.
119 # Often [0, 1, 0, 0] is top down for Panda.
120 place_quat = np.array([0.0, 1.0, 0.0, 0.0])
121
122 target_pos = position + np.array([0, 0, height_offset])
123
124 # Move over target (high)
125 hover_pos = target_pos + np.array([0, 0, 0.15])
126 joints = solve_ik(hover_pos, place_quat)
127 move_to_joints(joints)
128
129 # Move down to place
130 joints = solve_ik(target_pos, place_quat)
131 move_to_joints(joints)
132
133 # Open gripper
134 open_gripper()
135
136 # Move back up
137 joints = solve_ik(hover_pos, place_quat)
138 move_to_joints(joints)
139
140 def main():
141     obs = get_observation()
142
143     # --- Strategy ---
144     # The stack is Green (top) -> Yellow (mid) -> Blue (bottom).
145     # Goal: Blue on Yellow.
146     # 1. Move Green to the side.
147     # 2. Move Yellow to another spot (target base).
148     # 3. Pick Blue.
149     # 4. Place Blue on top of Yellow.
150
151     # Defined drop zones relative to the table center or just hardcoded offsets based
152     # on workspace knowledge.
153     # Let's dynamically find a spot by looking at the initial stack position.
154
155     # Find the stack location (using the green cube at the top)
156     green_center, _ = get_object_centroid(obs, "green cube")
157     if green_center is None: return
158
159     # Define temporary drop zones relative to the stack
160     # Drop zone 1 (for green): 20cm to the left
161     drop_zone_green = green_center + np.array([-0.2, 0.0, -green_center[2] + 0.02]) # Z
162     # is roughly table height
163     # Drop zone 2 (for yellow): 20cm to the right (this will be the final destination
164     # for yellow)
165     drop_zone_yellow = green_center + np.array([0.2, 0.0, -green_center[2] + 0.02])
166
167     # --- Step 1: Remove Green Cube ---
168     print("Moving Green Cube...")
169     success = pick_object(obs, "green cube")
170     if success:
171         place_at_position(drop_zone_green, height_offset=0.05) # Place on table
172     else:
173         print("Failed to move green cube.")
174         return
175
176     # Refresh observation
177     obs = get_observation()
178
179     # --- Step 2: Move Yellow Cube ---
180     # Now Yellow should be exposed.
181     print("Moving Yellow Cube...")
182     success = pick_object(obs, "yellow cube")
183     if success:
184         # We place it at drop_zone_yellow. This is where we want to stack the blue one
185         # later.
186         place_at_position(drop_zone_yellow, height_offset=0.05)
187     else:
188         print("Failed to move yellow cube.")

```

```

183         return
184
185     # Refresh observation to find the new position of the yellow cube and the exposed
186     # blue cube
187     obs = get_observation()
188
189     # Get precise location of yellow cube now that it's moved
190     yellow_center, _ = get_object_centroid(obs, "yellow cube")
191     if yellow_center is None:
192         print("Lost track of yellow cube.")
193         return
194
195     # --- Step 3: Pick Blue Cube ---
196     print("Picking Blue Cube...")
197     success = pick_object(obs, "blue cube")
198     if not success:
199         print("Failed to pick blue cube.")
200         return
201
202     # --- Step 4: Place Blue on Yellow ---
203     print("Placing Blue on Yellow...")
204     # Target is yellow center, but offset Z by cube height (approx 5cm usually for
205     # these cubes)
206     # We add a small buffer.
207     cube_height_approx = 0.05
208     place_at_position(yellow_center, height_offset=cube_height_approx + 0.02)
209
210     # Done, gripper is already opened in place_at_position
211     print("Task completed.")
212
213 main()

```

I.3. Additional Analysis Plots

We perform additional quantitative analysis on the cube stack task to understand the impact of multi-turn on success rate. We focus our analysis on Gemini 3 Pro (Google DeepMind, 2025), GPT 5.2 (OpenAI, 2025b), and Claude Opus 4.5 (Anthropic, 2025b), the three strongest performers on CaP-Bench. In particular, we plot the probability density function (PDF) of success over turns (Figure 24), PDF of success over tokens (Figure 25), multi-turn success rate vs turn for the cube stacking task in Figure 26. We find that for the cube stacking task, these models would perform the task on the first try, and use the subsequent turns for recovery. Successes for GPT 5.2 and Claude Opus 4.5 seem to have shorter code length than that of Gemini 3 Pro.

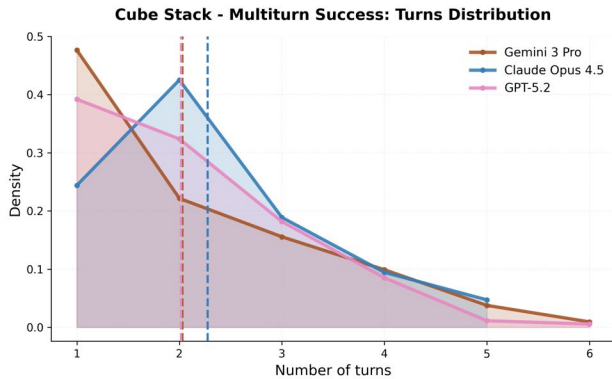


Figure 24. Distribution of multi-turn cube stack successes over turns. Results are averaged across M1, M2, and M3.

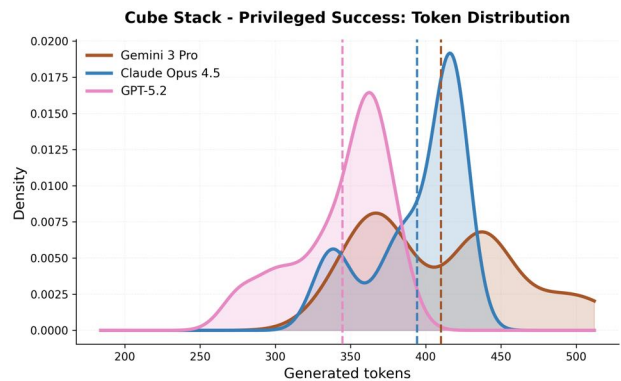


Figure 25. Distribution of cube stack successes (S1) over tokens

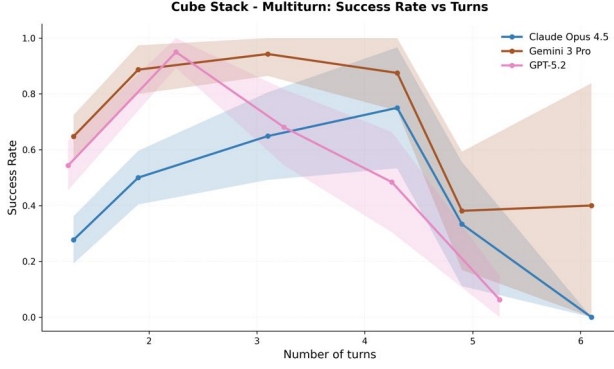


Figure 26. Cube stack multi-turn success rate vs. turns. Trials with too few or too many turns have lower success rate. Results are averaged across M1, M2, and M3.

J. LIBERO-PRO Evaluation Results

We present detailed task-wise performance of OpenVLA, π_0 , $\pi_{0.5}$, and CaP-Agent0 on LIBERO-PRO, which evaluates model generalization under initial position perturbations (Pos) and instruction perturbations (Task). Results are summarized in Table 6, Table 7 and Table 8. Each task was executed over 50 trials. In these tasks, CaP-Agent0 struggles with failures in perception, grasp generation, control APIs. For example, queries of "alphabet soup can" to SAM 3 often results in segmentations of the "tomato sauce can" also present in the scene. Also due to the occluded nature of these scenes and the camera not being top down, many grasps generated on desired objects will be at an angle, this, when combined with the lack of collision-aware motion planning often results in other objects in the scene being knocked over during execution, preventing a secure grasp on the desired object.

Table 6. Task-wise LIBERO-PRO performance of OpenVLA, π_0 , $\pi_{0.5}$ and CaP-Agent0 on the **libero-object** benchmark. **Action notation:** $Place(obj, loc)$ = pick up object obj and place obj into/onto target loc .

Task (Symbolic Form)	OpenVLA		π_0		$\pi_{0.5}$		CaP-Agent0	
	Pos	Task	Pos	Task	Pos	Task	Pos	Task
$Place(\text{alphabet_soup}, \text{basket})$	0.00	0.00	0.00	0.00	0.00	0.00	0.02	0.04
$Place(\text{bbq_sauce}, \text{basket})$	0.00	0.00	0.00	0.00	1.00	0.02	0.12	0.42
$Place(\text{butter}, \text{basket})$	0.00	0.00	0.00	0.00	0.54	0.00	0.26	0.18
$Place(\text{chocolate_pudding}, \text{basket})$	0.00	0.00	0.00	0.00	0.00	0.02	0.18	0.48
$Place(\text{cream_cheese}, \text{basket})$	0.00	0.00	0.10	0.00	0.00	0.00	0.12	0.06
$Place(\text{ketchup}, \text{basket})$	0.00	0.00	0.00	0.00	0.20	0.02	0.32	0.12
$Place(\text{milk}, \text{basket})$	0.00	0.00	0.00	0.00	0.00	0.00	0.38	0.02
$Place(\text{orange_juice}, \text{basket})$	0.00	0.00	0.00	0.00	0.00	0.02	0.30	0.02
$Place(\text{salad_dressing}, \text{basket})$	0.00	0.00	0.10	0.00	0.00	0.00	0.32	0.00
$Place(\text{tomato_sauce}, \text{basket})$	0.00	0.00	0.00	0.00	0.00	0.00	0.16	0.48
Average	0.00	0.00	0.00	0.00	0.17	0.01	0.218	0.182

Table 7. Task-wise LIBERO-PRO performance of OpenVLA, π_0 , $\pi_{0.5}$ and CaP-Agent0 on the **libero-goal** benchmark. **Action notation:** $Open(x, y)$ = open target y of container x ; $Put(obj, loc)$ = place object obj onto/into location loc ; $Push(obj, loc)$ = push object obj toward location loc ; $TurnOn(obj)$ = activate object obj .

Task (Symbolic Form)	OpenVLA		π_0		$\pi_{0.5}$		CaP-Agent0	
	Pos	Task	Pos	Task	Pos	Task	Pos	Task
$Open(cabinet, drawer_{mid})$	0.00	0.00	0.00	0.00	0.00	0.04	0.00	0.00
$Put(bowl, drawer_{top})$	0.00	0.00	0.00	0.00	0.94	0.02	0.04	0.00
$Push(plate, stove_{front})$	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.10
$Put(bowl, plate)$	0.00	0.00	0.00	0.00	0.00	0.02	0.36	0.38
$Put(bowl, stove)$	0.00	0.00	0.00	0.00	0.00	0.04	0.22	0.12
$Put(bowl, cabinet_{top})$	0.00	0.00	0.00	0.00	0.00	0.02	0.60	0.04
$Put(cream_cheese, bowl)$	0.00	0.00	0.00	0.00	0.98	0.02	0.04	0.34
$Put(wine.bottle, rack)$	0.00	0.00	0.00	0.00	0.88	0.02	0.02	0.12
$Put(wine.bottle, cabinet_{top})$	0.00	0.00	0.00	0.00	0.98	0.02	0.62	0.40
$TurnOn(stove)$	0.00	0.00	0.00	0.00	0.00	0.00	0.66	0.18
Average	0.00	0.00	0.00	0.00	0.38	0.00	0.256	0.168

Table 8. Task-wise LIBERO-PRO performance of OpenVLA, π_0 , $\pi_{0.5}$ and CaP-Agent0 on the **libero-spatial** benchmark. **Action notation:** $Pick(src, dst)$ = pick up object $bowl_{black}$ from location src and place $bowl_{black}$ onto/into target dst .

Task (Symbolic Form)	OpenVLA		π_0		$\pi_{0.5}$		CaP-Agent0	
	Pos	Task	Pos	Task	Pos	Task	Pos	Task
$Pick(between(plate, ramekin), plate)$	0.00	0.00	0.00	0.00	0.02	0.00	0.22	0.14
$Pick(table.center, plate)$	0.00	0.00	0.00	0.00	0.00	0.02	0.22	0.14
$Pick(drawer_{top}(cabinet_{wood}), plate)$	0.00	0.00	0.00	0.00	0.00	0.00	0.02	0.10
$Pick(next.to(cookie.box), plate)$	0.00	0.00	0.00	0.00	0.00	0.02	0.00	0.10
$Pick(next.to(plate), plate)$	0.00	0.00	0.00	0.00	0.00	0.00	0.10	0.20
$Pick(next.to(ramekin), plate)$	0.00	0.00	0.00	0.00	0.12	0.02	0.30	0.14
$Pick(on(cookie.box), plate)$	0.00	0.00	0.00	0.00	0.00	0.00	0.14	0.08
$Pick(on(ramekin), plate)$	0.00	0.00	0.00	0.00	0.98	0.02	0.02	0.20
$Pick(on(stove), plate)$	0.00	0.00	0.00	0.00	0.02	0.00	0.08	0.14
$Pick(on(cabinet_{wood}), plate)$	0.00	0.00	0.00	0.00	0.90	0.00	0.08	0.16
Average	0.00	0.00	0.00	0.00	0.20	0.01	0.118	0.14

K. Additional Clarifications

This section consolidates clarifications and added details requested by reviewers during the discussion period. It expands the protocol behind the human-expert baseline, the computational cost of CaP-Agent0, and the choice of VDM backbone.

K.1. Human Expert Baseline Protocol

The human-expert curve in Figure 1 is *not* a single-shot human attempt. The baseline was written by a subset of the paper authors ($N=7$), each with 2+ years of robotics programming experience. For each task and each tier, an author wrote a single Python script using exactly the same API primitives available to the model at that tier, and iterated through normal trial-and-error—reading execution traces, fixing bugs, and updating the script—until the program achieved high reliability. Human-written code uses the same primitives as the model; what differs is the iterative refinement loop performed offline by a human.

Resulting upper bound. The iterated human reference achieves 88.5% average success on single-turn tiers, which we treat as a near-upper-bound for what is achievable when a robotics engineer hand-writes static code with full access to development-time iteration.

Human effort budget. The effort required to reach this near-upper-bound varies sharply with task complexity:

- **Simple pick-and-place tasks** (Cube Lift, Cube Stack): arrived at a working solution in a few hours (<1 day), with both low-level primitive and high-level code implementations, hill-climbed with a few rounds of trial-and-error.
- **Contact-rich and bimanual tasks** (Peg Insertion, Two-Arm Handover): identifying the right primitives and the right overall strategy took 2–3 weeks of iterative development per task. This estimate also includes the overhead of comparing alternative primitives and updating implementations as the underlying toolchain evolved (e.g., the Molmo → Molmo2 transition).

Comparison to multi-turn agent recovery. In contrast, CaP-Agent0’s multi-turn recovery is fast at evaluation time: a complete evaluation of CaP-Agent0 on a Robosuite task takes approximately 2 minutes per trial (see Appendix K.2). The intended comparison is therefore between *static, single-turn code iterated on extensively by an experienced human* and an *interactive agent capable of online monitoring and recovery at deployment time*—which is precisely the practical gap that CaP-Agent0 is designed to address.

K.2. Computational Cost Analysis

We report code-generation time and total trial time (LLM + execution) averaged over $N=20$ trials of cube stacking using Gemini-3-Pro. All measurements were collected on the same hardware with identical perception and control stacks.

Tier	Avg LLM Code-Gen Time	Avg Trial Time
S1 (privileged)	6.8 s	12.9 s
S2 (non-privileged)	9.6 s	20.0 s
S3 (reduced API)	23.8 s	28.4 s
M1 (multi-turn, no VDM)	20.1 s	60.8 s
M3 (multi-turn + VDM)	15.8 s	113.6 s

Table 9. Per-trial timing breakdown across CaP-Bench tiers (Gemini-3-Pro, cube stack, $N=20$).

Comparison to VLAs. For reference, π_0 (Black et al., 2024) reports ~ 73 ms per action chunk and OpenVLA (Kim et al., 2024) reports ~ 167 ms per action. VLAs are substantially faster per inference step because they emit single motor commands at high frequency. CaP-Agent0 operates at a different level of abstraction: each code-generation iteration (6.8–23.8 s) produces an *entire manipulation sequence*, not a single action. Direct latency comparison is therefore misleading; the relevant axis is *cost per task attempt* rather than *cost per action*.

K.3. Visual Differencing Module: Model Choice

The Visual Differencing Module (VDM) used throughout the main paper is implemented with Gemini-3-Pro (Google DeepMind, 2025), which at the time of writing was the state-of-the-art on major multimodal benchmarks (MMMU-Pro: 81%; Video-MMMU: 87.6%). We choose the strongest available VLM to upper-bound the effect of language-grounded perception, ensuring that the observed performance gains in M3/M4 reflect the value of the *mechanism* (text grounding via an auxiliary VLM) rather than weakness of an under-powered captioner.

The VDM is *model-agnostic by design*: swapping the backbone requires a single configuration change in CaP-Gym, with no modifications to the coding-agent loop or to the rest of the benchmark. A systematic ablation across VDM backbones is an orthogonal axis of study that CaP-Bench supports out of the box and that we encourage the community to explore; the contribution of this paper is to demonstrate that VDM, as a *method*, materially helps multi-turn recovery.